Chương 1. TỔNG QUAN VỀ MÁY TÍNH ĐIỆN TỬ

1.1 Giới thiệu

1.1.1 Lịch sử phát triển

1.1.1.1 Máy tính cơ khí

Từ xa xưa con người đã luôn phải tính toán. Để nhớ số gia súc trong đàn, người ta đã đếm chúng và dùng sợi dây có các gút các nút. Để tính toán người Trung Hoa đã sử dụng bàn tính… Nhà khoa học thiên tài người Pháp Blaise Pascal năm 19 tuổi (năm 1642) vì thương cha (cha của ông vốn là một nhân viên thu thuế) đã xây dựng một máy đầu tiên thực hiện công việc tính toán. Đây là thiết bị hoàn toàn bằng cơ khí sử dụng các bánh răng. Máy tính của Blaise Pascal chỉ thực hiện được phép toán cộng bằng cách ấn vào phím các con số và dùng một tay quay. Năm 1671 nhà toán học người Đức Baron Gottfried Wilherm von Leibniz xây dựng một máy cơ khí và hoàn thiện vào năm 1694. Chiếc máy này không những thực hiện phép tính cộng mà còn thực hiện được phép tính nhân bằng cách cộng và lần lược chuyển số. Năm 1820 Chales Xavier Thomas đã phát triển chiếc máy tính cơ thành công về mặt thương mại đầu tiên thực hiện được bốn phép tính cộng trừ nhân chia.

Charles Babbage, giáo sư toán học trường Cambridge của Anh được nhiều người gọi là cha đẻ của Computer nhờ hai phát kiến vĩ đại của ông về 2 kiểu máy tính cơ khác nhau. Ông đã thiết kế và xây dựng máy sai phân (difference engine). Nó giải được phương trình đa thức bằng phương pháp sai phân. Năm 1834, Babbage thiết kế và xây dựng máy phân tích (analytical engine). Máy phân tích có các thành phần cơ bản của máy tính hiện đại: Thành phần nhập (đầu đọc thẻ đục lỗ), bộ điều khiển dùng để điều khiển hay lập trình cho bộ xử lý. Bộ xử lý (hay tính toán – calculator). Bộ lưu trữ (bộ nhớ), Thành phần xuất (in và đục lỗ). Bộ tính toán có thể nhận các toán hạng từ bộ lưu trữ, thực hiện phép toán cộng, trừ, nhân hay chia chúng và trả kết quả về bộ lưu trữ. Tuy nhiên chiếc máy này của Ông không thể hoàn thành vì kỹ thuật chế tạo thời đó không cho phép. Phát triển tiếp theo của máy phân tích là máy đa năng. Máy đọc lệnh từ các thẻ đục lỗ và thực thi chúng. Bằng cách đục lỗ một chương trình khác trên thẻ nhập, máy phân tích có khả năng thực hiện các tính toán khác. Lập trình viên máy tính đầu tiên là Ada Lovelace đã tạo ra

phần mềm cho máy phân tích.

Vào những năm 1930, Konrad Zuse xây dựng một chuỗi các máy tính toán tự động bằng cách sử dụng các rơle từ. Sau đó, John Atanasoff và George Stibbitz đã thiết kế các máy tính (calculator). Máy của Atanasoff sử dụng số nhị phân và có các tụ điện làm cho bộ nhớ được làm tươi theo chu kỳ. Tuy nhiên, máy này bị thất bại do công nghệ phần cứng không tương xứng với ý tưởng thiết kế.

Năm 1944, Aiken hoàn tất máy tính Mark 1, có tất cả 72 từ, mỗi từ 23 số thập phân và có thời gian một chu kỳ là 6 giây. Việc nhập và xuất thực hiện bằng các băng giấy đục lỗ.

1.1.1.2 Máy tính dùng đèn điện tử - thế hệ thứ nhất

Năm 1943, máy tính số điện tử đầu tiên trên thế giới bắt đầu hoạt động, máy Colossus. Colossus do Alan Turing thiết kế nhằm thực hiện giải mã các thông điệp đã mã hóa trong chiến tranh thế giới thứ 2. Cũng trong năm 1943, Mauchley và Presper Eckert bắt đầu tiến hành xây dựng máy tính ENIAC (Electronic Numerical Integrator And Computer). ENIAC gồm 1800 đèn điện tử và 1500 rơle, cân nặng 30 tấn, công suất tiêu thụ 140 KWh. Nó có tất cả 20 thanh ghi, mỗi thanh ghi có thể lưu trữ một số thập phân 10 chữ số.

Sau đó, John Von Neumann thiết kế máy IAS dựa cơ sở trên máy EDVAC, là một phiên bản nâng cao của ENIAC. Máy von Neumman có 5 phần cơ bản: bộ nhớ, đơn vị số học và logic (ALU – Arithmetich Logic Unit), đơn vị điều khiển chương trình, thiết bị nhập và thiết bị xuất. Bộ nhớ có tất cả 4096 từ, mỗi từ lưu trữ 40 bit. Mỗi từ chứa 2 lệnh 20 bit hay một số nguyên có dấu 39 bit. Mỗi lệnh 20 bit gồm có 8 bit xác định loại lệnh và 12 bit xác định 1 trong 4096 từ nhớ.

Vào cùng thời gian của máy IAS, các nhà nghiên cứu ở MIT cũng đang xây dựng một máy tính, máy Whirlwind 1. Nó có chiều dài từ là 16 bit và được thiết kế để có thể điều khiển với thời gian thực.

1.1.1.3 Máy tính transistor – thế hệ thứ hai

Năm 1948, John Bardeen, Walter Brattain và William Shockley phát minh ra transistor đã làm cuộc cách mạng trong lĩnh vực máy tính. Máy tính transistor đầu tiên được xây dựng tại MIT, máy TX-0 (Transistorized experimental computer 0),

có 16 bit, tương tự như Whirlwind 1.

Năm 1961, máy tính PDP-1 xuất hiện có 4KB bộ nhớ, chiều dài từ là 18 bit và khoảng thời gian của một chu kỳ là 5 μs. Vài năm sau, PDP-8 ra đời có 12 bit nhưng giá thành rẻ hơn PDP-1 rất nhiều (16.000 USD so với 120.000 USD). PDP-8 có một đổi mới đó là hình thành một bus đơn gọi là omnibus trong đó bus là tập hợp các dây nối song song dùng để kết nối các thành phần của máy tính.

Trong khi đó, IBM xây dựng một phiên bản của 709 bằng transistor, đó là máy tính 7094 có thời gian một chu kỳ là 2 μs và bộ nhớ 32KB và chiều dài từ là 36 bit. Năm 1964, công ty CDC giới thiệu máy 6600 có tốc độ nhanh hơn 7094 do bên trong CPU có một cơ chế song song. CPU có vài đơn vị thực hiện phép cộng, các đơn vị khác thực hiện phép nhân, phép chia và tất cả chúng đều hoạt động song song. Với một công việc, máy có khả năng thực thi 10 lệnh đồng thời.

1.1.1.4 Máy tính IC – thế hệ thứ ba

Vi mạch (IC – Integrated Circuit) được phát minh cho phép đặt vài chục transistor trong một chip đơn. Việc này giúp cho các máy tính xây dựng trên IC trở nên nhỏ hơn, nhanh hơn và rẻ hơn so với các máy tính transistor. Lúc này, IBM giới thiệu một sản phẩm đơn, máy System 360, được thiết kế dựa trên các vi mạch. Đổi mới quan trọng trong 360 là khả năng đa lập trình (multiprogramming), có vài chương trình trong bộ nhớ đồng thời để khi một chương trình đang chờ xuất/nhập dữ liệu thì chương trình khác có thể tính toán. Một đặc trưng khác của 360 là không gian địa chỉ lớn (thời điểm lúc đó), với 2^{24} byte nhớ (16 MB).

1.1.1.5 Máy tính cá nhân và VLSI – thế hệ thứ tư

Vào thập niên 80, vi mạch VLSI (Very Large Scale Integrate) có khả năng chứa vài chục ngàn, vài trăm ngàn và vài triệu transistor trên một chip đơn đã được chế tạo. Sự phát triển này dẫn đến việc sản xuất các máy tính nhỏ hơn và nhanh hơn. Do đó, giá cả đã giảm xuống đến mức một cá nhân có thể sở hữu một máy tính. Các máy tính cá nhân thường dùng cho việc xử lý từ, các bảng tính và các ứng dụng tương hỗ khác.

Quy luật Moore: *Năm 1965 Gordon Moore phát hiện ra một quy luật quan trọng trong xu hướng phát triển của máy tính. Ông nhận thấy trong*

vòng 18 đến 24 tháng thì năng lực tính toán, lưu trữ...của máy tính sẽ tăng gấp 2 lần. Trong 26 năm con số tranzitor trong 1 con chip đã tăng hơn 3.200 lần (từ 2.300 tranzitor ở chip 4004 năm 1971 tăng lên hơn 7,5 triệu tranzitor trên Pentium II). Quy luật này trở thành cơ sở cho rất nhiều dự báo về hiệu suất của ngành công nghiệp máy tính

1.1.2 Phân loại máy tính

Máy tính ngày nay có thể chia thành 5 loại: Siêu máy tính, Mainframe, siêu máy tính mini, máy tính mini, máy tính cá nhân.

Máy tính mini sử dụng trong các ứng dụng thời gian thực như điều khiển không lưu hay tự động hóa. Siêu máy tính mini dùng trong các hệ thống chia sẻ thời gian, các máy chủ. Mainframe dùng trong các nhóm công việc lớn hay đòi hỏi cơ sở dữ liệu lớn, … Siêu máy tính được thiết kế đặc biệt để cựa đại hóa số các thao tác dấu chấm động trong 1s (FLOP – floating point operations per second). Máy tính nào có tốc độ dưới 1 GF/s thì không được xem là siêu máy tính.

Máy tính cá nhân có thể chia ra làm 3 loại: Máy tính để bàn, máy tính xách tay và máy tính bỏ túi.

1.2 Tổ chức hệ thống máy tính

1.2.1 Sơ đồ cấu trúc chung

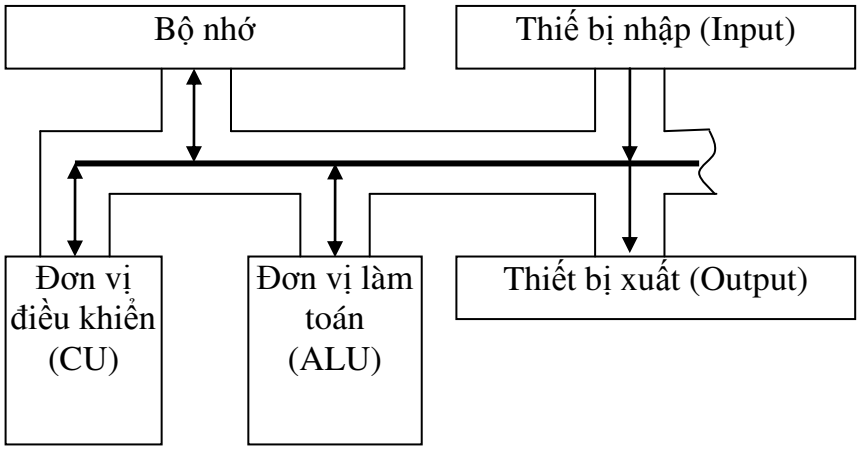
1.2.1.1 Nguyên lý máy Von Neumann

Vào những năm của thập niên 40 của thế kỷ 20, máy tính điện tử số bắt đầu được sử dụng. Có 2 dòng máy là máy Turing và máy Von Neumann.

Đại diện cho máy Turing có máy Colossus. Máy này do Alan Turing thiết kế nhằm thực hiện việc giải mã các thông điệp đã mã hóa trong chiến tranh thế giới thứ 2. Vào năm 1943, Mauchley và Presper Eckert bắt đầu tiến hành xây dựng máy tính ENIAC (Electronic Numerical Integrator And Computer). ENIAC gồm 1800 đèn điện tử và 1500 rơle, cân nặng 30 tấn, công suất tiêu thụ 140 KWh. Nó có tất cả 20 thanh ghi, mỗi thanh ghi có thể lưu trữ một số thập phân 10 chữ số.

Dòng máy tính Von Neumann được kế thừa và phát triển ở hầu hết các thế hệ máy tính sau này. Đại diện cho máy Von neumann thời đó có máy IAS dựa cơ sở trên máy EDVAC, là một phiên bản nâng cao của ENIAC. Máy IAS có bộ nhớ gồm

4096 word, mỗi word lưu trữ 40 bit. Mỗi word chứa 2 lệnh 20 bit hay một số nguyên có dấu 39 bit. Mỗi lệnh 20 bit gồm có 8 bit xác định loại lệnh và 12 bit xác định 1 trong 4096 word nhớ. Máy Von Neumman có 5 phần cơ bản: bộ nhớ, bộ số học và logic (ALU – Arithmetich Logic Unit), đơn vị điều khiển chương trình, thiết bị nhập và thiết bị xuất. Hình 1.1 là sơ đồ cấu trúc của máy Von Neumann.



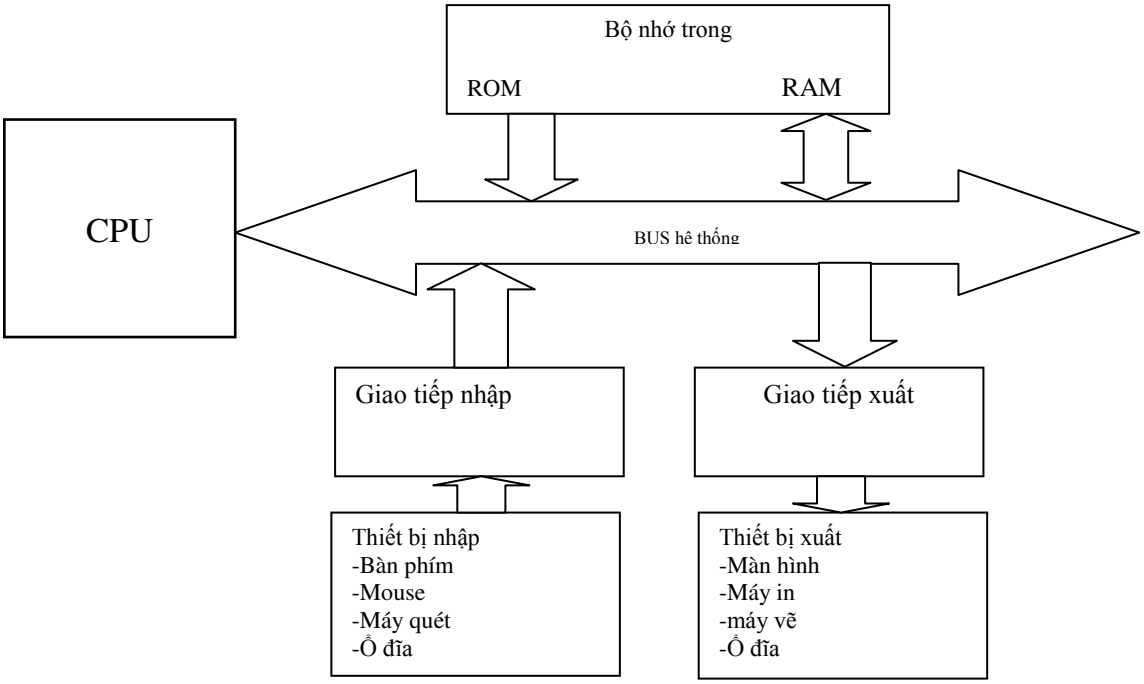
Hình 1.1: Máy Von Neumman

Nguyên tắc hoạt động của máy Von neumann:

- Chương trình điều khiển và xử lý dữ liệu cũng được xem là Data và được lưu trữ trong bộ nhớ và được xem là chương trình lưu trữ.
- Bộ nhớ chia làm nhiều ô, mỗi ô có 1 địa chỉ để có thể chọn lựa trong quá trình đọc/ghi dữ liệu (nguyên lý định địa chỉ)
- Các lệnh được thực hiện tuần tự nhờ vào bộ đếm chương trình (thanh ghi lệnh) nằm bên trong bộ xử lý.
- Chương trình có thể biểu diễn dưới dạng số và đặt vào trong bộ nhớ.

1.2.1.2 Sơ đồ tổ chức tổng quát của máy vi tính

Hình 1.2 là sơ đồ khối kiến trúc chung của một máy vi tính



Hình 1.2: Sơ đồ kiến trúc chung của máy vi tính

1.2.2 Chức năng cơ bản của các bộ phận

-CPU (Central Processing Unit): Là bộ não của máy tính bao gồm 2 thành phần đó là: bộ số học và logic (ALU) và bộ điều khiển (CU):

Bộ số học và logic (Arithmetic and logic unit – ALU):

Hầu hết các hoạt động tính toán cuả máy tính được thực hiện tại bộ số học và logic. Mọi phép toán số học và logic như: cộng; trừ; nhân; chia hay so sánh 2 số đều được thực hiện tại ALU. Không phải tất cả các toán hạng cần cho việc tính toán phải ở trong bộ nhớ chính. Vi xử lý có những phần tử nhớ tốc độ cao gọi là các thanh ghi (registers). Các thanh ghi dùng để lưu giữ những toán hạng được dùng thường xuyên trong khi tính toán. Mỗi thanh ghi có thể lưu giữ một word số liệu. Thời gian truy xuất thanh ghi là từ 5 đến 10 ns (nhanh hơn thời gian truy xuất bộ nhớ chính)

-Bộ điều khiển (control unit - CU):

Bộ điều khiển có chức năng thực hiện việc phối hợp hoạt động của tất cả các bộ phận khác như: Bộ số học và logic, bộ nhớ, các thiết bị ngoại vi…sao cho nhịp nhàng, đạt hiệu quả cao. Bộ điều khiển một mặt gửi các tín hiệu điều khiển đến các bộ phận, mặt khác nó luôn luôn theo dõi trạng thái của các bộ phận đó để đưa ra

những tín hiệu điều khiển hợp lý. Ví dụ việc chuyển số liệu giữa bộ nhớ và các thiết bị ngoại vi phải được điều khiển bởi bộ điều khiển. Bộ điều khiển tạo ra tín hiệu thời gian để các hành động của các bộ phận xảy ra đúng vào thời điểm mong đợi. Có thể nói rằng bộ điều khiển là một đơn vị độc lập và nó điều khiển tất cả các bộ phận khác trong máy. Trên thực tế, mạch điện cuả bộ điều khiển được phân bố đến khắp nơi trong máy. Một tập hợp các đường dây điều khiển (control bus) mang các tín hiệu thời gian. Các tín hiệu này dùng để đồng bộ hoạt động cuả tất cả các bộ phận trong máy.

ALU và bộ điều khiển thường là các thành phần có tốc độ hoạt động nhanh hơn các thành phần khác nối với máy tính. Điều này cho phép sử dụng chỉ một vi xử lý để điều khiển một số lớn các thiết bị ngoại vi có tốc độ hoạt động khác nhau như bộ nhớ ngoài, màn hình, máy in …

-Bộ nhớ (memory)

Chức năng cuả bộ nhớ là lưu giữ số liệu và chương trình. Có 2 loại bộ nhớ: bộ nhớ sơ cấp (bộ nhớ chính hay còn gọi là bộ nhớ trong - Primary Memory – Main Memory- Internal Memory) và bộ nhớ thứ cấp (bộ nhớ ngoài hay còn gọi là bộ nhớ phụ -Secondary Memory - Auxiliary Memory - External Memory).

Bộ nhớ sơ cấp hay bộ nhớ chính (Main memory) có tốc độ rất nhanh. Bộ nhớ chính bao gồm một số lớn các ô nhớ bán dẫn. Mỗi ô nhớ chứa được một bit thông tin. Việc đọc hoặc viết thông thường được tiến hành với 1 nhóm ô nhớ gọi là tứ nhớ hay word nhớ (memory word). Bộ nhớ chính được tổ chức sao cho nội dung của một word (n bit) có thể được truy xuất bằng lệnh đọc hoặc viết bộ nhớ. Số các bit trên một word nhớ gọi là độ dài word (word length) của máy tính. Độ dài word của các máy tính có thể là 8,16, 32 hoặc 64 bit.

Tổng số ô nhớ trên bộ nhớ gọi là dung lượng (capacity) bộ nhớ. Dung lượng bộ nhớ trong của PC có thể là vài MB đến hàng trăm MB. Hiện nay đã là TB.

Để có thể truy xuất tới mọi word nhớ trên bộ nhớ, mỗi word nhớ được gán một địa chỉ (Address) riêng biệt. Word nhớ trong bộ nhớ có thể được truy xuất bằng cách chỉ ra địa chỉ và lệnh đọc viết tương ứng. Số liệu được xử lý theo một word,

bội của word hoặc một phần của word. Khi bộ nhớ được truy xuất, thông thường một word của số liệu được đọc từ bộ nhớ ra hoặc được viết vào bộ nhớ.

Chương trình phải nằm trên bộ nhớ khi đang thi hành. Lệnh và số liệu có thể được viết vào bộ nhớ hay đọc ra dưới sự điều khiển của vi xử lý. Thời gian cần thiết để truy xuất tới một vị trí nhớ trong bộ nhớ gọi là thời gian truy xuất bộ nhớ (memory access time = Tacc). Các bộ nhớ có Tacc không phụ thuộc vị trí nhớ gọi là bộ nhớ truy xuất ngẫu nhiên (Random Access Memories = RAM). Tacc có giá trị vài chục ns (nano second) trong các máy tính hiện đại.

Mặc dù bộ nhớ chính rất quan trọng, nhưng giá thành lại quá cao. Vì vậy người ta dùng thêm bộ nhớ phụ (secondary memory) khi muốn lưu giữ một lượng lớn thông tin không sử dụng thường xuyên. Bộ nhớ phụ có thể là đĩa từ (magnetic disks) hay đĩa quang (Compact Disc), băng từ…

-Thiết bị nhập (input device)

Máy tính nhận thông tin đã được mã hóa thông qua thiết bị nhập. Quá trình này gọi là đọc số liệu. Thiết bị nhập thường dùng nhất là bàn phím (key board). Key board được nối sao cho khi một phím được nhấn, ký tự hoặc số tương ứng được tự động chuyển thành mã nhị phân và được gửi trực tiếp tới bộ nhớ hoặc vi xử lý. Có một số thiết bị nhập khác như: chuột (mouse), máy quét (scanner)...

-Thiết bị xuất (output device)

Thiết bị xuất làm việc theo nguyên tắc ngược lại với thiết bị nhập. Chức năng của thiết bị xuất là gửi các kết quả đã được xử lý ra ngoài. Một số thiết bị như màn hình, máy in (printer), máy vẽ…

Tóm lại, hoạt động tổng quát cuả một máy tính có thể tóm tắt như sau:

• Máy tính nhận thông tin dưới dạng chương trình và số liệu thông qua thiết bị nhập. Thông tin được lưu giữ trong bộ nhớ máy tính.

• Thông tin được lưu giữ trong bộ nhớ được lấy vào ALU dưới sự điều khiển cuả chương trình. Tại ALU, thông tin được xử lý.

• Thông tin đã được xử lý được đưa ra ngoài thông qua thiết bị xuất.

• Tất cả các hoạt động bên trong máy tính được chỉ dẫn bởi bộ điều khiển.

-***Bus hệ thống***: tập hợp các đường dây kim loại dẫn điện để CPU có thể liên kết với các bộ phận khác.

1.2.3 Quá trình thực hiện lệnh

1.2.3.1 Chu kỳ lệnh

Máy tính được điều khiển bởi các lệnh (instructions). Để thực hiện một nhiệm vụ cho trước, một chương trình tương ứng bao gồm một tập các lệnh sẽ được lưu giữ trong bộ nhớ chính. Các lệnh riêng rẽ sẽ được đọc từ bộ nhớ vào vi xử lý. Mỗi lệnh sẽ thực hiện một thao tác đã định sẵn. Số liệu lưu giữ trên bộ nhớ được sử dụng như các toán hạng (operands).

Việc chuyển số liệu giữa bộ nhớ chính và vi xử lý được bắt đầu bằng việc vi xử lý gửi địa chỉ của ô nhớ cần truy xuất và tín hiệu điều khiển hợp lý tới bộ nhớ. Số liệu sau đó sẽ được chuyển giữa bộ nhớ và vi xử lý một cách tương ứng.

Cùng với ALU và bộ điều khiển, vi xử lý còn có các thanh ghi dùng cho việc lưu trữ tạm thời số liệu. Thanh ghi lệnh (IR - Instruction Register) giữ lệnh đang được thực hiện. Bộ đếm chương trình (PC - Program Counter) chứa địa chỉ bộ nhớ của lệnh sẽ được thực hiện. Sau khi thực hiện lệnh, nội dung cuả thanh PC sẽ được cập nhật tới địa chỉ của lệnh tiếp theo. Bình thường nội dung cuả PC sẽ tăng lên 1 nếu chương trình không bị một lệnh nhảy hay lệnh ngắt nào khác.

1.2.3.2 Tiến trình xử lý lệnh

Có hai thanh ghi dùng để giao tiếp với bộ nhớ là: Thanh ghi địa chỉ bộ nhớ (MAR Memory Address Register) và thanh ghi số liệu bộ nhớ (MDR Memory Data Register). MAR chứa địa chỉ cuả ô nhớ cần truy xuất, còn MDR giữ nội dung của số liệu cần chuyển (giữa Vi xử lý và bộ nhớ). Các bước hoạt động cơ bản khi thực hiện 1 chương trình như sau: Chương trình chứa trong bộ nhớ chính. Việc thực thi chương trình bắt đầu khi PC chỉ tới vị trí nhớ mà lệnh đầu tiên của chương trình được lưu giữ. Nội dung của PC được chuyển tới MAR và một tín hiệu điều khiển đọc được gửi tới bộ nhớ. Sau thời gian cần cho việc truy xuất bộ nhớ, nội dung của từ (word) nhớ (lệnh đầu tiên của chương trình) được đọc vào MDR. Tiếp theo, nội dung của MDR được chuyển đến IR. Tại thời điểm này lệnh đã sẵn sàng cho việc giải mã và thực hiện . Nếu lệnh là một phép toán mà ALU phải tính thì cần phải lấy

các toán hạng cần thiết. Nếu toán hạng đã có sẵn bên trong bộ nhớ (các toán hạng cũng có thể được chứa trong các thanh ghi dùng chung) thì nó phải được đọc vào ALU bằng cách gửi địa chỉ toán hạng tới MAR và khởi tạo chu trình đọc (read cycle). Toán hạng sau đó được đọc từ bộ nhớ vào MDR rồi chuyển tới ALU. Sau khi một hoặc nhiều toán hạng được đọc theo cách này, ALU có thể thực hiện phép toán. Nếu kết quả phép toán được lưu trữ trong bộ nhớ thì nó được gửi tới MDR. Địa chỉ ô nhớ tại đó kết quả được lưu trữ sẽ được gửi tới MAR và chu trình viết (write cycle) sẽ được bắt đầu. Trong khi một lệnh đang được thực hiện, nội dung cuả PC sẽ tăng lên sao cho nó chỉ tới địa chỉ của lệnh kế tiếp. Ngay sau khi lệnh hiện hành được hoàn tất, lệnh mới sẽ được lấy vào vi xử lý để giải mã và thực hiện.

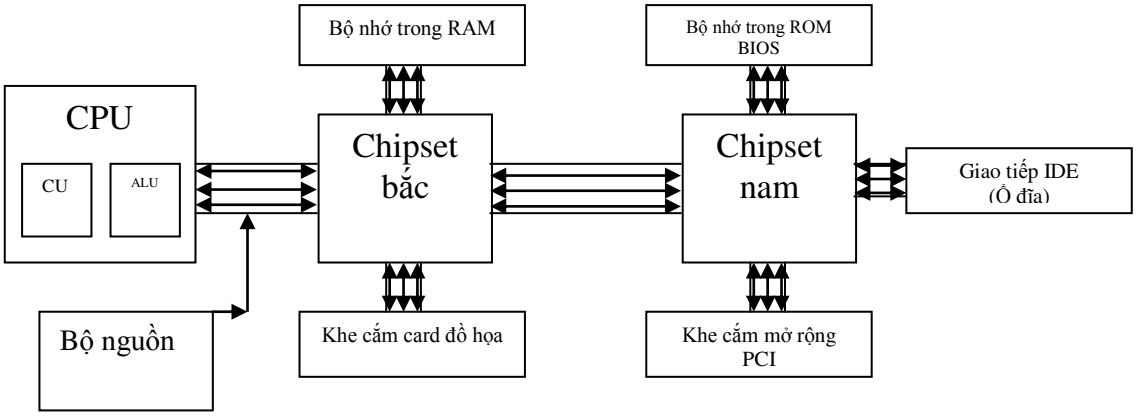
Cùng với việc chuyển số liệu giữa bộ nhớ chính và vi xử lý, máy tính truy xuất số liệu từ các thiết bị vào và gửi kết quả đến thiết bị ra. Do đó có một số lệnh máy cho phép chuyển số liệu vào ra.

Việc thực hiện một chương trình có thể bị ngắt nếu một hoặc nhiều thiết bị ngoại vi yêu cầu được phục vụ. Tín hiệu ngắt được tạo ra bởi các thiết bị ngoại vi để yêu cầu vi xử lý phải phục vụ chúng. Ngắt cũng được tạo ra bởi lệnh INT trong các chương trình. Vi xử lý sẽ cung cấp một dịch vụ theo yêu cầu bằng cách thi hành một chương trình con phục vụ ngắt (Interrupt service routine). Trước khi phục vụ ngắt, trạng thái bên trong của vi xử lý phải được cất giữ vào bộ nhớ. Bình thường, nội dung cuả PC, các thanh ghi đa chức năng và một số thông tin điều khiển phải được cất giữ. Khi chương trình con phục vụ ngắt kết thúc, trạng thái của chương trình chính sẽ được phục hồi sao cho chương trình chính đã bị ngắt có thể tiếp tục.

1.3 Giới thiệu tổ chức bên trong của máy vi tính

1.3.1 Sơ đồ

Hình 1.3 là sơ đồ cấu trúc của 1 máy vi tính



Hình 1.3: Sơ đồ cấu trúc máy vi tính

1.3.2 Cấu hình và các đặc trưng kỹ thuật

Tuỳ theo công việc mà người ta chọn máy tính có cấu hình phù hợp. Một máy tính thường được chú ý bởi các thành phần sau:

- Mainboard: Là loại mainboard có chipset hỗ trợ các loại CPU nào, tốc độ đồng hồ máy là bao nhiêu?
- CPU: Các thông số thường được chú ý: Hãng sản xuất, loại CPU, cache nội, tốc độ…
- RAM: Bộ nhớ trong thường được chú ý bởi các tham số: Loại RAM; dung lượng; tốc độ
- Ổ đĩa cứng: Các thông số thường được chú ý: Hãng sản xuất; dung lượng; tốc độ quay…
- Ổ đĩa quang: Gồm có các loại CD; DVD…Các thông số thường được chú ý là: Hãng sản xuất; tốc độ đọc ghi…
- Màn hình: Các thông số thường được chú ý: Là loại màn hình; độ rộng; độ phân giải; hãng sản xuất.
- Bàn phím: Các thông số thường được chú ý: Loại bàn phím; Hãng sản xuất
- Chuột: Các thông số: Loại chuột; hãng sản xuất

Chương 2. CƠ SỞ SỐ VÀ LOGIC

2.1. Các hệ thống số dùng trong máy tính điện tử

Trong máy tính người ta thường sử dụng các hệ đếm sau:

- Hệ hai (Cơ số 2) hay còn gọi là hệ nhị phân: Các con số của hệ nhị phân được biểu diễn bởi 2 ký số là: 0; 1.

- Hệ mười (Cơ số 10) hay còn gọi là hệ thập phân: Các con số trong hệ thập phân được biểu diễn bởi 10 ký số là: 0; 1; 2; 3; 4; 5; 6; 7; 8; 9.

- Hệ mười sáu (Cơ số 16) hay còn gọi là hệ thập lục phân: Các con số của hệ thập lục phân được biểu diễn bởi 16 ký số là: 0; 1; 2; 3; 4; 5; 6; 7; 8; 9; A; B; C; D; E; F.

Các hệ đếm trên đều là hệ đếm theo vị trí dựa trên cơ sở của các ký số. Giá trị của một con số được biểu diễn bởi 1 dãy các ký số, trong đó giá trị của 1 ký số tuỳ thuộc vào vị trí của nó trong dãy.

Công thức tổng quát tính giá trị của 1 con số trong 1 hệ đếm:

Nếu gọi:

- x là giá trị của con số; C là cơ số; K là ký số
- i là vị trí của ký số tính tại dấu ngăn cách thập phân; n được tính cho phần nguyên; m được tính cho phần lẻ.

$$\textbf{Ta có: } x = \sum_{i=-m}^{n-1} C^i * K_i$$

Để phân biệt các số trong các hệ đếm, người ta quy ước như sau:

- Con số của hệ thập phân kết thúc bằng D hoặc d hoặc có thể bỏ qua
- Con số của hệ nhị phân kết thúc bằng B hoặc b
- Con số của hệ thập lục phân kết thúc bằng H hoặc h.

Máy tính sử dụng hai trạng thái ứng với 0 và 1, như vậy để biểu diễn 1 số cần quá nhiều ký số, cho nên người ta thường dùng 1 ký số của hệ thập lục phân để biểu diễn 1 nhóm 4 ký số của hệ nhị phân.

Bảng tương ứng giữa nhóm 4 ký số của hệ nhị phân và hệ thập phân và hệ thập lục phân:

Decimal	Hexa	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111

Decimal	Hexa	Binary
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

2.2 Biểu diễn thông tin trong máy tính điện tử

Máy tính số (Computer) là một mạng điện phức tạp, hoạt động trên nguyên tắc có hay không có dòng điện chạy trong từng đoạn mạch trong mạng điện đó. Dữ liệu được ghi nhớ bằng các mạch điện có khả năng thể hiện một số trạng thái xác định. Mỗi mạch điện như vậy là một trạng thái nhớ căn bản. Số giá trị một phần tử nhớ căn bản có thể biểu diễn cũng chính là số trạng thái mạch điện tạo nên nó. Do khả năng chế tạo mạch, hiện nay các phần tử nhớ trên máy tính thường được thiết lập từ các mạch điện có hai trạng thái xác định bởi mức điện áp cao và thấp. Tương ứng với hai trạng thái của mạch điện, mỗi phần tử nhớ mang một trong hai giá trị là 0 hoặc 1. Một phần tử nhớ như vậy được gọi là 1 bit. Bằng các thiết bị điện tử người ta có thể chế tạo các mạch thực hiện các phép toán logic NOT, AND, OR, XOR với bảng sự thật của 2 phần tử A và B có các trạng thái 0 hoặc 1 như sau:

A	B	NOT A	A AND B	A OR B	A XOR B
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

Trong máy tính để biểu diễn được nhiều trạng thái hơn, người ta thường dùng 1 nhóm các bit và có tên gọi như sau:

- Nhóm 4 bit được gọi là nibble

- Nhóm 8 bit được gọi là Byte

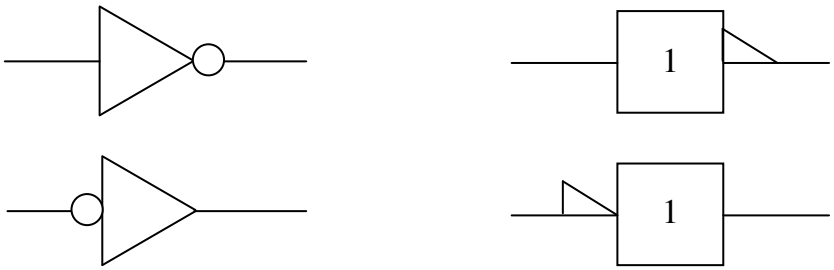
- Nhóm 16 bit được gọi là Word

- Nhóm 32 bit được gọi là double word (dword)

2.3 Các mạch điện tử số trong máy tính

2.3.1 Bộ đảo pha.

Ký hiệu trên bản vẽ cho bộ đảo pha như hình 2.1. Trong đó hình a) có hình thù phân biệt, phần b) thể hiện các hình vuông góc.



a) Các ký hiệu có hình phân biệt với chỉ thị âm

a) Ký hiệu nét vuông góc với chỉ thị phân cực

Hình 2.1: Các ký hiệu logic tiêu chuẩn cho bộ đảo pha

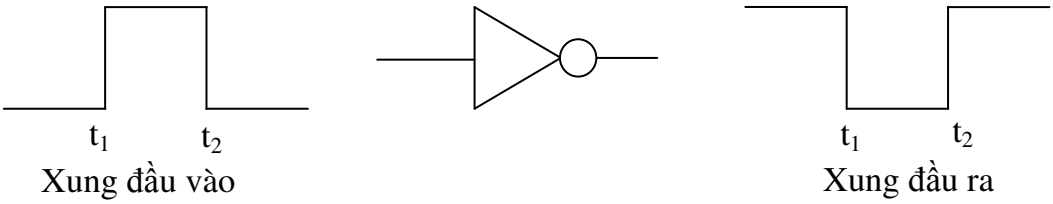
+ Dụng cụ chỉ thị âm và chỉ thị phân cực:

Dụng cụ chỉ thị âm có hình bọt bóng chỉ sự đảo pha khi nó xuất hiện ở đầu vào hoặc đầu ra của phần tử logic. Đầu ra có bọt bóng có nghĩa là mức 0 đang hoạt động hoặc trạng thái đầu ra được xác nhận. Nếu không có bọt bóng ở đầu vào hoặc đầu ra có nghĩa là mức 1 đang hoạt động hoặc trạng thái được xác nhận.

Chiều phân cực hoặc dụng cụ chỉ thị định mức là 1 hình tam giác như ở hình b chỉ ra sự đảo pha khi nó xuất hiện ở đầu vào hoặc đầu ra. Khi nó xuất hiện ở đầu vào có nghĩa là mức thấp đang hoạt động hoặc trạng thái đầu vào đã được xác nhận. Khi xuất hiện ở đầu ra thì có nghĩa là mức thấp đang hoạt động hoặc trạng thái đầu ra đã được xác nhận.

+ Thực hiện đảo pha:

Hình 2.2 thể hiện đầu ra của 1 bộ đảo pha với đầu vào là 1 xung và xung nhận được ở đầu ra



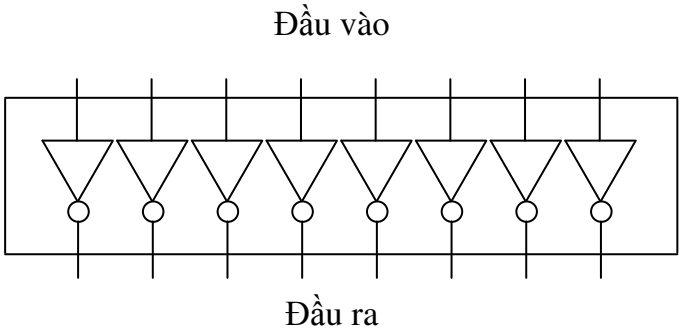
Hình 2.2: Bộ đảo pha với xung đầu vào xung và xung đầu ra

+ Bảng sự thật của bộ đảo pha:

Đầu vào	**Đầu ra**
Thấp (0)	Cao (1)
Cao (1)	Thấp (0)

+ Một ứng dụng của bộ đảo pha:

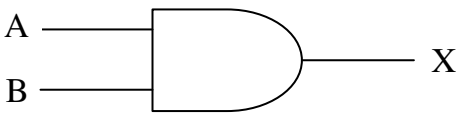
Hình 2.3 cho thấy 1 ứng dụng của 1 bộ đảo pha thực hiện việc lấy phần bù của 1 số nhị phân 8 bit. Các bit của số nhị phân được đưa vào tại đầu vào và ở đầu ra ta sẽ có phần bù của số nhị phân đó.



Hình 2.3: Mạch đảo pha 8 bit

2.3.2 Cổng AND.

+ Cổng AND được sử dụng để thực hiện phép toán logic AND. Cổng AND có thể có nhiều đầu vào và chỉ có 1 đầu ra. Hình 2.4 là ký hiệu cổng AND có 2 đầu vào.



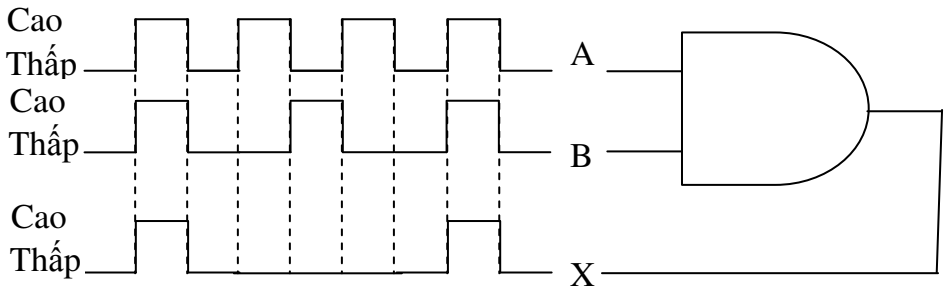
Hình 2.4: Ký hiệu cổng AND 2 đầu vào và 1 đầu ra

+ Phép toán logic của cổng AND: Cổng AND tại đầu ra là mức cao (1) khi tất cả đầu vào đều là mức cao (1); trong tất cả các trường hợp còn lại thì đầu ra là mức thấp (0).

+ Bảng sự thật của cổng AND 2 đầu vào:

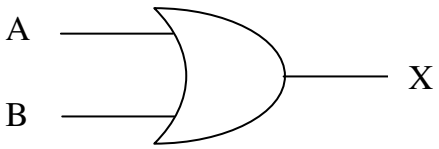
Đầu vào		**Đầu ra**
A	**B**	**X**
0	0	0
0	1	0
1	0	0
1	1	1

+ Ví dụ: Hai dạng sóng A và B được đưa vào đầu vào của 1 cổng AND ở hình 2.5 và dạng sóng tại đầu ra của nó



Hình 2.5: Dạng sóng đầu vào và đầu ra của cổng AND 2 đầu vào

2.3.3 Cổng OR.



Hình 2.6: Ký hiệu cổng OR 2 đầu vào

+ Cổng OR được sử dụng để thực hiện các phép toán logic OR. Cổng OR có thể có nhiều đầu vào nhưng chỉ có 1 đầu ra. Hình 2.6 là ký hiệu của 1 cổng OR có 2 đầu vào.

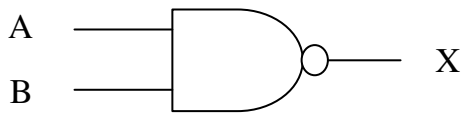
+ Các phép toán trên cổng OR: Cổng OR có giá trị đầu ra là thấp (0) khi tất cả tất cả các đầu vào là thấp; tất cả các trường hợp khác đầu cho đầu ra là cao.

+ Bảng sự thật của cổng OR có 2 đầu vào: X = A + B

Đầu vào		Đầu ra
A	**B**	**X=A+B**
0	0	0
0	1	1
1	0	1
1	1	1

2.3.4 Cổng NAND.

+ Cổng NAND là viết tắt của Not AND và bao hàm một hàm AND và một đầu ra bù. Cổng NAND có thể có nhiều đầu vào và chỉ có 1 đầu ra. Hình 2.7 là ký hiệu của cổng NAND 2 đầu vào.



Hình 2.7: Ký hiệu cổng NAND 2 đầu vào

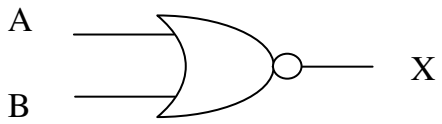
+ Các phép toán logic của cổng NAND: Cổng NAND có giá trị đầu ra ở mức thấp khi tất cả các giá trị đầu vào là cao; trong tất cả các trường hợp còn lại đầu ra của nó có giá trị cao.

+ Bảng sự thật của cổng NAND có 2 đầu vào: X = $\overline{A.B}$

Đầu vào		Đầu ra
A	**B**	**X=** $\overline{A..B}$
1	1	0
0	1	1
1	0	1
1	1	1

2.3.5 Cổng NOR.

+ Cổng NOR là viết tắt của Not OR; nó bao hàm 1 hàm OR và một đầu ra bù. Cổng NOR có thể có nhiều đầu vào, nhưng chỉ có 1 đầu ra. Hình 2.8 là ký hiệu của 1 cổng NOR có 2 đầu vào.



Hình 2.8: Ký hiệu cổng NOR 2 đầu vào

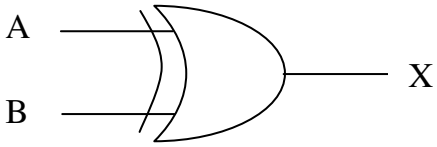
+ Các phép toán logic của cổng NOR: Cổng NOR có giá trị đầu ra ở mức cao nếu tất cả các giá trị của đầu vào đều là thấp; trong tất cả các trường hợp còn lại đầu ra sẽ có giá trị thấp.

+ Bảng sự thật của cổng NOR 2 đầu vào: X = $\overline{A+B}$

Đầu vào		**Đầu ra**
A	**B**	**X=** $\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

2.3.6 Cổng XOR.

+ Cổng OR-loại trừ (Viết tắt là XOR) được dùng để thực hiện phép toán XOR. Cổng XOR chỉ có 2 đầu vào và 1 đầu ra. Hình 2.9 là ký hiệu của cổng XOR.



Hình 2.9: Ký hiệu cổng XOR

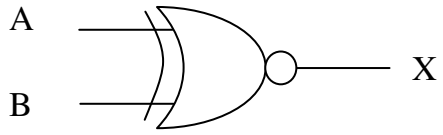
+ Các phép toán trên cổng XOR: Đầu ra của cổng XOR là cao nếu tại 2 đầu vào của nó có giá trị đối nghịch nhau; trong các trường hợp còn lại thì đầu ra của nó có giá trị thấp.

+ Bảng sự thật của phép toán XOR

Đầu vào		**Đầu ra**
A	**B**	**X**
0	1	1
1	0	1
1	0	0
1	1	0

2.3.7 Cổng XNOR.

+ Cổng XNOR là cổng NOR-loại trừ. Cổng XNOR chỉ có 2 đầu vào và 1 đầu ra. Hình 2.10 là ký hiệu của cổng XNOR.



Hình 2.10: Ký hiệu cổng XNOR

+ Các phép toán trên cổng XNOR: Đầu ra của cổng XNOR là cao nếu cả 2 đầu vào cùng giá trị; trong các trường hợp còn lại thì đầu ra có giá trị thấp.

+ Bảng sự thật của cổng XNOR:

Đầu vào		**Đầu ra**
A	**B**	**X**
0	0	1
1	1	1
1	0	0
0	1	0

2.3.8 Các phép tính trên hệ nhị phân

Các phép tính trên số nhị phân như: Cộng (+); trừ (-); nhân (*); chia (/) được thực hiện với các quy tắc giống như trên hệ thập phân.

Trên máy tính các số nhị phân được lưu trữ bởi 1 số bit xác định (4bit/8bit/16bit/32bit…), do vậy nó có chiều dài không đổi.

2.3.8.1 Phép cộng nhị phân:

Quy tắc:

0 + 0 = 0

0 + 1 = 1

1 + 0 = 1

1 + 1 = 0 nhớ 1 (10)

Ví dụ: Giả sử có 8 bit để lưu trữ. Ta có:

+	0	0	1	1	0	1	1	1
	0	1	0	0	1	1	0	0
	1	0	0	0	0	0	1	1

+	1	1	0	0	1	0	0	0
	1	0	0	0	0	1	1	1
1	0	1	0	0	1	1	1	1

↑

Bit tràn

2.3.8.2 Phép trừ nhị phân

Quy tắc:

0 - 0 = 0

1 - 1 = 0

1 – 0 = 1

0 – 1 = 1 nhớ 1

Ví dụ:

-	1	1	1	1	0	1	1	1
	0	1	0	0	1	1	0	0
	1	0	1	0	1	0	1	1

-	1	1	0	0	1	0	0	0
	1	0	0	0	0	1	0	1
	0	1	0	0	0	0	1	1

2.3.8.3 Phép nhân nhị phân

Quy tắc:

0 * 0 = 0

1 * 0 = 0

0 * 1 = 0

1 * 1 = 1

Ví dụ:

				1	0	0	1	Số bị nhân (9)
			x	0	1	1	0	Số nhân (6)
				0	0	0	0	Thành phần thứ 1 của tổng tích luỹ
			1	0	0	1		Thành phần thứ 2 của tổng tích luỹ
		1	0	0	1			Thành phần thứ 3 của tổng tích luỹ
	0	0	0	0				Thành phần thứ 4 của tổng tích luỹ
0	1	1	0	1	1	0		Tổng tích lũy (54)

2.3.8.4 Phép chia nhị phân.

Phép chia là phép tính ngược của phép nhân.

Ví dụ:

```
1 0 0 0 1 1 | 1 0 1
0 0 0       | 0 1 1 1
-------------
1 0 0 0
  1 0 1
-----------
0 0 1 1 1
    1 0 1
  -------
  0 1 0 1
    1 0 1
  -------
  0 0 0
```

2.3.8.5 Biểu diễn số nguyên và phân số:

+ Số nhị phân không dấu biểu diễn cho các số nguyên lớn hơn 0 và 0. Một giá trị n (n được tính từ 0) bit biểu diễn số nguyên không dấu là:

$X_n*2^n + X_{n-1}*2^{n-1} + ….+ X_o*2^0$

Với X_n là 0 hoặc 1

+ Số nguyên âm được biểu diễn ở bit dấu cao nhất với dạng bù 2

+ Phân số: Một phân số được coi là tỉ số của 2 số nguyên. Máy tính biểu diễn phân số bằng các số 0 và 1 bằng cách đặt bit bên trái nhất của số biểu diễn là 1/2, tiếp theo là 1/4, 1/8, 1/16,…Ví dụ để biểu diễn phân số 0.1001 (Bit 0 đầu tiên cho biết là phân số dương, tiếp theo là 1/2+0/4+0/8+1/16 = 9/16).

2.3.8.6 Biểu diễn số có dấu phảy (chấm) động (Floating Point Number)

Để tính toán những số lớn và những số quá bé người ta biểu diễn số dưới dạng:

$$N=M*B^E \quad (1)$$

Trong đó:

M: là phần định trị (Mantissa, Significand or fraction)

E: là số mũ (Exponent)

B: là cơ số

Việc biểu diễn các số dưới dạng (1) sẽ rất thuận tiện khi tính toán. Ví dụ :

$(a*10^m)*(b*10^n) = a*b*10^{n+m}$

$(a*10^m)/(b*10^n) = (a/b)*10^{m-n}$

Nếu m=n thì:

$(a*10^m) + (b*10^n) = (a+b)*10^m$

Việc làm cho m = n gọi là sự mở rộng số (Scaling Number)

Như vậy thì rất khó có thể đạt được độ chính xác đối với những số quá lớn hoặc quá bé. Để giải quyết vấn đề này người ta dùng 2 kỹ thuật sau:

* Trên máy tính dùng một chương trình con dấu chấm động (Floating –point routines) để Scaling các số nhằm đạt được độ chính xác mong muốn. Chương trình con này chỉ được sử dụng trên các máy tính có các phép toán dấu chấm cố định (fixed-Point Operations)

* Thêm các phép toán dấu chấm động (Floating –Point Operations) vào phần cứng máy tính, mạch Logic của máy tính sẽ thực hiện Scaling tự động và theo dõi số mũ khi tính toán. Để làm được điều này người ta dùng hệ thống số dấu chấm động (Floating Point System). Số có dấu chấm động trên máy tính dùng hệ số mũ như đã mô tả ở trên. Trong khi tính toán máy tính theo dõi cả phần mũ lẫn phần định trị.

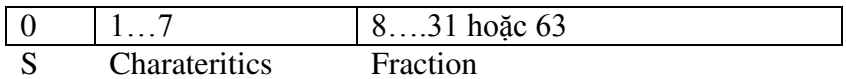
Máy tính IBM có mạch thực hiện phép toán có dấu chấm động. Một số dấu chấm động theo IBM có 3 phần:

* Bit dấu (Signal Bit): Chỉ ra số đó âm hay dương.

* Số mũ: Gọi là Charateristic.

* Phần định trị: Gọi là Fraction

Hình vẽ sau mô tả một số dấu chấm động trong PC IBM có độ dài từ 32 hoặc 64 Bit



Hình 2.11 : Số có dấu châm động trong PC IBM

* Bit đầu tiên bên trái là Bit dấu S: Bit dấu bằng 0 là số dương, bằng 1 là số âm

* Bit từ 1 đến 7 là phần đặc trưng (Charateritics) gọi là C. Giá trị của C từ 0 đến 127

* Các bit 8..31 (Đối với máy có độ dài từ (word) 32bit) hoặc 63 (Đối với máy có độ dài từ (word) 64bit) gọi là phần định trị (phần sau dấu chấm).

- Giá trị bằng số trong hệ 10 được tính như sau:

$(-1)S * 16^{C-64} * Fraction$

Với: C là giá trị thập phân tương đương của phần Characteritic.

Fraction là phân số.

Ví dụ1: Số có chấm động 0 1000001 111000..0

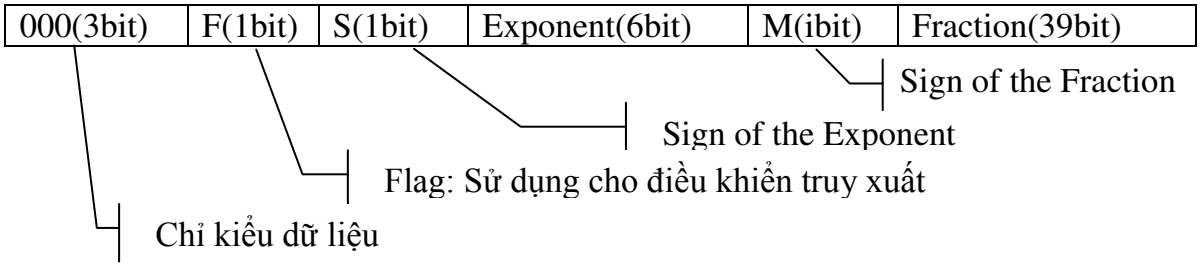
Có giá trị bằng: $+16^{65-64}$ x 7/8 = +14

Ví dụ2: Số có dấu chấm động: 1 1000011 1100..0

Có giá trị bằng: $- 16^{67-64}$ x 3/4 = - 3072

2.3.8.7 Các cách biểu diễn số có dấu chấm động:

*B-5500 (của Burroughs coporation)



* Floating Point for the 32 bit IBM System/370:

S (1bit)	Exces- 64 Exponent (7bit)	Fraction(24bit)

* IEEE (Institute of Electrical and Electronics Engineers)

- Single procision:

S (1bit)	Exponent (8bit)	Fraction(23bit)

-Double Procision:

S (1bit)	Exponent (11bit)	Fraction(52bit)

2.4 Cơ sở logic trong máy tính

2.4.1 Hàm Boolean và bảng sự thật

2.4.1.1 Các biểu thức và phép toán Boolean

+ Biến số, phần bù:

Biến số, phần bù là các số hạng trong đại số logic. Biến A; biến B; biến… là các ký hiệu thường được sử dụng để đại diện cho các toán hạng logic. Bất kỳ 1 biến đơn nào cũng có giá trị là 0 hoặc 1. Phần bù của nó được biểu diễn bằng tên biến và có dấu gạch ngang trên đầu. Ví dụ: A thì phần bù của nó là $\overline{A}$.

+ Phép cộng Boolean:

Phép cộng Boolean tương đương với phép toán OR (Cổng OR), được biểu diễn bằng dấu cộng (+) giữa các toán hạng. Quy tắc như sau:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1$$

+ Phép nhân Boolean:

Phép nhân Boolean tương đương phép AND (Cổng AND), được biểu diễn bằng dấu chấm (.) giữa 2 toán hạng hoặc 2 toán hạng viết liền nhau. Quy tắc như sau:

$$0 . 0 = 0$$

$$0 . 1 = 0$$

$$1 . 0 = 0$$

$$1 . 1 = 1$$

2.4.1.2 Các quy luật và quy tắc của đại số Boolean.

a/ Các quy luật của đại số Boolean:

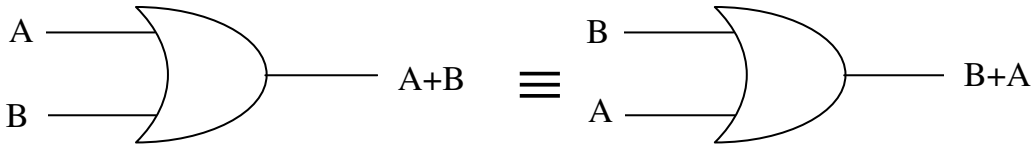
** Luật giao hoán:*

Luật giao hoán của phép cộng và phép nhân 2 biến được thể hiện như sau (Có thể mở rộng cho nhiều biến)

- Luật giao hoán của phép cộng:

$$A + B = B + A$$

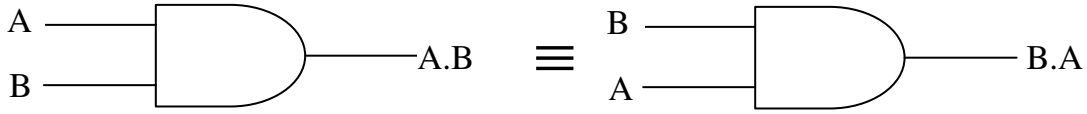
Mạch logic được biểu diễn như sau:



- Luật giao hoán của phép nhân

$$A \cdot B = B \cdot A$$

Mạch logic:

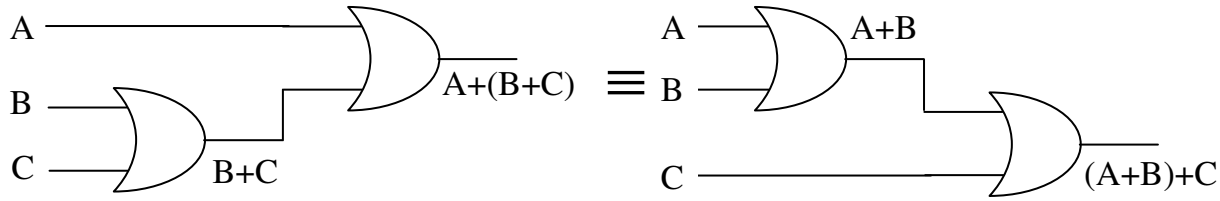


** Luật kết hợp:*

+ Luật kết hợp với phép cộng 3 biến như sau:

$$A + (B + C) = (A + B) + C$$

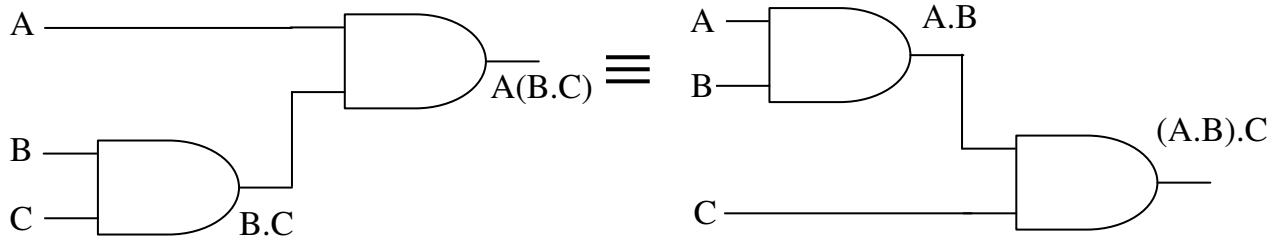
Mạch logic biểu diễn cho luật kết hợp của phép cộng 3 biến



+ Luật kết hợp với phép nhân 3 biến như sau:

$$A.(B.C) = (A.B).C$$

Mạch logic biểu diễn cho luật kết hợp của phép nhân 3 biến

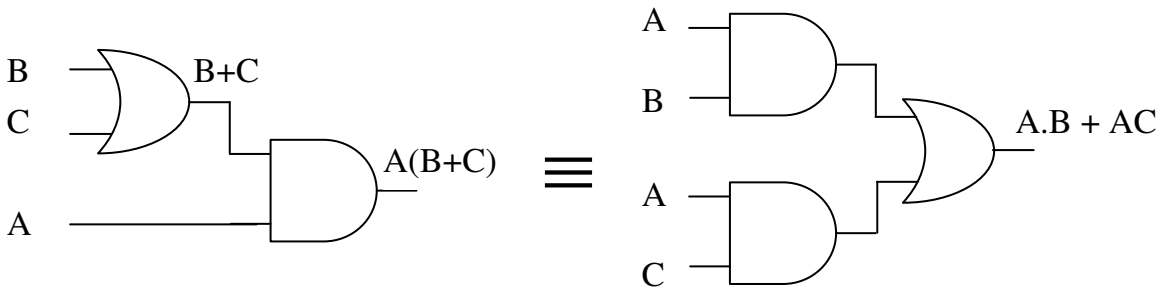


** Luật phân phối:*

- Luật phân phối của 3 biến được viết như sau:

$$A.(B+C) = A.B+A.C$$

- Mạch logic của luật phân phối 3 biến:

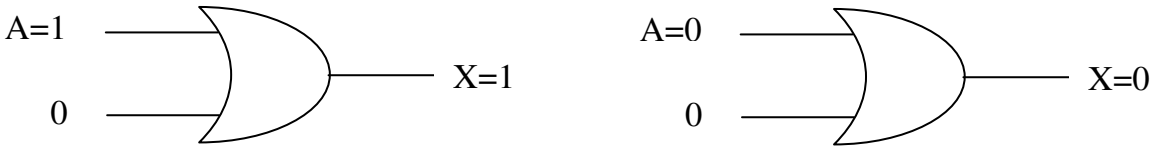


b/ Các quy tắc của đại số Boolean:

Các quy tắc sau, các biến A; B; C có thể đại diện cho biến đơn hoặc tổ hợp cho các biến.

Quy tắc 1: A + 0 = A

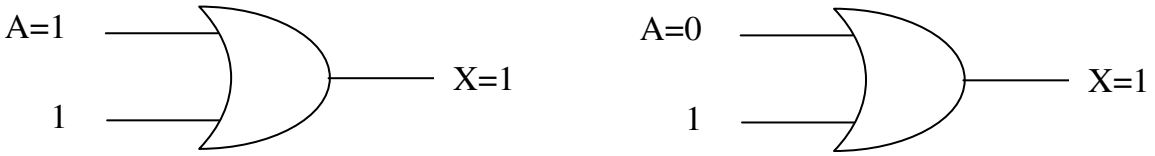
Biến A được OR với 0 thì luôn có giá trị bằng biến đó. Mạch logic 2 đầu vào như sau:



X = A + 0 = A

Quy tắc 2: A + 1 = 1

Biến A được OR với 1 thì luôn có giá trị là 1. Mạch logic 2 đầu vào như sau:



X = A + 1 = 1

Quy tắc 3: A.0 = 0

Biến A được AND với 0 thì luôn có giá trị là 0. Mạch logic 2 đầu vào như sau:



A.0 = A

Quy tắc 4: A.1 = A

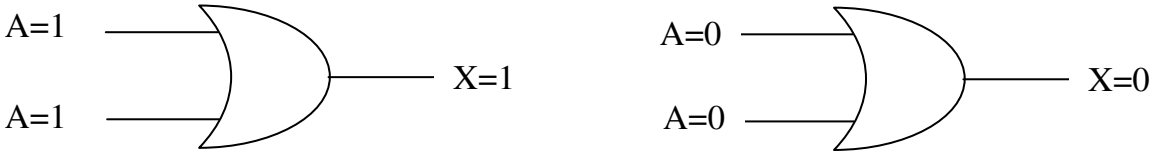
Biến A được AND với 1 thì luôn cho giá trị bằng chính nó. Mạch logic như sau:



A.1 = A

Quy tắc 5: A + A = A

Biến A được OR với chính nó thì luôn bằng nó. Mạch logic như hình sau:



A + A = A

Quy tắc 6: $A + \overline{A} = 1$

Biến A được OR với phần bù của nó thì luôn cho giá trị bằng 1. Mạch logic như hình sau:



A + $\overline{A}$ = 1

Quy tắc 7: A.A = A

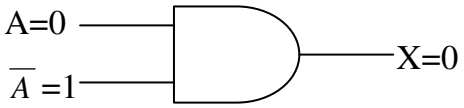
Biến A được AND với chính nó thì luôn cho giá trị bằng chính nó. Mạch logic 2 đầu vào như hình sau:

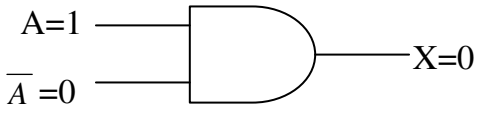


A.A = A

Quy tắc 8: $A.\overline{A} = 0$

Biến A được AND với phần bù của nó thì luôn cho giá trị là 0. Mạch logic 2 đầu vào như hình sau:

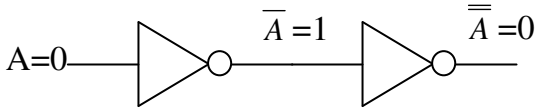


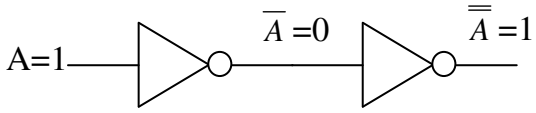


A. $\overline{A}$ **= 0**

Quy tắc 9: $\overline{\overline{A}} = A$

Bù 2 lần của 1 biến sẽ bằng chính nó. Mạch logic cho mạch bù 2 lần như hình sau:





$\overline{\overline{A}} = A$

Quy tắc 10: $A + A.B = A$

Quy tắc này thực ra là sự suy ra của quy tắc 2 và quy tắc 4

Ta có bảng sự thật sau:

A	B	A.B	A+A.B
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Quy tắc 11: $A + \overline{A}.B = A + B$

Quy tắc này có thể chứng minh bằng cách dựa vào các quy tắc trên. Ta có bảng sự thật như sau:

A	**B**	$\overline{A}.B$	$A + \overline{A}.B$	**A+B**
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

Quy tắc 12: $(A+B).(A+C) = A + B.C$

Quy tắc này có thể chứng minh bằng cách dựa trên các quy tắc đã nêu trên. Bảng sự thật như sau:

A	B	C	A+B	A+C	(A+B)(A+C)	B.C	A+B.C
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

2.4.2 Cực tiểu hàm Boolean

2.4.2.1 Định lý De Morgan:

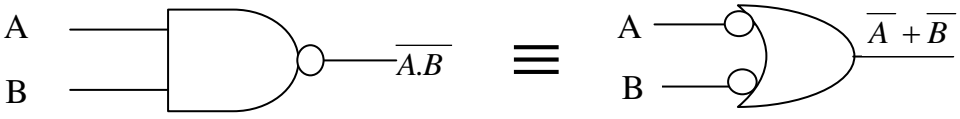
Định lý 1: ***Phần bù của 1 tích bằng tổng phần bù của các biến trong tích đó.***

Nói cách khác, phần bù của 2 hay nhiều biến trong phép AND tương đương với phép OR của các phần bù của từng biến riêng lẻ.

Công thức với 2 biến của định lý 1 như sau:

$$\overline{A.B} = \overline{A} + \overline{B}$$

Mạch logic biểu diễn cho công thức của định lý 1 như sau:



NAND **XOR**

Bảng sự thật của định lý 1 với 2 biến

A	B	$\overline{AB}$	$\overline{A} + \overline{B}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

Định lý 2: ***Phần bù của 1 tổng các biến bằng tích các phần bù của các biến trong tổng đó.***

Nói cách khác, phần bù của 2 hay nhiều biến trong phép OR tương đương với phép AND của các phần bù của từng biến riêng lẻ.

Công thức với 2 biến của định lý 2 như sau:

$$\overline{A+B} = \overline{A} \,.\, \overline{B}$$

Mạch logic biểu diễn cho định lý 2 với 2 biến như sau:



NOR **XAND**

Bảng sự thật của định lý 2 với 2 biến:

A	**B**	$\overline{A+B}$	$\overline{A} \,.\, \overline{B}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

2.4.2.2 Đơn giản hoá và thiết kê mạch logic đơn giản

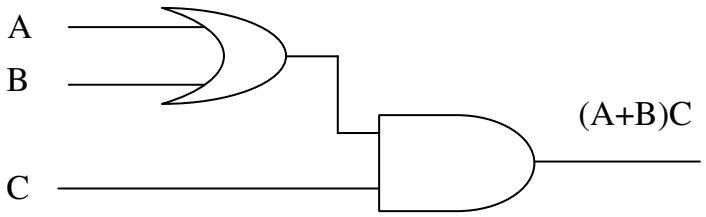
Để xây dựng 1 mạch logic theo 1 yêu cầu nào đó, các bước cần thực hiện như sau:

1. Xác định nội dung của yêu cầu
2. Xây dựng bảng sự thật theo cách đã mô tả
3. Tìm biểu thức logic theo bảng sự thật. Bước này gồm các bước sau:
   - Viết các số hạng AND chứa các biến vào đối với các trường hợp X = 1
   - Các biến đối với số hạng AND có thể ở dạng đảo hoặc không đảo tuỳ vào giá trị bằng 0 hoặc bằng 1
   - Tất cả các số hạng AND được OR với nhau để có được biểu thức của X
   - Đơn giản hoá biểu thức của X
4. Xây dựng mạch logic thực hiện biểu thức X.

Ví dụ 1: Cho biểu thức logic: $\overline{\overline{(A+B)}+\overline{C}}$. Hãy đơn giản hoá biểu thức và xây dựng mạch logic.

Ta có: $\overline{\overline{(A+B)}+\overline{C}} = \overline{\overline{(A+B)}}.\overline{\overline{C}} = (A+B)C$

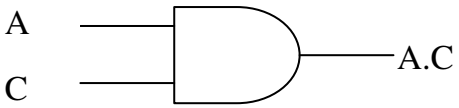
Mạch logic cho biểu thức:



Ví dụ 2: Cho biểu thức: (A+AB)C. Hãy đơn giản biểu thức và xây dựng mạch logic cho biểu thức đã đơn giản đó.

Ta có: (A+A.B).C = A.(1+B).C = A.1.C = A.C

Mạch logic như sau:



Ví dụ 3: Cho biểu thức: (A+$\overline{A}$B)C. Đơn giản hoá biểu thức

Ta có: (A+$\overline{A}$B)C = ((A+AB)+$\overline{A}$B)C

= ((AA+AB)+$\overline{A}$B)C

= (AA+AB+A$\overline{A}$ + $\overline{A}$B)C

= (A+$\overline{A}$)(A+B)C

= 1.(A+B)C

= (A+B)C

Ví dụ 4: Một nhóm quyết định gồm 3 người. Nhóm người này bỏ phiếu bí mật để thông qua 1 vấn đề nào đó. Vấn đề được thông qua nếu số phiếu quá bán. Xây dựng mạch logic cho trường hợp trên.

Từ yêu cầu trên ta có bảng sự thật như sau:

A	**B**	**C**	**X**
0	0	0	0
0	0	1	0

0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Ta có: $X = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$

Đơn giản hóa biểu thức:

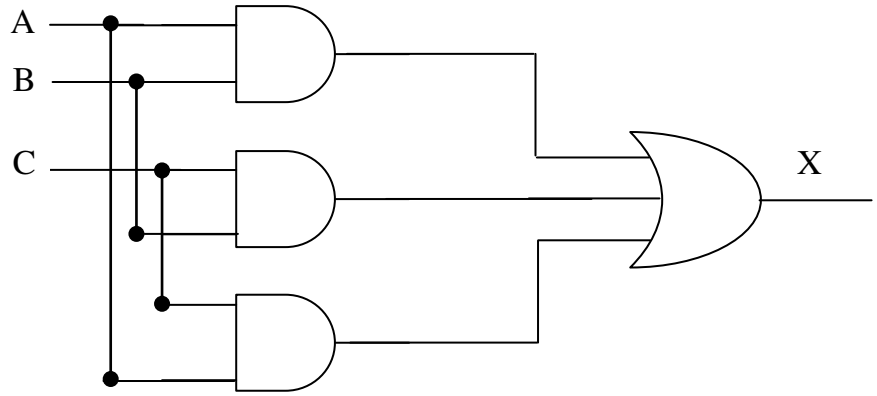
$$\overline{A}BC + A\overline{B}C + AB\overline{C} + ABC = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC + ABC + ABC$$

$$= BC(\overline{A} + A) + AB(\overline{B} + B) + AC(\overline{C} + C)$$

$$= BC + AC + AB$$

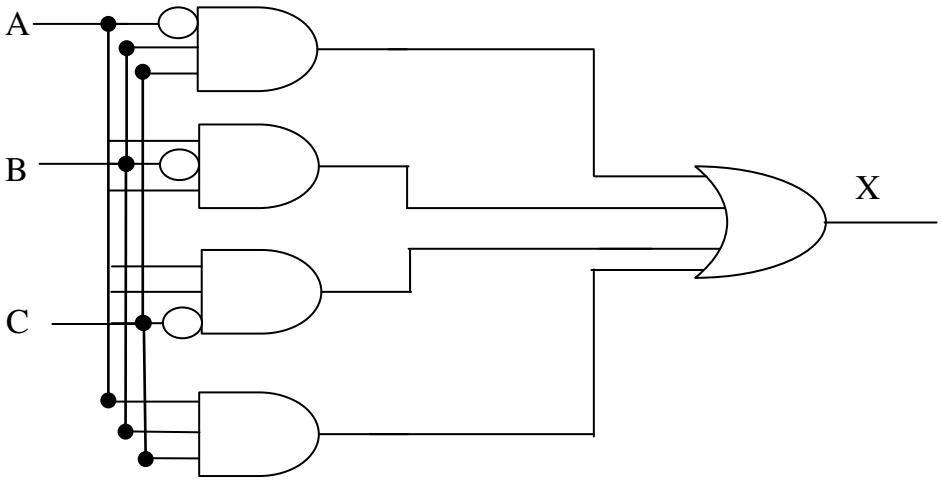
Hay: X = C(B+A)+AB

+ Mạch logic ban đầu chưa tối ưu:



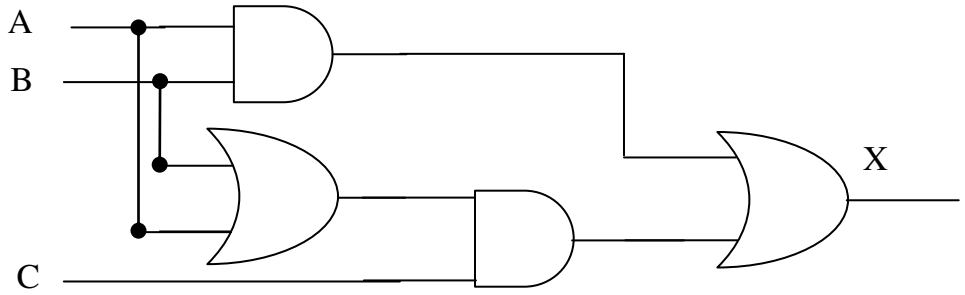
+ Mạch logic theo bảng sự thật chưa tối ưu:

$$X = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$



+ Mạch logic sau khi đã tối ưu:

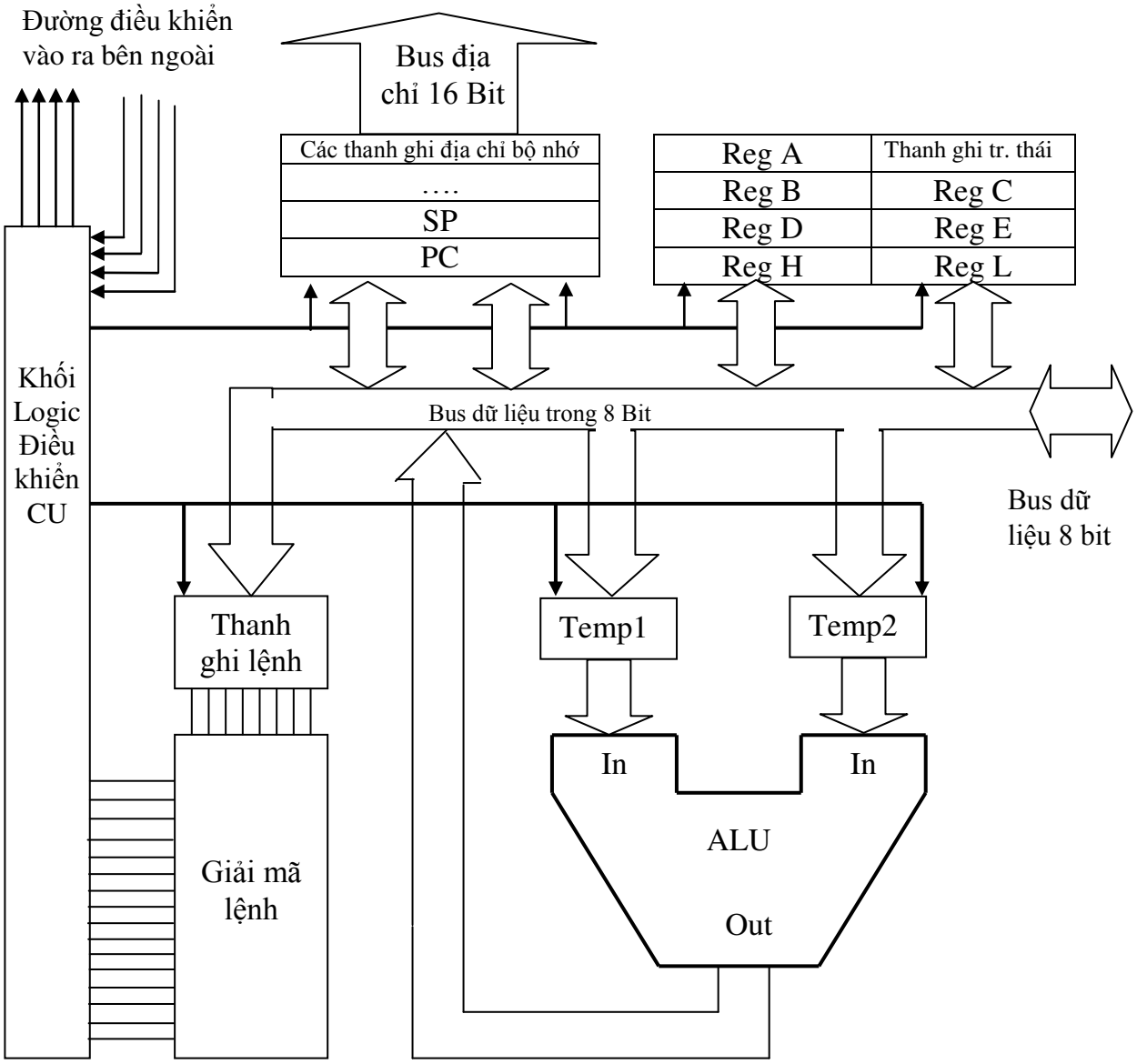
$$X= C(B+A)+AB$$



Rõ ràng mạch logic sau khi đã tối ưu tốn ít dây nối và thực hiện cũng nhanh hơn.

Chương 3: TỔ CHỨC BỘ XỬ LÝ TRUNG TÂM

3.1 Sơ đồ khối tổng quát của bộ xử lý trung tâm (CPU - Central Processing Unit)



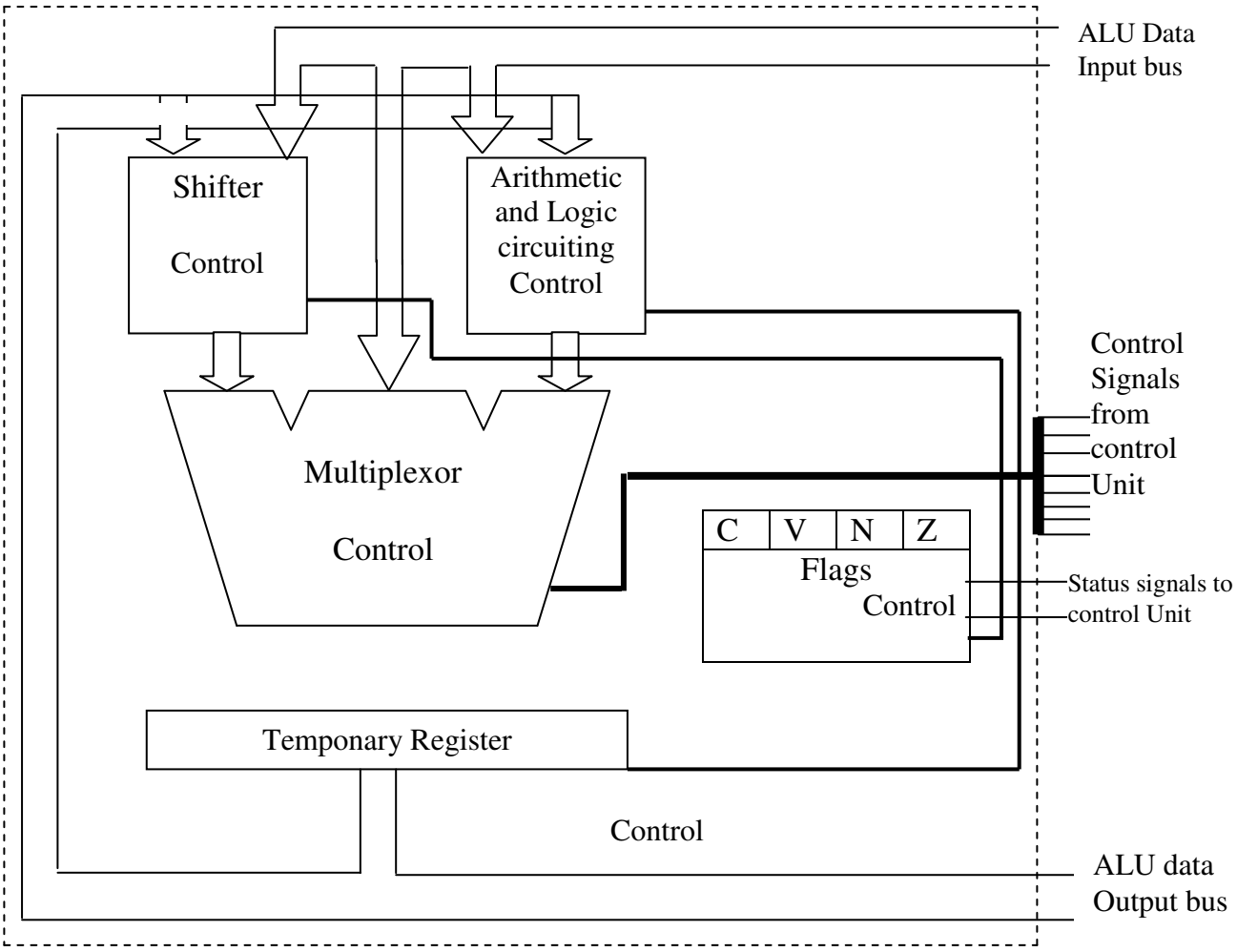
Hình 3.1 Cấu trúc bên trong của bộ vi xử lý 8 Bit

CPU của một máy tính đơn giản có ba thành phần chủ yếu đó là: Tập thanh ghi, bộ xử lý số học và logic ALU (Arithmetic-Logic Unit) và bộ điều khiển CU (Control Unit). Các thành phần này liên kết với nhau thông qua hệ thống BUS. Các máy tính đều có chức năng thực hiện các phép tính số học, logic và các thao tác

Shift. Một số CPU có thể chỉ có một đơn vị ALU nhưng một số CPU khác thì nó được thiết kế thành một số đơn vị chức năng độc lập. Hình 3.1 mô tả một CPU 8 bit

3.2 ALU (Arithmetic Logic Unit):

Hình 3.2 là sơ đồ khối của 1 ALU đơn giản gồm 1 đơn vị Shift và 1 đơn vị Arithmetic and Logic và các cờ: C (Cary), V (Overflow, N (Negative), Z (Zero)



Hình 3.2 – Sơ đồ khối ALU

Đơn vị ALU là một trong những thành phần chính bên trong CPU. ALU chứa khối logic thực hiện xử lý dữ liệu. ALU có 2 cổng vào (In) và 1 cổng ra (Out). Cổng vào để nhận từ dữ liệu vào ALU, cổng ra để lấy dữ liệu đã được xử lý ra ngoài. Các thanh ghi Temp1, Temp2 làm nhiệm vụ nhận dữ liệu từ các nơi khác nhau bên trong CPU, thông qua bus dữ liệu bên trong và lưu trữ trung gian dữ liệu trong quá trình xử lý dữ liệu trong ALU. Tương tự, cổng ra kết nối với bus dữ liệu bên trong, do đó

kết quả phép toán có thể được đưa tới các nơi khác nhau. Thông thường dữ liệu được gởi tới bộ cộng (Accumulator). Ví dụ khi ALU cộng 2 toán hạng, một toán hạng có trong bộ cộng, toán hạng thứ 2 có thể ở trong thanh ghi nào đó hoặc được chuyển từ ngăn nhớ bên ngoài CPU vào ALU, thông qua bus bên trong. Các lệnh máy được ALU xử lý có thể là 1 hay 2 word (hai toán hạng). Có một số lệnh chỉ có 1 toán hạng . Ví dụ lệnh lấy mã bù nghĩa là chỉ cần sử dụng 1 cổng vào của ALU. Lệnh máy được đọc từ bộ nhớ vào CPU, được giải mã từ bộ giải mã lệnh (Instruction decoder) để tạo ra chuỗi các tín hiệu đi tới ALU điều khiển quá trình xử lý dữ liệu trong ALU.

ALU của đa số các loại vi xử lý có thể thực hiện các chức năng sau đây:

* Add (Cộng)
* Sub (Trừ)
* AND (Và Logic)
* OR (Hoặc Logic)
* Exclusive (Hoặc đảo)
* Complement (Bù nhị phân)
* Shift Right (Dịch phải)
* Shift Left (Dịch trái)
* Increment (Tăng)
* Decrement (Giảm)

2.1.4 Tổ chức thanh ghi

Các thanh ghi bên trong bộ vi xử lý 8 bit được chia thành các nhóm theo mục đích sử dụng có thể chia thành 2 nhóm sau:

+ Nhóm thanh ghi dùng chung: Gồm các thanh ghi A (bộ cộng), B, C, D, E, H và L. Đây là các thanh ghi 8 bit. Các phép tính với các bytes dữ liệu đều có thể thực hiện với nội dung của các thanh ghi này. Để xử lý 16 bit có các lệnh thực hiện với các cặp thanh ghi BC, DE và HL. Cặp thanh ghi HL được dùng để chứa địa chỉ ngăn nhớ đối với các lệnh có truy nhập bộ nhớ. Riêng thanh ghi A còn gọi là bộ cộng, nó là thanh ghi đặc biệt.

+ Các thanh ghi khác gồm: Thanh ghi trạng thái SR, con trỏ ngăn xếp SP, thanh ghi đếm lệnh PC thanh ghi địa chỉ ngăn nhớ, thanh ghi tạm thời Temp1 và Temp2 và thanh ghi lệnh.

Thanh ghi lệnh (Instruction Register – IR): Dùng để lưu trữ lệnh. Người lập trình không thể trực tiếp thao tác lên thanh ghi này. Do đó nó không được coi như là một phần của tập thanh ghi của máy.

Thanh ghi địa chỉ (Memory Address Register – MAR) và thanh ghi nhớ đệm (Memory Buffer Register – MBR): Khi máy tính truy xuất bộ nhớ chúng giữ địa chỉ tham chiếu trong thanh ghi MAR và giữ dữ liệu ghi/đọc trong MBR. Người lập trình cũng không trực tiếp can thiệp vào 2 thanh ghi này nên nó cũng không được coi là một phần của tập thanh ghi của máy tính.

Chức năng của các thanh ghi như sau:

- ***Thanh tổng A (Accumulator):***

Thanh tổng A tham gia vào phần lớn các phép tính. Độ dài của thanh tổng bằng độ dài từ mà CPU xử lý. Nghĩa là nếu CPU xử lý 8 bit thì thanh tổng A có độ dài 8 bit. Tuy nhiên ở một số loại CPU có thể có độ dài thanh tổng gấp đôi độ dài xử lý của CPU hay còn gọi là thanh tổng độ dài kép. Thậm chí một số loại CPU còn có nhiều thanh tổng. Trong trường hợp này phải có lệnh chỉ cho bộ vi xử lý chuyển kết quả tính của ALU vào thanh tổng thứ nhất và một lệnh chỉ cho bộ vi xử lý chuyển dữ liệu vào thanh tổng tiếp theo. Tương tự có lệnh xóa dữ liệu trên thanh tổng thứ nhất, có lệnh xóa dữ liệu trên thanh tổng thứ hai. Những lệnh vào ra (I/O) với ngoại vi là nhóm lệnh trao đổi dữ liệu của thanh tổng A với các thanh ghi của điều khiển ngoại vi thông qua bus hệ thống và bus dữ liệu bên trong của CPU.. Lệnh nhập dữ liệu IN port là lệnh đọc dữ liệu từ cổng (Port) của ngoại vi (địa chỉ Port) vào thanh ghi A và lệnh xuất OUT Port đọc nội dung thanh ghi A ra Port ngoại vi.

- ***Thanh đếm chương trình PC (Program Counter)***:

Thanh đếm chương trình có độ dài lớn hơn so với độ dài từ xử lý của CPU. Các bộ vi xử lý thường có PC dài 16 bit. Một chương trình được bộ vi xử lý thực hiện phải chứa trong bộ nhớ chính (Main Memory). Thanh PC chứa địa của chỉ lệnh trong bộ nhớ và chỉ ra cho bộ vi xử lý biết lệnh tiếp theo nằm ở ngăn nhớ nào để lấy ra thực hiện. Như vậy độ dài của thanh PC chính là khả năng đánh địa chỉ bộ nhớ chính có thể đạt được của bộ vi xử lý. Ví dụ nếu thanh PC có độ dài 16 bit thì ta có thể quản lý dung lương 2^{16} bit = 64KB. Nghĩa là có thể đánh địa chỉ từ 0 đến 65.535 ngăn nhớ. Như vậy thanh PC luôn được nạp địa chỉ lệnh (địa chỉ ngăn nhớ chứa lệnh máy) tiếp theo trong quá trình thực hiện bất kỳ một chương trình nào. Lệnh nào, nằm ở đâu tùy thuộc vào trình tự thực hiện từng chương trình. Trong các bộ vi

xử lý công nghệ cao có cơ chế quản lý bộ nhớ ảo ((Virtual Memory). Bộ nhớ ảo bao gồm bộ nhớ chính và bộ nhớ ngoài. Cơ chế đánh địa chỉ ảo có sự biến đổi địa chỉ ảo thành địa chỉ vật lý (địa chỉ thực của bộ nhớ), như vậy nội dung của địa chỉ lệnh sẽ phức tạp hơn.

Thanh ghi địa chỉ ngăn nhớ (16 bit) và thanh đếm PC có độ dài giống nhau. Thanh ghi địa chỉ ngăn nhớ có kết nối với bus địa chỉ. Địa chỉ của ngăn nhớ chứa lệnh đầu tiên của chương trình là nội dung của thanh PC được chuyển tới thanh ghi địa chỉ ngăn nhớ để ra bus địa chỉ, nghĩa là trên bus địa chỉ có địa chỉ của lệnh đầu tiên ở trạng thái tích cực. Bộ nhớ chính kết nối trên bus sẽ nhận được các đường địa chỉ tích cực này, giải mã chọn được ngăn nhớ chứa lệnh. Quá trình tiếp theo sẽ là đọc lệnh và thực hiện lệnh.

- ***Thanh ghi trạng thái SR (Status Register)***

Thanh ghi trạng thái dùng để ghi kết quả của các lệnh kiểm tra, so sánh và một số lệnh tính toán với các thanh ghi. Thanh ghi trạng thái còn gọi là thanh ghi cờ (Flag Register). Sử dụng các bit của thanh ghi trạng thái (còn gọi là các bit cờ) có thể thực hiện rẽ nhánh chương trình bằng các lệnh nhảy và rẽ nhánh có điều kiện. Thanh ghi trạng thái của bộ vi xử lý 8 bit chỉ có 8 bit. Mỗi bit của thanh ghi này có một ý nghĩa và bị tác động tùy theo lệnh.

- ***Con trỏ ngăn xếp SP (Stack Point)***

Ngăn xếp là bộ nhớ có cơ chế truy cập theo kiểu LIFO (Last In First Out), nghĩa là byte dữ liệu nào ghi vào ngăn xếp cuối cùng sẽ được đọc ra đầu tiên. Ngăn xếp có thể là một vùng nào đó của bộ nhớ chính, hay là một mảng thanh ghi riêng biệt. Ngăn xếp luôn được truy cập ở đỉnh (Top). Nó làm nhiệm vụ lưu trữ những thông tin phải dùng đi dùnglại nhiều lần trong quá trình thực hiện chương trình. Các lệnh tác động đến ngăn xếp là nhóm lệnh gọi (CALL) đến chương trình con, nhảy trở lại chương trình chính từ chương trình con (RET), lệnh ngắt (INT) và nhảy từ chương trình xử lý ngắt (IRET). Có các lệnh chuyên dùng để cất giữ nội dung các thanh ghi vào ngăn xếp (vì sau đó phải dùng lại nội dung các thanh ghi đó) như lệnh (PUSH) và lệnh lấy ra nội dung các thanh ghi từ ngăn xếp (POP). Các lệnh này luôn làm thay đổi đỉnh của ngăn xếp. Con trỏ ngăn xếp SP chứa địa chỉ của đỉnh ngăn xếp và

nội dung của SP sẽ thay đổi khi thực hiện các lệnh vừa nói trên, tức là nó luôn trỏ đến vùng nhớ tiếp theo. Khi khởi động hệ thống máy tính, con trỏ ngăn xếp luôn được khởi tạo về địa chỉ đỉnh của ngăn xếp. Nếu nó không được khởi tạo ngăn xếp có thể là bất kỳ của vùng nào của bộ nhớ.

- ***Thanh ghi địa chỉ bộ nhớ logic (Memory address register and logic)***

Thanh ghi địa chỉ bộ nhớ có độ dài như thanh ghi PC (16 bit), các đầu ra của thanh ghi này được điều khiển nối ra bus địa chỉ của hệ thống máy tính để thực hiện chọn ngăn nhớ, hoặc để chọn cổng ngoại vi nào đó. Trong chu kỳ đọc lệnh, một lệnh máy được đọc từ bộ nhớ. Nội dung của thanh ghi địa chỉ ngăn nhớ và nội dung của thanh đếm lệnh PC lúc này là như nhau, nghĩa là thanh ghi địa chỉ bộ nhớ trỏ tới từ (Word) lệnh đang được đọc từ bộ nhớ. Thanh ghi địa chỉ bộ nhớ không thể tự động tăng hay giảm nội dung mà nó nhận địa chỉ lệnh từ PC, từ SP và thanh ghi chỉ số. Một số lệnh đặc biệt cho phép thiết lập trực tiếp nội dung thanh ghi địa chỉ bộ nhớ. Trong hầu hết các họ vi xử lý 8 bit thanh ghi địa chỉ bộ nhớ có độ dài 16 bit. Tuy nhiên một số vi xử lý có thanh ghi này có độ dài 32 hoặc 64 bit.

- ***Thanh ghi lệnh IR (Instruction Register)***

Thanh ghi lệnh chứa lệnh đang thực hiện. Trong chu kỳ đọc mã lệnh (Instruction Cycle) từ bộ nhớ, mã lệnh được nạp vào thanh ghi lệnh IR thôngqua bus dữ liệu nội bộ. Thanh IR được coi như là bộ đệm duy trì nội dung mã lệnh. Đầu ra của thanh IR được đưa tới bộ giải mã lệnh (Instruction decode) để tạo ra các tín hiệu điều khiển thực hiện lệnh. Độ dài của thanh IR tùy thuộc vào từng bộ vi xử lý, trong một số loại, độ dài của nó bằng độ dài của từ (Word) xử lý. Cũng có thể dựa vào số lượng lệnh của tập lệnh ta có thể đoán được độ dài của thanh IR. Vì 2^N = Số lượng tối đa các lệnh (Maximal Number of instruction), trong đó N = số bit của IR. Đôi khi độ dài của thanh IR chỉ có từ 3 đến 4 bit.

- ***Các thanh ghi dữ liệu tạm thời Temp (Temporary data register)***

Trong sơ đồ cấu trúc bộ vi xử lý các thanh ghi này có ký hiệu là Temp1 và Temp2. ALU không có bộ đệm, do đó khi tính toán, các dữ liệu phải được lấy từ bus dữ liệu hoặc đưa trở lại bus dữ liệu bên trong theo thời điểm nhịp nào đó. Những việc này nếu thiếu thanh ghi dữ liệu tạm thời sẽ không thực hiện được.

Ngoài ra trong quá trình xử lý cần sự ổn định dữ liệu ở đầu vào ALU, do đó cần có thanh ghi đệm để lưu trữ dữ liệu trong thời gian ngắn để cho ALU thực hiện xong phép tính

3.4 Đơn vị điều khiển

Đơn vị điều khiển tạo các tín hiệu điều khiển di chuyển số liệu (tín hiệu di chuyển số liệu từ các thanh ghi đến bus hoặc tín hiệu viết vào các thanh ghi), điều khiển các tác vụ mà các bộ phận chức năng phải làm (điều khiển ALU, điều khiển đọc và viết vào bộ nhớ trong...). đơn vị điều khiển cũng tạo các tín hiệu giúp các lệnh được thực hiện một cách tuần tự.

Việc cài đặt đơn vị điều khiển có thể dùng một trong hai cách sau: dùng mạch điện tử hoặc dùng vi chương trình (microprogram).

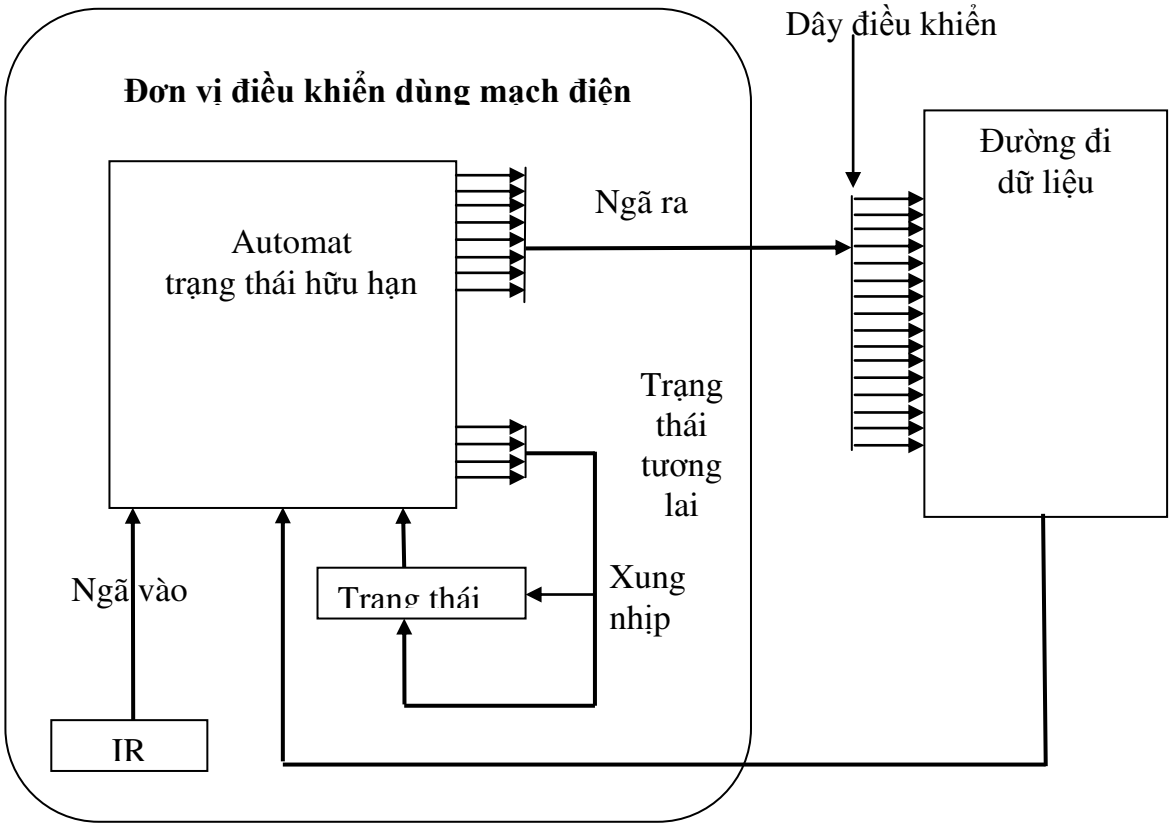
3.4.1. Điều khiển bằng mạch điện tử

Để hiểu được vận hành của đơn vị điều khiển mạch điện tử, chúng ta xét đến mô tả về Automate trạng thái hữu hạn: có nhiều hệ thống hay nhiều thành phần mà ở mỗi thời điểm xem xét đều có một trạng thái (state). Mục đích của trạng thái là ghi nhớ những gì có liên quan trong quá trình hoạt động của hệ thống. Vì chỉ có một số trạng thái nhất định nên nói chung không thể ghi nhớ hết toàn bộ lịch sử của hệ thống, do vậy nó phải được thiết kế cẩn thận để ghi nhớ những gì quan trọng. Ưu điểm của hệ thống (chỉ có một số hữu hạn các trạng thái) đó là có thể cài đặt hệ thống với một lượng tài nguyên cố định. Chẳng hạn, chúng ta có thể cài đặt Automate trạng thái hữu hạn trong phần cứng máy tính ở dạng mạch điện hay một dạng chương trình đơn giản, trong đó, nó có khả năng quyết định khi chỉ biết một lượng giới hạn dữ liệu hoặc bằng cách dùng vị trí trong đoạn mã lệnh để đưa ra quyết định.

Hình 3.3 mô tả nguyên tắc của một đơn vị điều khiển bằng mạch điện. Các đường điều khiển của phần đường đi số liệu là các ngã ra của một hoặc nhiều Automate trạng thái hữu hạn. Các ngã vào của Automate gồm có thanh ghi lệnh, thanh ghi này chứa lệnh phải thi hành và những thông tin từ bộ đường đi số liệu. Ứng với cấu hình các đường vào và trạng thái hiện tại, Automate sẽ cho trạng thái tương lai và các đường ra tương ứng với trạng thái hiện tại. Automate được cài đặt

dưới dạng là một hay nhiều mạch mảng logic lập trình được (PLA: Programmable Logic Array) hoặc các mạch logic ngẫu nhiên.

Kỹ thuật điều khiển này đơn giản và hiệu quả khi các lệnh có chiều dài cố định, có dạng thức đơn giản. Nó được dùng nhiều trong các bộ xử lý RISC.



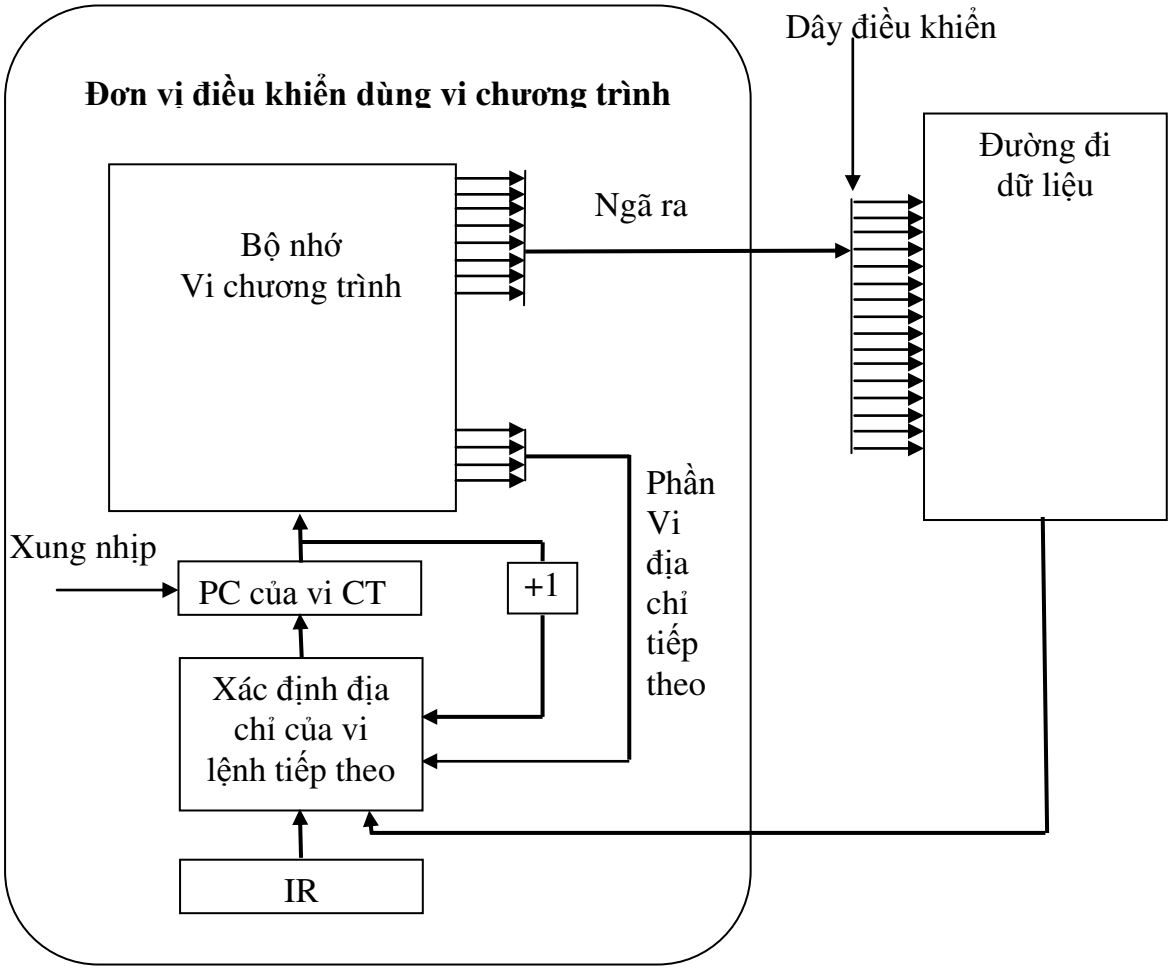
Hình 3.3: Nguyên tắc vận hành của đơn vị điều khiển dùng mạch điện

3.4.2. Điều khiển bằng vi chương trình:

Sơ đồ nguyên tắc của bộ điều khiển dùng vi chương trình được trình bày ở hình 3.4. Trong kỹ thuật này, các đường dây điều khiển của bộ đường đi dữ liệu ứng với các ngã ra của một vi lệnh nằm trong bộ nhớ vi chương trình. Việc điều khiển các tác vụ của một lệnh mã máy được thực hiện bằng một chuỗi các vi lệnh. Một vi máy tính nằm bên trong bộ điều khiển thực hiện từng lệnh của vi chương trình này. Chính vì máy tính này điều khiển việc thực hiện một cách tuần tự các vi lệnh để hoàn thành tác vụ mà lệnh mã máy phải thực hiện. Các tác vụ của lệnh mã máy cũng tuỳ thuộc vào trạng thái của phần đường đi dữ liệu.

Bộ điều khiển bằng vi chương trình được dùng rộng rãi trong các bộ xử lý CISC. Bộ xử lý này có tập lệnh phức tạp với các lệnh có chiều dài khác nhau và có

dạng thức phức tạp. Trong các bộ xử lý CISC, người ta cài đặt một lệnh mã máy bằng cách viết một vi chương trình. Như vậy công việc khá đơn giản và rất hữu hiệu. Các sai sót trong thiết kế automat điều khiển cũng dễ sửa đổi.



Hình 3.4: Nguyên tắc vận hành của đơn vị điều khiển dùng vi chương trình

3.5. Các bộ vi xử lý cơ bản của Intel

3.5.1 Một số đặc điểm của các vi xử lý

Các máy vi tính họ IBM bao gồm các model IBM PC, PC XT, PC AT, PS/1 và PS/2. Chúng đều dựa trên các vi xử lý họ 8086 cuả hãng Intel. Đó là các vi xử lý 8086, 8088, 80286, 80386, 80486 và Pentium. Vi xử lý 8088 được dùng trong PC và PC XT. 80286 dùng trong PC AT và PS/1. Các PS/2 dùng 8086, 80286, 80386 và 80486.

+ Intel 8080 là vi xử lý 8 bit ra đời vào tháng 4 năm 1971, tương đương 8000 transistor, tốc độ 2MHz. Với bus địa chỉ 16 bit có thể địa chỉ được 64Kbyte bộ nhớ.

+ Intel giới thiệu 8086 lần đầu tiên vào tháng 6 năm 1978. 8086 là vi xử lý 16 bit. Nó chạy ở tốc độ 4.77MHz. Với 20 đường địa chỉ, 8086 truy xuất được 1MB (Mega Byte). 8088 được giới thiệu vào tháng 6 năm 1979. Bên trong 8088 giống hệt như 8086 chỉ khác là đường số liệu bên ngoài của 8088 chỉ có 8 bit. 8086 và 8088 có cùng tập lệnh và nó là tập lệnh cơ bản của họ x86.

+ 80186 và 80188 là 2 bản nâng cấp tương ứng của 8086 và 8088 cộng thêm với sự hỗ trợ của một vài chip. Chúng có thể thực thi một tập lệnh mới so với 8086 và 8088 gọi là tập lệnh mở rộng (Extended instruction set). Tuy nhiên 80186 và 80188 không được sử dụng trong thương mại vì chúng nhanh chóng bị thay thế bởi 80286

+ 80286 được giới thiệu vào tháng 1 năm 1882, nó có khoảng 139.000 transistor, tốc độ 8MHz, có 16 bit. 80286 có những cải tiến cơ bản sau:

a) Có thể hoạt động ở 2 chế độ là chế độ thực (Real mode) và chế độ bảo vệ (Protected virtual address mode). Trong chế độ thực, 80286 hoạt động giống như 8086. Các chương trình viết cho 8086 có thể chạy trên 80286 ở chế độ thực mà không cần phải thay đổi gì. Trong chế độ bảo vệ, 80286 hỗ trợ đa nhiệm (multitasking). Đa nhiệm là chế độ nhiều chương trình có thể thực hiện đồng thời tại cùng một thời điểm với chức năng bảo vệ bộ nhớ. Bảo vệ bộ nhớ là khả năng giữ cho một bộ nhớ đang được dùng bởi một chương trình khỏi bị sử dụng bởi một chương trình khác. Bộ nhớ được địa chỉ nhiều hơn: 80286 trong chế độ bảo vệ có thể địa chỉ đến 16 MB bộ nhớ vật lý. Trong khi đó 8086 và 8088 chỉ có 1 MB bộ nhớ vật lý.

b) Bộ nhớ ảo trong chế độ bảo vệ: Điều này có nghĩa là 80286 có thể dùng bộ nhớ ngoài (đĩa cứng) như là bộ nhớ vật lý để thực thi các chương trình lớn (có thể đến 1GB).

+ 80386 và 80386SX: Tháng 10 năm 1985 Intel giới thiệu 80386 như là bộ vi xử lý 32 bit đầu tiên. Nó có khoảng 275.000 transistor, đồng hồ 16MHz. các phiên bản sau của nó có thể chạy ở tốc độ 20; 25; 33; 40 MHz. Với 32 bit địa chỉ nên có thể quản lý 4GB bộ nhớ vật lý. 80386 có thể sử dụng tới 64TB bộ nhớ ảo. Nó có 1 cache nội nhỏ và nó cũng cho phép làm việc với board mẹ có cache.

Một tính năng mới cuả 386 là khả năng mô phỏng một hoặc nhiều bộ vi xử lý 8086 chạy đồng thời. Điều này làm cho Windows thực sự trở thành một hệ điều hành mạnh. Máy tính được khởi động từ DOS (chế độ thực) rồi chuyển sang chế độ bảo vệ, trong đó nhiều 8086 ảo chạy đồng thời dưới những cửa sổ khác nhau.

80386 SX (Sixteen) là phiên bản cuả 80386. Nó có 16 dường địa chỉ ngoài thay vì 32 đường như 80386. Khi đã có 386SX thì 80386 nguyên thủy được đổi tên thành 80386DX (deluxe).

Intel 80386SL là bản tiết kiệm điện cuả 80386SX. Nó được dùng trong các máy tính xách tay (note book).

+ Intel 80486DX là vi xử lý 32 bit được giới thiệu vào tháng 4 năm 1989. Nó chứa khoảng 1,2 triệu transistor, tốc độ 25MHz. 486 không có đổi mới nào đáng kể so với 386DX nhưng đã áp dụng các kỹ thuật sau: Xử lý ống dẫn cho phép xử lý hầu hết các lệnh trong một xung đồng hồ. Thêm nữa, 486DX có gắn sẵn đồng xử lý toán học (Co-Processor) bên trong nên tốc độ thực hiện lệnh tăng nhiều so với 386DX có cùng đồng hồ.

Bên cạnh 486DX còn có các bản 486DX2 và 486DX4. Đây là các bản mà đồng hồ trên board mẹ chỉ cần bằng 1/2 (DX2) hay bằng 1/3 (DX4) thường là DX4-100, DX4-20. 486SX là phiên bản khác cuả 486DX. 486SX giống như 486DX chỉ khác là nó không có đồng xử lý toán học.

Pentium là vi xử lý 32 bit được giới thiệu vào tháng 5-1993. Pentium có khoảng 3 triệu transistor, đồng hồ 60 MHz. Giống như 486DX, Pentium có 32 đường địa chỉ. Đường số liệu trong là 32 bit, nhưng đường số liệu ngoài là 64 bit. Thế hệ Pentium đầu tiên gọi là K5, được thiết kế theo CISC nhưng đã áp dụng công nghệ sau: Dùng ống dẫn đôi, cấu trúc siêu vô hướng và dự đoán rẽ nhánh. Trong điều kiện lý tưởng Pentium có thể thực hiện 2 lệnh trong 1 chu kỳ máy nên chạy nhanh hơn 486 cùng đồng hồ. Mặt khác K5 vẫn tương thích hoàn toàn với 386 và 486, nghĩa là nó vẫn chạy được với các chương trình viết cho DOS và Windows lúc bấy giờ. Một đổi mới quan trọng nữa là nó có 1 bộ FPU được thiết kế tối ưu do đó nó chạy các phép toán dấu chấm động nhanh gấp 5 lần 486DX2/66. K5 có một cache nội 8KB dùng cho lệnh và một cache nội 8KB khác dùng cho số liệu. Bus 64 bit của

Pentium cho phép dẫn số liệu với tốc độ rất nhanh. Một đĩa cúng dung lượng 528MB có thể truyền vào cho CPU trong vòng 1 giây.

+ Pentium Pro ra đời vào tháng 11-1995 với tốc độ 150 MHz.

+ Pentium MMX (Multimedia Extensions) được giới thiệu vào tháng 1 năm 1997 với tốc độ 166MHz. Cải tiến quan trọng cuả MMX so với Pentium và Pentium Pro là tốc độ bus (đồng hồ xung nhịp điều khiển truyền số liệu trên bus) là 66MHz so với 60 MHz cuả K5. Pentium MMX được thiết kế cho các ứng dụng Multimedia chuyên nghiệp.

+ Pentium II được giới thiệu vào tháng 5-1997 có đồng hồ là 233MHz. Ngoài cấu trúc superscalar như Pentium pro, Pentium II còn có đơn vị MMX. Hoạt động của Pentium II có thể tổng kết như sau:

• CPU lấy lệnh từ bộ nhớ theo thứ tự như chương trình nguyên thủy.

• Lệnh được chuyển thành 1 hoặc nhiều lệnh RICS có độ dài cố định gọi là các micro-operations (micro-ops)

• CPU thi hành các micro-ops bằng ống dẫn superscalar sao cho các lệnh có thể out of order.

Thực tế Pentium II gồm một vỏ CISC và 1 lõi RICS. Lõi RISC là một ống dẫn có 11 tầng (thay vì 4 như x86 và Pentium). Bộ đệm đọc lệnh có thể đọc 32 Byte đồng thời. Bộ giải mã lệnh sẽ chuyển mỗi lệnh thành 1 đến 4 micro-ops dài 118 bit (thay vì 32 bit trong các CPU trước)

+ Pentium III được giới thiệu vào tháng 2 –1999 có đồng hồ 450 MHz.

+ Pentium IV được giới thiệu vào tháng 11 năm 2000, nó có khoảng 42 triệu transistor, hoạt động ở điện áp 1,5V. So với các CPU trước Pentium IV có những thay đổi quan trọng sau:

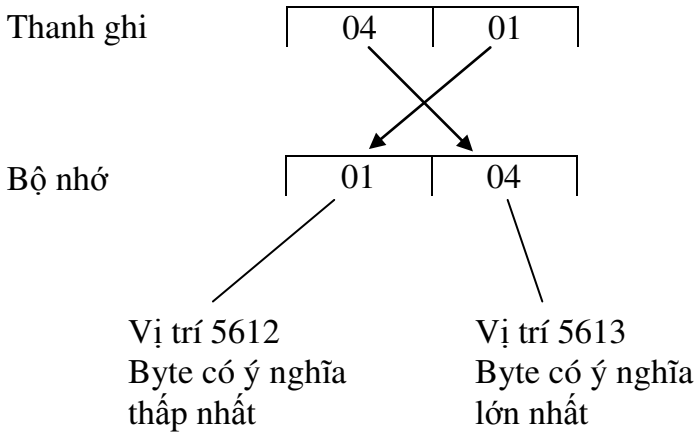
Pentium 4 không phải là kiến trúc x86 mở rộng. Pentium 4 là kiến trúc mới trong đó khai thác tối đa khả năng tốc độ cao của microchip thế hệ mới bằng kỹ thuật xử lý song song. Word lệnh rất dài có thể đến 128 bit, gọi là bó lệnh. Bó lệnh chứa 3 lệnh (120 bit) và 8 bit template. Mỗi lệnh dài 40 bit. CPU có thể lấy 1 bó lệnh tại 1 thời điểm. Trường template được CPU dùng để biết được những bó lệnh nào có thể xử lý song song. Nó còn cho phép nạp dữ liệu từ bộ nhớ trước khi

chương trình yêu cầu. Pentium 4 có nhiều thanh ghi: 128 thanh ghi cho số nguyên và 128 thanh ghi cho dấu chấm động. Tốc độ của Pentium IV vào tháng 8 năm 2002 là 2,5 GHz

3.5.2 Cách ghi dữ liệu lên bộ nhớ

Các byte trong bộ nhớ được đánh số liên tiếp bắt đầu từ 00 sao cho mỗi ô nhớ (hay 1 vị trí nhớ) 1 byte chỉ có duy nhất 1 địa chỉ

Bộ vi xử lý có thể truy xuất 1 hoặc nhiều byte của bộ nhớ ở 1 thời điểm. Ví dụ hãy khảo sát số thập phân 1025; biểu diễn bằng số hexa của nó là 0401h, số này chiếm 2 byte (hay 1 word) nhớ. Word này bao gồm byte cao chứa 04; byte thấp chứa 01. Hệ thống lưu trữ chúng trong bộ nhớ theo trình tự ngược lại: Byte thấp ở địa chỉ thấp và byte cao ở địa chỉ cao trong bộ nhớ. Ví dụ có lệnh cần chuyển giá trị có trong 1 thanh ghi là 0401h vào trong các vị trí của bộ nhớ có địa chỉ 5612 và 5613 thì ta có hình ảnh như sau:



Bộ vi xử lý chứa dữ liệu trong bộ nhớ theo trình tự byte ngược lại với trình tự byte trong thanh ghi và xử lý dữ liệu tương ứng như vậy. Khi bộ vi xử lý lấy lại dữ liệu từ bộ nhớ để nạp vào thanh ghi thì nó cũng đảo ngược để có giá trị chính xác; quá trình này hoàn toàn tự động.

3.5.3 Sơ đồ tổ chức của 8088 và 80286

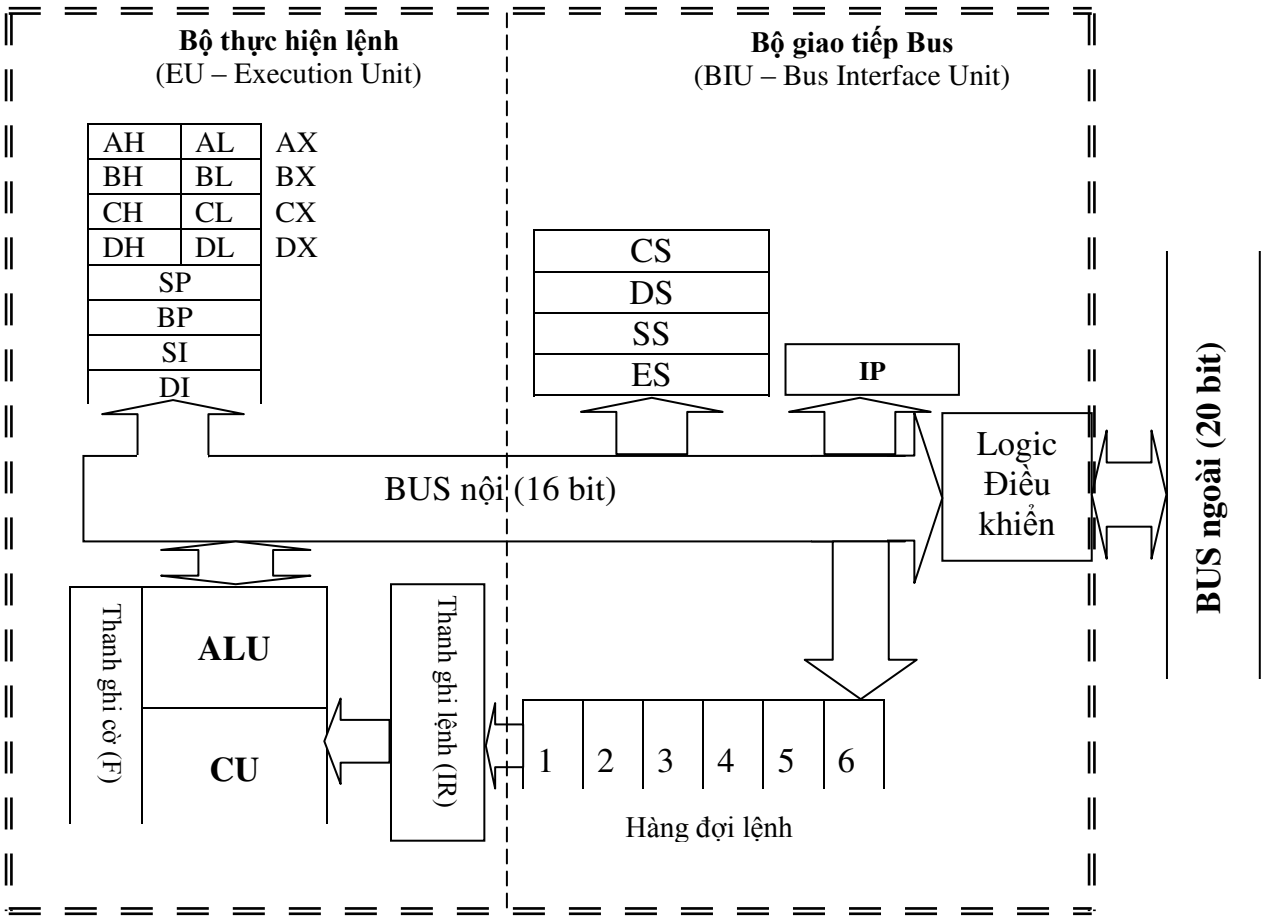
3.5.3.1 Cấu trúc tổng quát

Intel-8086/88 là một CPU 16 bit (bus dữ liệu ngoại có 16 dây). Nó được dùng để chế tạo các máy vi tính PC-AT đầu tiên của hãng IBM vào năm 1981. Trong

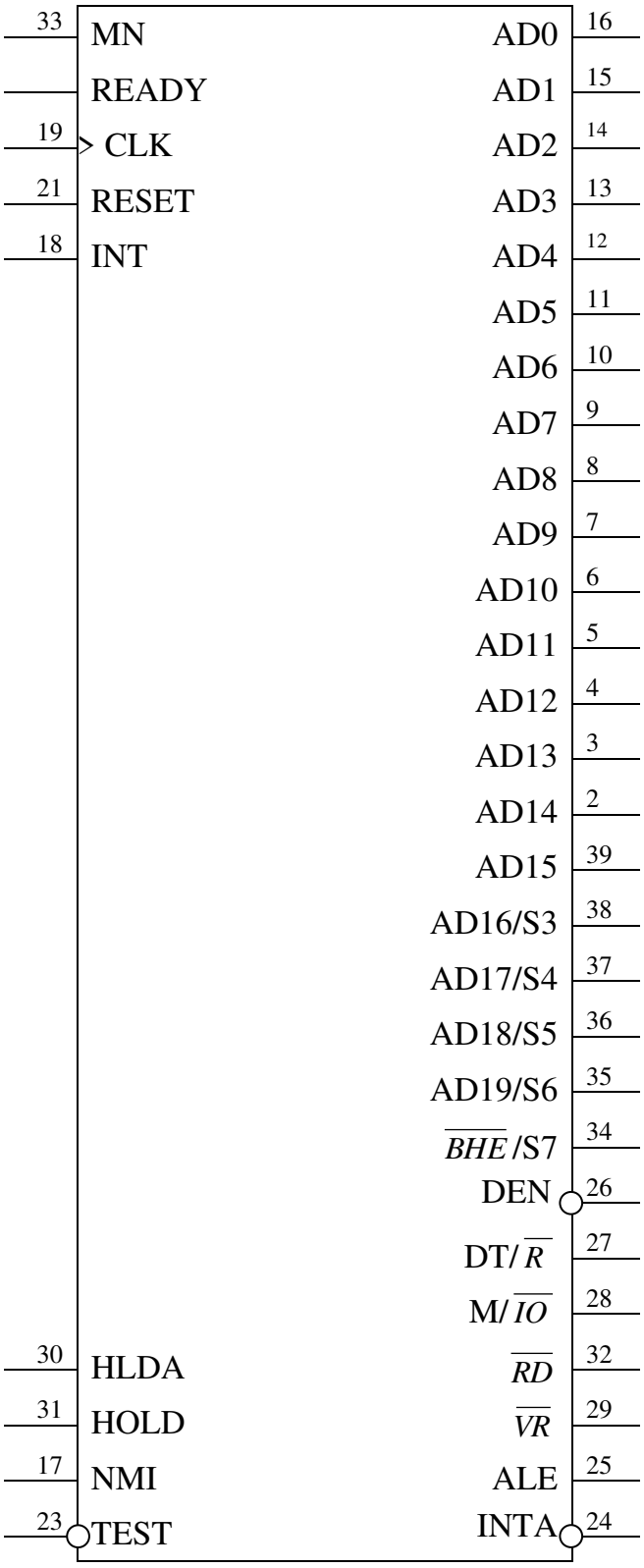
thực tế, hãng IBM đã dùng CPU 8088 (là một dạng của CPU 8086 với bus số liệu giao tiếp với ngoại vi là 8 bit) để chế tạo máy vi tính cá nhân đầu tiên gọi là PC-XT.

CPU 8086 đã không ngừng cải tiến và đã trải qua các phiên bản 80186, 80286, 80386, 80486, Pentium (80586), Pentium Pro, Pentium MMX, Pentium 2, 3, 4. Các CPU trên tương thích từ trên xuống (downward compatible) nghĩa là tập lệnh của các CPU mới chế tạo gồm các tập lệnh của CPU chế tạo trước đó được bổ sung thêm nhiều lệnh khác.

Cấu trúc logic của CPU-8088/86 có dạng như hình 3.5, gồm 2 bộ phận chính là: Bộ thực hiện lệnh EU (Execution Unit) và bộ phận giao tiếp bus BIU (Bus Interface Unit). Hình 3.6 là cấu trúc các chân ra của 8086 ở chế độ MIN. Ở chế độ Min 8086 tự tạo tra các tín hiệu điều khiển bộ nhớ và Bus. Ở chế độ MAX nó cần phải có 1 chip hỗ trợ bên ngoài để giải mã các tín hiệu điều khiển.



Hình 3.5: Sơ đồ khối của bộ vi xử lý 8088/8086



Hình 3.6: Sơ đồ tổng quát của 8086/8088

Các chân ra có ý nghĩa như sau:

AD0-AD15: 16 đường địa chỉ và số liệu dùng chung

DEN (Data Enable): Cho phép số liệu trên Bus

ALE (Address latch Enable): Cho phép địa chỉ trên Bus

M/IO (Memory/IO): Cho phép bộ nhớ hay ngoại vi

RD (Read): Đọc

WR (Write): Viết

INTR (Interrup): Đầu vào ngắt (che được)

MNI (None Maskable Interrup): Đầu vào ngắt không che được

RESET: Reset MPU

CLK (Clock): Lối vào của đồng hồ

8088 có cấu trúc bên trong giống hệt 8086 chỉ khác là bus số liệu ngoài của nó là 8 bit.

3.5.3.2 Bộ phận thực hiện lệnh (EU)

Thi hành các tác vụ mà lệnh yêu cầu như: Kiểm soát các thanh ghi (đọc/ghi), giải mã và thi hành lệnh. Trong EU có Bộ số học và logis (ALU) thực hiện các phép toán số học và logic. Các thanh ghi đa năng là các ô nhớ bên trong CPU chứa dữ liệu tương tự như ô nhớ trong bộ nhớ. Cờ (flag) cũng là một thanh ghi dùng để ghi lại trạng thái hoạt động của ALU. Thanh ghi lệnh chứa nội dung lệnh hiện tại mà CPU đang thực hiện. Các thanh ghi và bus trong EU đều là 16 bit. EU không kết nối trực tiếp với bus hệ thống bên ngoài. Nó lấy lệnh từ hàng đợi lệnh mà BIU cung cấp. Khi có yêu cầu truy xuất bộ nhớ hay ngoại vi thì EU yêu cầu BIU làm việc. BIU có thể tái định địa chỉ để cho phép EU truy xuất đầy đủ 1 MB (8086/88 có 20 đường địa chỉ ngoại).

3.5.3.3 Bộ giao tiếp bus (BIU)

BIU thực hiện chức năng giao tiếp giữa EU với bên ngoài (Bộ nhớ, thiết bị ngoại vi …) thông qua hệ thống BUS ngoại (bus dữ liệu và bus địa chỉ). BIU thực hiện tất cả các tác vụ về bus mỗi khi EU có yêu cầu. Khi EU cần trao đổi dữ liệu với bên ngoài, BIU sẽ tính toán địa chỉ và truy xuất dữ liệu để phục vụ theo yêu cầu của EU. Trong BIU có 5 thanh ghi CS, DS, ES, SS và IP chứa địa chỉ. Thanh ghi IP chứa địa chỉ của lệnh sẽ được thi hành kế tiếp nên gọi là con trỏ lệnh.

EU và BIU liên lạc với nhau thông qua hệ thống bus nội. Trong khi EU đang thực hiện lệnh thì BIU lấy lệnh từ bộ nhớ trong nạp vào hàng đợi lệnh (có 6 bytes). Do đó, EU không phải đợi lấy lệnh từ bộ nhớ. Đây là một dạng đơn giản của cache để tăng tốc việc đọc lệnh.

3.5.3.4 Các thanh ghi của 8086

Thanh ghi (register) là thành phần lưu trữ dữ liệu bên trong CPU, mỗi thanh ghi có độ dài nhất định (16 bit hoặc 8 bit) và được nhận biết bằng một tên riêng. Tùy vào độ dài và chức năng mà thanh ghi có công dụng chứa dữ liệu hoặc kết quả của phép toán, hoặc là các địa chỉ dùng để định vị bộ nhớ khi cần thiết. Nội dung của thanh ghi được truy xuất thông qua tên riêng của nó, do đó tên thanh ghi là từ khóa quan trọng cần phải lưu ý trong lập trình. CPU-8086 có 16 thanh ghi, mỗi thanh ghi có độ dài 16 bit, có thể chia 4 nhóm sau:

*** *Nhóm thanh ghi đoạn (Segment Register):*** Gồm 4 thanh ghi 16 bit: CS, DS, ES, SS. Đây là những thanh ghi dùng để chứa địa chỉ đoạn của các ô nhớ khi cần truy xuất. Mỗi thanh ghi đoạn quản lý 1 đoạn tối đa 64K ô nhớ trong bộ nhớ trong. Người sử dụng chỉ được phép truy xuất ô nhớ dựa vào địa chỉ tương đối. CPU (cụ thể là BIU) có nhiệm vụ chuyển đổi địa chỉ tương đối thành địa chỉ tuyệt đối để truy xuất vào ô nhớ tuyệt đối tương ứng trong bộ nhớ.

CS (Code Segment): Thanh ghi đoạn mã lệnh, lưu địa chỉ đoạn chứa mã lệnh chương trình của người sử dụng

DS (Data Segment): Thanh ghi đoạn dữ liệu, lưu địa chỉ đoạn chứa dữ liệu (các biến) trong chương trình.

ES (Extra data Segment): Thanh ghi đoạn dữ liệu thêm, lưu địa chỉ đoạn chứa dữ liệu thêm trong chương trình.

SS (Stack Segment): Thanh ghi đoạn ngăn xếp, lưu địa chỉ đoạn của vùng ngăn xếp.

Thông thường bốn thanh ghi đoạn có thể chứa những giá trị khác nhau, do đó chương trình có thể được truy cập trên bốn đoạn khác nhau và chương trình chỉ có thể truy cập cùng 1 lúc tối đa bốn đoạn. Mặc khác, đối với những chương trình nhỏ,

chỉ sử dụng 1 đoạn duy nhất, khi đó cả bốn thanh ghi đều chứa cùng giá trị địa chỉ đoạn.

**** Nhóm thanh ghi đa năng (General Register):*** Bao gồm bốn thanh ghi đa năng 16 bit (AX, BX, CX, DX). Mỗi thanh ghi đa năng có thể được sử dụng với nhiều mục đích khác nhau, tuy nhiên từng thanh ghi có công dụng riêng của nó.

AX (Accumulator): Được chia thành 2 phần là AH và AL, mỗi phần 8 bit. AX còn được gọi là thanh ghi tích lũy cơ bản. Mọi tác vụ vào/ra đều dùng thanh ghi này, tác vụ dùng số liệu tức thời, một số tác vụ chuỗi ký tự và các lệnh tính toán đều dùng thanh AX.

BX (Base register): Được chia làm 2 phần là BH và BL. BX còn được gọi là thanh ghi nền, dùng để tính toán địa chỉ ô nhớ.

CX (Count register): Được chia làm 2 phần CH và CL. Nó cón được gọi là thanh ghi đếm, thường được dùng để đếm số lần lặp trong vòng lặp hoặc lệnh xử lý chuỗi ký tự.

DX (Data register): Được chia làm 2 phần DH và DL. Nó còn được gọi là thanh ghi dữ liệu, thường chứa địa chỉ của một số lệnh vào/ra, lệnh tính toán số học (kể cả lệnh nhân và chia).

**** Nhóm thanh ghi con trỏ và chỉ số (Pointer and Index register):*** Chức năng chung của nhóm thanh ghi này là chứa địa chỉ độ dời của ô nhớ trong vùng dữ liệu hay ngăn xếp.

SI (Source Index Register): Thanh ghi chỉ số nguồn

DI (Destination Index Register): Thanh ghi chỉ số đích

BP (Base Pointer Register): Thanh ghi con trỏ nền dùng để lấy số liệu từ ngăn xếp.

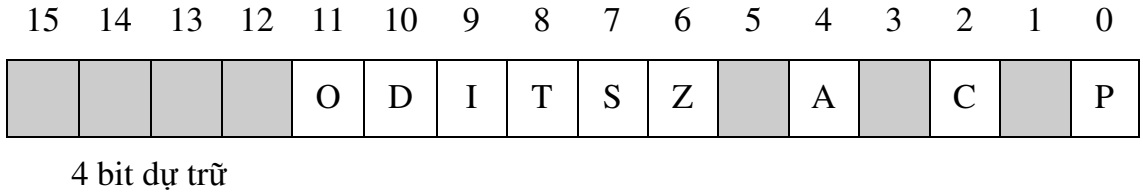
SP (Stack Pointer Register): Thanh ghi con trỏ ngăn xếp luôn chỉ vào đỉnh ngăn xếp.

SI và DI chứa địa chỉ độ dời của ô nhớ tương ứng trong đoạn có địa chỉ chứa trong DS hoặc ES *(dữ liệu, còn gọi là Biến)*. Còn BP và SP chứa địa chỉ độ dời của ô nhớ tương ứng trong đoạn có địa chỉ chứa trong SS, dùng để thâm nhập số liệu trong ngăn xếp.

*** *Thanh ghi Đếm chương trình và thanh ghi trạng thái (Cờ):***

• Thanh ghi con trỏ lệnh IP (Instruction Register) *(còn gọi là PC – đếm chương trình)* là thanh ghi 16 bit chứa địa chỉ của lệnh kế tiếp mà CPU sẽ thực hiện trong. Các lệnh của chương trình có địa chỉ đoạn trong CS.

• Thanh ghi Cờ (F – Flag register) dài 16 bit, mỗi bit là một cờ. Mỗi cờ có giá trị 1 *(gọi là Set –Đặt)* hoặc 0 *(gọi là Clear – Xóa)*. Hình 3.7 mô tả vị trí 9 bit tương ứng với 9 cờ trạng thái (các bit còn lại dùng cho dự trữ mở rộng khi thiết kế các CPU khác)



Hình 3.7: Thanh ghi cờ (Flag register)

Thanh ghi cờ được chia thành hai nhóm:

+ Nhóm cờ điều khiển bao gồm các cờ dùng để điều khiển sự hoạt động của CPU và giá trị của cờ được thiết lập bằng các lệnh phần mềm. Bảng 3.1 và 3.2 cho biết giá trị của các cờ điều khiển

Bảng 3.1

Tên cờ	**Giá trị**	**Ý nghĩa**
TF (Trap Flag): Cờ bẩy	1	Cho phép chương trình chạy từng bước
IF (Interrup Flag) Cờ ngắt	1	Cho phép ngắt phần cứng
DF (Direction Flag): Cờ hướng)	1	SI và DI giảm 1 cho mỗi vòng lặp

+ Nhóm cờ trạng thái: Bao gồm các cờ phản ánh kết quả thực hiện lệnh cũng như trạng thái của CPU

Bảng 3.2

Tên cờ	**Giá trị**	**Ý nghĩa**
CF (Carry Flag): Cờ nhớ	1	Khi có số nhớ hoặc mượn từ bit cao nhất (bit dấu) trong phép cộng hoặc trừ
PF (Parity Flag): Cờ chẵn lẻ	1	Khi byte thấp của thanh ghi kết quả

		một phép toán có số lượng bit 1 là lẻ
AF (Auxiliary cary Flag): Cờ nhớ phụ	1	Khi có nhớ hoặc mượn từ bit 3 trong phép cộng hoặc trừ. Thường được dùng trong các lệnh với số BCD
ZF (Zero Flag): Cờ zero	1	Khi kết quả của 1 phép toán bằng 0
SF (Sing Flag): Cờ dấu	1	Khi kết quả phép tính là âm
	0	Khi kết quả phép tính là dương
OF (Overflow Flag): Cờ tràn	1	Khi kết quả vượt quá khả năng tính toán của CPU

3.5.3.5 Trạng thái tràn

Trạng thái tràn có thể không xảy ra (nếu không tràn) hoặc xảy ra (chỉ tràn dấu, tràn không dấu, đồng thời tràn cả dấu và tràn cả trị). Nói chung là có 3 trạng thái tràn là: chỉ tràn không dấu; tràn trị; tràn cả dấu và trị

Lưu ý: Nếu một giá trị có bit cao nhất (bit dấu) hay còn gọi là MSB bằng 1 thì CPU luôn luôn cho đó là số có dấu.

**** Tràn không dấu: CF=1***

Ví dụ: ADD AX, BX ; với AX = 0FFFFh, BX = 0001h

Ta biểu diễn phép cộng trên dưới dạng nhị phân và chú ý rằng thanh ghi AX dài 16 bit.

```
    1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
+   0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
  ---------------------------------
1   0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Nếu coi kết quả dưới dạng không dấu thì trên AX = 0000h là sai; kết quả đúng phải là 100000000h = 65536d. Nhưng nếu xem rằng phép cộng trên là phép cộng của 2 số có dấu thì kết quả trên là đúng; vì FFFF = -1; còn 0001h = +1; do vậy kết quả phải bằng 0.

- Nếu xem đây là các số không dấu thì AX không đủ chứa kết quả nên tràn không dấu, vậy CF = 1

- Nếu xem đây là các số có dấu thì kết quả sẽ là 0 (vì AX = -1) nên không tràn, do đó OF = 0

**** Tràn có dấu: OF = 1***

Ví dụ: ADD AX, BX ; với AX = BX = 7FFFh = 32767

- Nếu xem đây là các số không dấu thì AX = 7FFFh + 7FFFh = 0FFFEh = 65534 nên không tràn.

- Nếu xem đây là các số có dấu thì tràn vì kết quả vượt quá phạm vi cho phép đối với số có dấu (cộng 2 số dương, kết quả lại là số âm). Thật sự trong trường hợp này, CPU sẽ làm cho OF = 1.

**** Tràn cả dấu và trị***

Tràn cả dấu và trị thường xảy ra khi làm việc với số bù 2 có dấu. Ví dụ xét lệnh NEG AX; với AX = 8000h

Ta có: 8000h = 100000000000

Bù 1: = 011111111111

Bù 2: = 100000000000

Rõ ràng khi lấy bù 2 của 1 số nhưng trong thanh ghi AX vẫn bằng toán hạng nguồn.

Để đơn giản, giả sử ta làm việc với số bù 2 có dấu dài 8 bit; kết quả đặt vào thanh ghi AL. Gọi C_{67} là cờ nhớ từ bit 6 (gọi là b_6) lên bit 7 (gọi là b_7); trong đó b_7 là MSB và cũng chính là bit dấu (SF) của AL. Quan hệ giữa cờ OF với các cờ CF và C_{67} theo biểu thức sau (phép + là phép OR):

$$OF = CF + C_{67}$$

Nghĩa là khi thực hiện các phép toán với số bù 2 có dấu, hiện tượng tràn sẽ xảy ra (cờ OF = 1) nếu có nhớ từ MSB (tức là từ SF) sang CF nhưng lại không có nhớ vào chính nó (SF) hoặc ngược lại. Điều này cũng đúng cho các trường hợp khi làm việc với số bù 2 có dấu có độ dài 16; 32 bit

3.5.3.6 Vi xử lý 80286

Giống như 8086, 80286 là vi xử lý 16 bit. 80286 có tất cả các thanh ghi của 8086 và nó có thể thực thi các lệnh cuả 8086. CPU 80286 được thiết kế với 2 chế độ hoạt động: chế độ địa chỉ thực (real address mode hay real mode) và chế độ địa chỉ

ảo được bảo vệ (protected virtual address mode hay protected mode). Trong chế độ thực, 80286 hoạt động giống như 8086 và nó có thể chạy các chương trình viết cho 8086 mà không cần thay đổi gì. Tập lệnh của 80286 ngoài các lệnh mà 8086 có nó còn một số lệnh mới gọi là tập lệnh mở rộng (Extended instruction set).

Trong chế độ bảo vệ, 80286 hỗ trợ chế độ đa nhiệm (Multitasking). Nó có thể thực hiện một số lệnh cho mục đích đa nhiệm và nó cũng có 1 số thanh ghi dành cho mục đích này

+ Chế độ thực: CPU 8086 có 20 đường địa chỉ, do đó nó truy xuất được 1MB bộ nhớ. CPU 80286 có 24 đường địa chỉ nên nó có thể truy xuất được 16MB bộ nhớ. Các chương trình chạy trong DOS không sử dụng được bộ nhớ mở rộng (từ 1MB trở lên) vì DOS được viết cho 8086/8088. Trong chế độ thực 80286 tạo địa chỉ giống như 8086 nghĩa là nó dùng địa chỉ Segment:Offset 16 bit. Tất nhiên là 80286 có thể truy xuất bộ nhớ trên 1MB. Ví dụ địa chỉ segment là FFFFh và Offset là FFFFh. Địa chỉ vật lý là FFFFh+FFFFh=10FFEFh. Trong 8086 bit tràn bị bỏ qua. Trong 80286 vì nó có 24 đường địa chỉ nên địa chỉ 1FFEFH vẫn được truy xuất. Do đó 80286 truy xuất nhiều hơn 64KB bộ nhớ so với 8086. Vùng nhớ này được dùng đối với các DOS version từ 5.0 trở đi. Vùng nhớ này có tên là vùng nhớ cao (High memory are – HMA).

+ Chế độ bảo vệ (Protected mode): Trong chế độ bảo vệ, 80286 hỗ trợ chế độ địa chỉ ảo (Virtual addressing). Địa chỉ ảo là khả năng làm cho các chương trình có được bộ nhớ lớn hơn bộ nhớ vật lý. 80286 có khả năng định địa chỉ ảo lên đến 1GB. Chế độ bảo vệ còn cho phép đa nhiệm, nghĩa là vài chương trình có thể chạy cùng lúc. 80286 được thiết kế để hoạt động ở chế độ thực khi bật máy. Việc chuyển giữa chế độ thực và chế độ ảo là nhiệm vụ của hệ điều hành.

- Địa chỉ ảo (Virtual address): Các chương trình ứng dụng chạy trong chế độ bảo vệ vẫn dùng đoạn và offset để truy xuất bộ nhớ. Tuy nhiên số đoạn không tương ứng với phân đoạn bộ nhớ mà người ta dùng bộ chọn đoạn (segment selector). Bộ chọn đoạn được sử dụng bởi hệ thống để xác định 1 đoạn vật lý trong bộ nhớ.

- Các nhiệm vụ (Task): Còn gọi là các tiến trình, nó là đơn vị cơ bản để thực thi chương trình trong chế độ bảo vệ. Một task tương tự như 1 chương trình trong chế độ thực. Mỗi task có bảng mô tả cục bộ của mình. Tại 1 thời điểm chỉ có 1 task được thực hiện, nhưng hệ điều hành có thể chuyển giữa các task nhờ một ngắt. Như vậy 1 task có thể gọi 1 task khác.

Trong chế độ bảo vệ, các thanh ghi đoạn không được xem như địa chỉ bắt đầu của đoạn mà là thanh ghi chọn đoạn có gán các ưu tiên khác nhau cho các task. Phần cốt lõi của hệ điều hành có ưu tiên cao nhất và người sử dụng có ưu tiên thấp nhất.

80286 không thể truy cập toàn bộ không gian nhớ ở chế độ thực. Bộ nhớ trên 1MB được gọi là vùng nhớ mở rộng. Thông thường vùng nhớ này không dùng cho các chương trình chạy trên DOS. Để sử dụng vùng nhớ mở rộng người ta dùng 1 chương trình quản lý nó. Đại diện cho chương trình này có tập HIMEM.SYS của DOS.

+ Chế độ ảo: Chế độ này cho phép thiết lập một kiểu vận hành đa nhiệm trong đó các chương trình dùng trong chế độ thực, có thể chạy song song với các tiến trình khác.

3.5.3.7 Vi xử lý 80386 và 80486

CPU-80486DX được đưa ra thị trường năm 1989. Đó là bộ xử lý 32bit chứa khoảng 1.2 triệu transistor. Khả năng quản lý bộ nhớ tối đa giống như 80386 nhưng tốc độ thi hành lệnh đạt được 26.9 MIPS *(Mega Instructions Per Second - triệu lệnh mỗi giây)* tại xung nhịp 33 MHz. Nếu bộ xử lý 80386 là bộ xử lý CISC thuần túy với bộ đồng xử lý toán học 80387 nằm bên ngoài bộ xử lý, thì bộ xử lý 80486 là một bộ xử lý hỗn hợp giữa CISC và RISC, với bộ đồng xử lý toán học và với 8K cache nằm bên trong bộ xử lý 80486.

Trong bộ xử lý 80486, một số lệnh thường dùng, ví dụ như lệnh MOV, dùng mạch điện (kỹ thuật RISC) để thực hiện lệnh thay vì dùng vi chương trình như trong các CPU CISC thuần túy. Như vậy thì các lệnh thường dùng này được thi hành với tốc độ nhanh hơn. Kỹ thuật ống dẫn cũng được đưa vào trong bộ xử lý 80486. Với

các kỹ thuật RISC được đưa vào, bộ xử lý 80486 nhanh hơn bộ xử lý 80386 đến 3 lần (nếu tốc độ xung nhịp là như nhau).

Bộ xử lý 80486 hoàn toàn tương thích với 2 bộ xử lý 80386 và 80387. Do vậy. nó có các chế độ vận hành giống như 80386. Bộ xử lý 80486 tỏ ra rất mạnh đối với các chương trình cần tính toán nhiều và các chương trình đồ họa, vì bộ đồng xử lý toán học nằm ngay trong bộ xử lý 80486. Hàng chờ lệnh của bộ xử lý 80486 là 32 byte.

3.6 Kiến trúc CISC – RISC và Pentium của Intel

3.6.1 Hệ vi xử lý SISC (Complex Instruction Set Computer)

CISC (Complex Instruction Set Computer) có nghĩa là máy tính với tập lệnh đầy đủ. CISC chỉ các máy tính có bộ xử lý được chế tạo theo kiểu đưa toàn bộ tập lệnh vào chi tiết phần cứng. Do đó CPU có cấu trúc phức tạp.

Các kiến trúc với tập lệnh đầy đủ CISC thiết kế từ những năm 1960. Vào thời kỳ này, người ta nhận thấy các chương trình dịch khó dùng các thanh ghi, các vi lệnh được thực hiện nhanh hơn các lệnh; cần thiết phải làm giảm độ dài các chương trình. Với những đặc điểm đó khiến người ta ưu tiên chọn các kiểu ô nhớ - ô nhớ và ô nhớ - thanh ghi, với những lệnh phức tạp và dùng nhiều kiểu định vị. Nếu như các lệnh có chiều dài thay đổi và thì kiểu kiến trúc CSIC không thích hợp. Do vậy, dùng bộ điều khiển vi chương trình là hiệu quả hơn.

Bảng 3.3 mô tả các đặc tính của vài máy CISC tiêu biểu. Ta nhận thấy cả ba máy đều có điểm chung là có nhiều lệnh, các lệnh có chiều dài thay đổi. Nhiều cách thực hiện lệnh và nhiều vi chương trình được dùng.

Bảng 3.3

Bộ xử lý	IBM 370/168	DEC 11/780	iAPX 432
Năm sản xuất	1973	1978	1982
Số lệnh	208	303	222
Bộ nhớ vi chương trình	420 KB	480 KB	64 KB
Chiều dài lệnh (tính bằng bit)	16 - 48	16 - 456	6 - 321
Kỹ thuật chế tạo	ECL - MSI	TTl – MSI	NMOS VLSI

Cách thực hiện lệnh	Thanh ghi- thanh ghi Thanh ghi - bộ nhớ Bộ nhớ - bộ nhớ	Thanh ghi - thanh ghi Thanh ghi - bộ nhớ Bộ nhớ - bộ nhớ	Ngăn xếp Bộ nhớ- bộ nhớ
Dung lượng cache	64 KB	64 KB	0

3.6.2 Hệ vi xử lý RISC (Reduced Instruction Set Computer)

RISC (Reduced Instruction Set Computer) có nghĩa là máy tính có tập lệnh rút gọn, nó chỉ đưa một số ít lệnh chủ yếu vào CPU. Do vậy CPU sẽ có cấu trúc đơn giản cho nên tốc độ làm việc sẽ nhanh hơn.

Tiến bộ trong lĩnh vực mạch kết (IC) và kỹ thuật dịch chương trình làm cho các nhận định trước đây phải được xem xét lại, nhất là khi đã có một khảo sát định lượng về việc dùng tập lệnh các máy CISC. Vào thời điểm này chương trình dịch đã sử dụng được các thanh ghi, không có sự khác biệt đáng kể nào khi sử dụng ô nhớ cho các vi chương trình hay ô nhớ cho các chương trình. Điều này dẫn tới việc hình thành một máy tính với tập lệnh rút gọn RISC vào đầu những năm 1980. Các máy RISC dựa chủ yếu trên một tập lệnh cho phép thực hiện kỹ thuật ống dẫn một cách thích hợp nhất, bằng cách thiết kế các lệnh có chiều dài cố định, có dạng đơn giản, dễ giải mã. Máy RISC dùng kiểu thực hiện lệnh thanh ghi - thanh ghi. Chỉ có các lệnh ghi hoặc đọc ô nhớ mới cho phép thâm nhập vào ô nhớ.

Bảng sau diễn tả ba mẫu máy RISC đầu tiên: mẫu máy của IBM (IBM 801) của Berkeley (RISC1 của Patterson) và của Stanford (MIPS của Hennessy). Cả ba máy đó đều có bộ điều khiển bằng mạch điện (không có ô nhớ vi chương trình), có chiều dài các lệnh cố định (32 bits), có một kiểu thi hành lệnh (kiểu thanh ghi - thanh ghi) và chỉ có một số ít lệnh.

Bộ xử lý	**IBM 801**	**RISC1**	**MIPS**
Năm sản xuất	1980	1982	1983
Số lệnh	120	39	55
Dung lượng bộ nhớ vi chương trình	0	0	0
Độ dài lệnh (tính bằng bit)	32	32	32

Kỹ thuật chế tạo	ECL MSI	NMOS VLSI	NMOS VLSI
Cách thực hiện lệnh	Thanh ghi-thanh ghi	Thanh ghi-thanh ghi	Thanh ghi-thanh ghi

Tóm lại, ta có thể định nghĩa mạch xử lý RISC bởi các tính chất sau:

- Có một số ít lệnh (thông thường dưới 100 lệnh).

- Có một số ít các kiểu định vị (thông thường hai kiểu: định vị tức thì và định vị gián tiếp thông qua một thanh ghi).

- Có một số ít dạng lệnh (một hoặc hai)

- Các lệnh đều có cùng chiều dài.

- Chỉ có các lệnh ghi hoặc đọc ô nhớ mới thâm nhập vào bộ nhớ.

- Dùng bộ tạo tín hiệu điều khiển bằng mạch điện để tránh chu kỳ giải mã các vi lệnh làm cho thời gian thực hiện lệnh kéo dài.

- Bộ xử lý RISC có nhiều thanh ghi để giảm bớt việc thâm nhập vào bộ nhớ trong.

Ngoài ra các bộ xử lý RISC đầu tiên thực hiện tất cả các lệnh trong một chu kỳ máy.

* Bộ xử lý RISC có các lợi điểm sau :

- Diện tích của bộ xử lý dùng cho bộ điều khiển giảm nhiều. Như vậy có thể tích hợp thêm vào bên trong bộ xử lý các thanh ghi, các cổng vào ra và bộ nhớ cache .....

- Tốc độ tính toán cao do: việc giải mã lệnh đơn giản, có nhiều thanh ghi (ít thâm nhập bộ nhớ), thực hiện kỹ thuật ống dẫn liên tục và hiệu quả (các lệnh đều có thời gian thực hiện giống nhau và có cùng dạng).

- Giảm thiểu thời gian thiết kế bộ điều khiển, điều này góp phần làm giảm chi phí sản xuất.

- Bộ điều khiển trở nên đơn giản, gọn nhẹ. Do đó, ít rủi ro do mắc phải sai sót.

* Kiến trúc RISC có một số bất lợi:

+ Các chương trình dài ra so với chương trình viết cho bộ xử lý CISC. Điều này do các nguyên nhân sau :

- Cấm xâm nhập bộ nhớ đối với tất cả các lệnh ngoại trừ các lệnh đọc và ghi vào bộ nhớ. Do đó buộc người lập trình phải dùng nhiều lệnh để làm một công việc nhất định.

- Cần phải tính các địa chỉ hiệu dụng vì không có nhiều cách định vị.

- Tập lệnh có ít lệnh nên các lệnh không có sẵn phải được thay thế bằng một chuỗi lệnh.

- Các chương trình dịch gặp nhiều khó khăn, vì nó có ít lệnh cho nên có ít chọn lựa để diễn dịch các cấu trúc của chương trình gốc. Sự cứng nhắc của kỹ thuật ống dẫn cũng gây khó khăn.

- Có ít lệnh trợ giúp cho ngôn ngữ cấp cao.

Các bộ xử lý CISC trợ giúp mạnh hơn các ngôn ngữ cao cấp nhờ có tập lệnh phức tạp. Hãng Honeywell đã chế tạo một máy có một lệnh cho mỗi động từ của ngôn ngữ COBOL.

Các tiến bộ gần đây cho phép xếp đặt trong một vi mạch, một bộ xử lý RISC nền và nhiều toán tử chuyên dùng. Bộ xử lý 860 của Intel bao gồm một bộ xử lý RISC, bộ làm toán với các số lẻ và một bộ tạo tín hiệu đồ hoạ.

3.6.3 Tổ chức chung của các vi xử lý Pentium

Bộ xử lý Pentium được đưa ra thị trường vào giữa năm 1993, các bộ xử lý Pentium được chế tạo với hơn 3 triệu transistor, thụ hưởng các tính năng kỹ thuật của các bộ xử lý 80486 DX/2 và được có thêm nhiều tính năng mới.

Theo hãng sản xuất Intel, nếu hoạt động ở với xung nhịp 66 MHz thì tốc độ thi hành lệnh của Pentium là 112 MIPS thay vì 54 MIPS cho bộ xử lý 80486 DX/2 cùng xung nhịp.

Các tính năng nổi bật của bộ xử lý Pentium là :

• Hoàn toàn tương thích với các bộ xử lý được chế tạo trước đó (8086, 80286, 80386, 80486).

• Dùng kỹ thuật ống dẫn tốt hơn với 2 ống dẫn số nguyên độc lập và một ống dẫn số lẻ. Kỹ thuật siêu vô hướng và tiên đoán lệnh nhảy cũng được đưa vào.

• Dùng cache lệnh và cache dữ liệu riêng biệt.

• Bus số liệu là 64 bit với cách vận chuyển theo từng gói.

• Dùng cách quản lý hệ thống cho phép tiết kiệm năng lượng tiêu hao trong bộ xử lý Pentium.

• Có tối ưu hóa các chuỗi mã lệnh. Petium MMX: Các lệnh xử lý multimedia được đưa vào tập lệnh của CPU nên việc thi hành chương trình multimedia được cải thiện rất nhiều.

3.7 Tập lệnh hợp ngữ của họ vi xử lý

Chương trình là một tập hợp các lệnh được đưa vào bộ nhớ trong của máy tính để bộ xử lý thực hiện. Các lệnh có thể được thể hiện ở những dạng (ngôn ngữ) khác nhau.

Bộ xử lý hoạt động dựa trên kỹ thuật số nên chỉ hiểu được những giá trị nhị phân. Do đó, ngôn ngữ mà CPU hiểu được gọi là ngôn ngữ máy (Machine language). Mỗi bộ xử lý có tập giá trị nhị phân, trong đó mỗi giá trị nhị phân sẽ điều khiển CPU thực hiện một tác vụ (gọi là mã lệnh), tập hợp các mã lệnh nhị phân gọi là tập lệnh. Những bộ xử lý khác nhau sẽ có tập lệnh khác nhau. Do đó, ngôn ngữ máy của CPU nào thì chỉ thực hiện được trên CPU đó.

Bảng 3.4 trình bày 1 đoạn chương trình ngôn ngữ máy viết cho họ Intel-8086.

Bảng 3.4

TT lệnh	**Giá trị thập lục phân**	**Mã lệnh nhị phân**
1	B4 09	10110100 00001001
2	BA 03 01	10111010 00000011 00000001
3	40	01000000

Đối với CPU có kiến trúc CISC, chiều dài các lệnh có thể khác nhau, do đó trong đoạn chương trình trên gồm 3 lệnh có chiều dài lần lượt là 2 byte, 3 byte và 1 byte. Byte đầu tiên của mỗi lệnh là Tác vụ (Op-Code) mà CPU phải thực hiện, các byte còn lại là Tác tử (Operand) xác định dữ liệu hoặc nơi chứa dữ liệu mà lệnh tác động vào. Để có thể lập trình với ngôn ngữ máy, người lập trình phải hiểu rõ tổ chức phần cứng của máy đang sử dụng. Vì là ngôn ngữ riêng của CPU và được CPU thực hiện ngay khi đọc được lệnh, nên chương trình được viết bằng ngôn ngữ máy thì sẽ được thực hiện rất nhanh và chiếm ít bộ nhớ trong. Tuy nhiên, dạng nhị phân của ngôn ngữ máy rất khó nhớ và dễ nhầm lẫn. Để khắc phục nhược điểm khó

nhớ của dạng nhị phân, người ta dùng những từ gợi nhớ thay cho những mã lệnh nhị phân (gọi là từ gợi nhớ mã lệnh –memonic). Chương trình viết bằng từ gợi nhớ mã lệnh gọi là Hợp ngữ (Assembly).

Mỗi mã lệnh nhị phân có một lệnh tương ứng trong Hợp ngữ và mỗi CPU khác nhau sẽ có Hợp ngữ khác nhau. Các tài liệu tham khảo về tập lệnh của CPU thường được cung cấp cho các lập trình viên là tập lệnh của CPU ở dạng Hợp Ngữ.

3.8 Đa xử lý và các kiến trúc tiên tiến

3.8.1 Cơ chế đường ống (pipeline)

+Quá trình nhận lệnh và thực thi lệnh:

1. BIU xuất nội dung của thanh ghi con trỏ lệnh IP (Instruction Pointer) ra bus địa chỉ để chọn byte hay word đọc vào BIU.

2. Thanh ghi IP được tăng lên để chuẩn bị nhận lệnh kế (số byte tăng lên của IP tùy thuộc vào kích thước lệnh trước đó).

3. Khi lệnh ở trong BIU, nó được đưa sang hàng lệnh (queue). Đây là một thanh ghi lưu trữ dạng FIFO (First In First Out – Vào trước ra trước), dùng cơ chế xử lý xen kẽ liên tục các dòng mã lệnh (kỹ thuật đường ống – pipelining).

4. Giả sử ban đầu hàng lệnh trống, EU sẽ không làm gì cả cho đến khi bắt đầu xuất hiện một lệnh trong hàng, EU sẽ lấy lệnh ra khỏi hàng và bắt đầu thực thi lệnh đó.

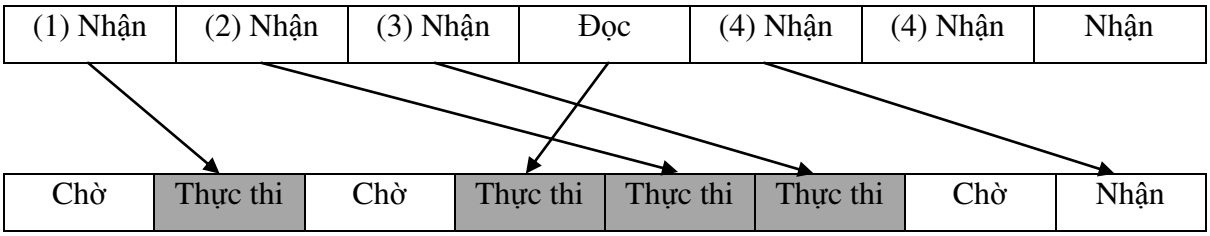
5. Trong khi EU đang thực thi lệnh, BIU tiến hành nhận lệnh mới. Tuỳ theo thời gian thực thi lệnh mà BIU có thể đưa vào hàng lệnh nhiều lệnh mới trước khi EU thực hiện lệnh xong và tiếp tục lấy lệnh mới.

BIU được lập trình để có thể nhận một lệnh mới bất kỳ lúc nào hàng lệnh có chỗ cho 1 byte (8088) hay 2 byte (8086). Lợi ích của phương pháp xử lý theo cơ chế pipeline là EU có thể thực thi các lệnh gần như liên tục thay vì phải đợi BIU nhận thêm lệnh mới.



Hình 3.8

Hình 3.8 là một CPU thông thường dùng chu kỳ nhận và thực thi lệnh tuần tự



Hình 3.9

Hình 3.9 là kiến trúc dạng pipeline của 8086/8088 cho phép thực thi các lệnh mà không bị trễ do quá trình nhận lệnh trong đó:

(1): lệnh thực thi không cần dữ liệu trong hàng đợi lệnh

(2): lệnh thực thi cần dữ liệu trong hàng đợi lệnh

(3): lệnh nhảy

(4): các lệnh bị bỏ qua do lệnh nhảy

+ Có 3 điều kiện làm cho EU ở chế độ chờ:

- Điều kiện thứ nhất xảy ra khi lệnh cần truy xuất đến một vị trí bộ nhớ không ở trong hàng đợi lệnh. BIU phải treo quá trình nhận lệnh và xuất ra địa chỉ của ô nhớ này. Sau khi truy xuất bộ nhớ, EU có thể tiếp tục quá trình thực thi lệnh từ hàng lệnh và BIU có thể tiếp tục đưa các lệnh vào hàng đợi lệnh.

- Điều kiện thứ hai xảy ra khi lệnh được thực thi là lệnh nhảy (jump). Trong trường hợp này, thay vì dùng địa chỉ lệnh kế tiếp, ta phải chuyển đến địa chỉ mới (không tuần tự). Tuy nhiên, BIU vẫn luôn đặt các lệnh theo tuần tự và do đó sẽ lưu các lệnh không sử dụng. Trong khi nhận lệnh kế tiếp tại địa chỉ do lệnh jump chỉ đến, EU phải đợi và tất cả các byte trong hàng phải bỏ.

- Điều kiện thứ ba có thể làm BIU treo quá trình nhận lệnh đó là khi thực thi các lệnh có thời gian thực thi lớn. Như vậy BIU phải đợi cho đến khi lệnh được thực hiện xong và EU nhận mã lệnh từ hàng thì mới có thể tiếp tục quá trình nhận lệnh.

3.8.2 Cơ chế siêu phân luồng (hyper-threading)

Internet, thương mại điện tử và phần mềm ứng dụng doanh nghiệp đang ngày càng đòi hỏi nhiều năng lực tính toán của các máy chủ hơn. Để nâng cao tốc độ, phần mềm cần phải được phân luồng - các chỉ thị sẽ được chia thành nhiều dòng lệnh để có thể xử lý đồng thời trên nhiều bộ xử lý. Intel đã đưa ra kỹ thuật phân luồng cho phép nâng cao tốc độ và khả năng tính toán song song cho những ứng

dụng đa luồng. Công nghệ mới của Intel mô phỏng mỗi bộ vi xử lý vật lý như là hai bộ vi xử lý luận lý (logic), tài nguyên vật lý được chia sẻ và có cấu trúc chung giống hệt nhau cho cả hai bộ xử lý logic. Hệ điều hành và phần mềm ứng dụng sẽ xem như như đang chạy trên hai hay nhiều bộ xử lý, kết quả là tốc độ xử lý trung bình có thể tăng lên xấp xỉ 40% đối với một bộ xử lý vật lý, Intel gọi kỹ thuật này là siêu phân luồng.

Kỹ thuật siêu phân luồng cho phép các phần mềm ứng dụng được viết cho những máy chủ đa luồng có thể thực hiện các chỉ thị song song đồng thời trên mỗi bộ xử lý riêng, bằng cách này sẽ cải thiện tức thì tốc độ giao dịch cũng như thời gian đáp ứng và các yêu cầu đặc thù khác của phần mềm nghiệp vụ và thương mại điện tử. Kỹ thuật này tương thích với các phần mềm ứng dụng và hệ điều hành sẵn có trên các máy chủ (server), nó cho phép hỗ trợ nhiều người dùng hơn và tăng khối lượng công việc được xử lý trên một máy chủ. Với các máy trạm (workstation) cao cấp, kỹ thuật siêu phân luồng cũng sẽ tăng đáng kể tốc độ các phần mềm ứng dụng đòi hỏi năng lực tính toán cao, ví dụ như phần mềm thiết kế 3 chiều, xử lý ảnh hay video… Trong thời gian tới sẽ xuất hiện ngày càng nhiều phần mềm được thiết kế đặc biệt và tối ưu hoá cho Kỹ thuật này.

Từ tháng 01/2002, kỹ thuật siêu phân luồng đã được Intel đưa vào các bộ vi xử lý Xeon đời mới, khởi đầu với các bộ xử lý có tốc độ 1.8GHz và 2.0GHz với 512KB cache thứ cấp, sản xuất bằng công nghệ 0.13 micron (Xeon 1.7GHz, 1.8GHz, 2.0GHz với 256KB cache thứ cấp được sản xuất bằng công nghệ 0.18 không hỗ trợ siêu phân luồng). Tại thời điểm đầu tiên khi Intel giới thiệu bộ xử lý Xeon cùng với chipset 860, chỉ có một số rất ít các nhà sản xuất hàng đầu như IBM, Compaq, Dell, SuperMicro, Tyan… hỗ trợ bộ vi xử lý này, số lượng sản phẩm cũng rất ít. Tuy nhiên, khi có thêm các chipset hỗ trợ bộ xử lý Xeon như E7500 và Serverworks GC, nhiều nhà sản xuất khác đã có sản phẩm hỗ trợ bộ xử lý Xeon. Tuy nhiên, đối với đa số người dùng, nhất là người dùng máy tính để bàn (desktop) thì kỹ thuật siêu phân luồng còn khá xa lạ. Intel chỉ chuẩn bị đưa ra bộ xử lý Pentium IV dành cho desktop áp dụng kỹ thuật siêu luồng (tốc độ khởi điểm là 3.06GHz).

Kỹ thuật siêu phân luồng (hyper-threading) cho phép các ứng dụng đa luồng thực hiện các luồng song song. Trong các kỹ thuật trước, sự phân luồng thực hiện bằng cách cắt các lệnh thành nhiều dòng (stream) khác nhau, mỗi dòng sẽ do một vi xử lý thực hiện (trong hệ thống đa xử lý). Với kỹ thuật siêu phân luồng, sự phân luồng sử dụng các tài nguyên của vi xử lý hiệu quả hơn do quá trình song song là tốt hơn.

Kỹ thuật siêu phân luồng cung cấp trạng thái song song ở cấp độ luồng (TLP – thread level parallelism) cho mỗi vi xử lý, kết quả là gia tăng khả năng tận dụng tài nguyên của vi xử lý. Siêu phân luồng là một dạng của kỹ thuật đa luồng song song (SMT – Simultaneous Multi Threading) trong đó nhiều luồng có thể được thực thi tại cùng một thời điểm trên một vi xử lý. Vấn đề này thực hiện bằng cách kết hợp 2 AS (Architectural State) trong mỗi vi xử lý, các AS sẽ dùng chung tài nguyên của vi xử lý. Kỹ thuật này làm đáp ứng thời gian của vi xử lý sẽ nhanh hơn trong môi trường đa nhiệm và cho phép thực hiện nhanh các hoạt động đa luồng và đa nhiệm bằng cách sử dụng các tài nguyên nhàn rỗi.

3.8.3 Kỹ thuật siêu phân luồng và đa luồng song song (SMT -Simultaneous Multi-Threading)

Intel phát triển SMT từ một công nghệ gốc có tên mã là Jackson với cái tên khác là Hyper-Threading – kỹ thuật siêu phân luồng.

Một chương trình muốn thực thi thì chương trình đó phải được nạp vào bộ nhớ chính; Thanh ghi đếm chương trình (PC- Program Counter) luôn trỏ đến vị trí nơi chứa các mã lệnh sẽ được thực thi. Như vậy một khi chuỗi lệnh được gửi đến CPU thì nội dung thanh PC sẽ bị ghi đè bởi chỉ thị tiếp theo. Chuỗi lệnh có thể bị ngắt bởi một yêu cầu khác, khi đó CPU sẽ lưu giá trị hiện tại của PC trong ngăn xếp (stack) và nạp giá trị mới vào PC, tuy nhiên hạn chế là tại mỗi thời điểm chỉ có thể có duy nhất một chuỗi lệnh được thực thi. Một hướng giải quyết chung cho vấn đề này là sử dụng hai hay nhiều CPU, nếu tại mỗi thời điểm một CPU chỉ có thể thực thi một chuỗi lệnh thì hai hay nhiều CPU sẽ thực thi được hai hay nhiều chuỗi lệnh. Tuy vậy, lại có nhiều vấn đề nảy sinh với cách giải quyết này: Giá thành máy tăng cao, việc quản lý hai hay nhiều CPU để chúng chia sẻ tốt tài nguyên chung là vấn đề

khó. Ví dụ, cho tới trước khi chipset AMD 760MP được đưa ra, tất cả các nền tảng x86 đa xử lý chỉ hỗ trợ việc chia băng thông sẵn có giữa các CPU, điều quan trọng nhất là các ứng dụng và hệ điều hành cần phải có khả năng hỗ trợ tính năng này. Hiện nay, để giải quyết nhanh các chuỗi lệnh phức tạp, phần cứng nói chung phải nhờ vào phương án xử lý đa luồng, hệ điều hành phải hỗ trợ xử lý đa luồng, và phải tăng tốc độ một cách thật sự, giống như có nhiều bộ xử lý (trong hầu hết các trường hợp). Kỹ thuật siêu phân luồng của Intel giải quyết vấn đề bằng cách thực hiện nhiều hơn một chuỗi lệnh tại cùng một thời điểm.

+Hiệu quả của các bộ vi xử lý

Lấy P4 làm ví dụ, CPU này có tổng cộng 7 đơn vị thực thi, hai trong số đó có thể thực hiện hai lệnh mỗi xung clock (gọi là double pumped ALUs). Nhưng ngay cả như vậy thì cũng không thể tìm được phần mềm nào tận dụng hết các đơn vị thực thi đó. Hầu hết các phần mềm cho máy tính cá nhân đang sử dụng chỉ làm việc với một ít phép tính số nguyên như nạp và lưu trữ mà không hề động đến đơn vị thực thi dấu chấm động. Còn một số phần mềm chỉ tập trung vào mỗi đơn vị xử lý dấu chấm động mà không sử dụng đến đơn vị xử lý số nguyên. Ngay cả ứng dụng chủ yếu sử dụng phép tính số nguyên cũng không tận dụng tất cả các đơn vị xử lý số nguyên, đặc biệt là một thành phần trong CPU chuyên dùng cho phép dịch hay quay.

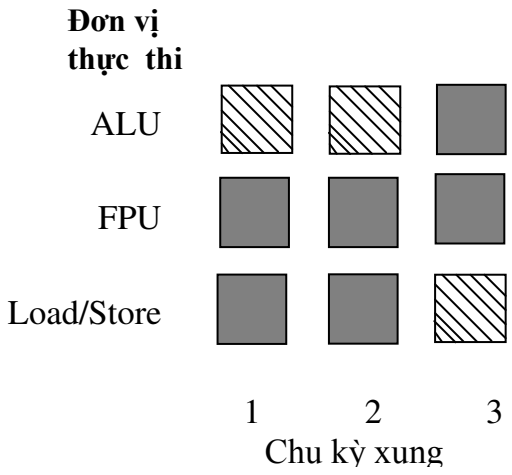
Giả sử một CPU với 3 đơn vị thực thi: một đơn vị làm tóan (ALU – Arithmetic Logic Unit), một đơn vị dấu chấm động (FPU – Floating Point Unit) và một đơn vị nạp/lưu trữ (đơn vị dùng để đọc/ghi bộ nhớ). Giả sử CPU có thể thực hiện mọi lệnh trong vòng một chu kỳ xung clock và đồng thời giải quyết nhiều lệnh tới cả ba đơn vị thực thi. Ta cần CPU thực thi chuỗi lệnh sau:

1+1

10+1

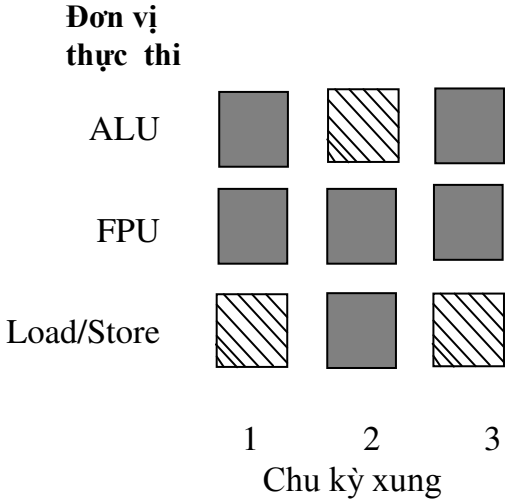
Lưu trữ kết quả

Biểu đồ dưới đây sẽ giúp minh họa mức độ của các đơn vị thực thi, màu xám biểu thị đơn vị thực thi không sử dụng, gạch chéo cho biết đơn vị thực thi hoạt động.



Có thể thấy rằng trong mỗi xung clock sẽ chỉ có 33% trong số các đơn vị được sử dụng, và trong các phép toán này hoàn toàn không sử dụng FPU.

Giả sử gửi một chuỗi lệnh khác đến các đơn vị thực thi của CPU, lần này là các lệnh lấy dữ liệu, cộng và lưu trữ:



Ta thấy rằng cũng chỉ sử dụng có 33% số các đơn vị thực thi. Thuật toán xử lý song song được gọi là ILP (instruction level parallelism), ở đó các chỉ dẫn phức tạp được thực hiện đồng thời bởi vì CPU có khả năng tận dụng các đơn vị xử lý song song, tức là có nhiều hơn 33% số đơn vị xử lý được sử dụng. Tuy nhiên trên thực tế hầu hết các mã lệnh x86 không phải là ILP, vì vậy ta phải tìm những cách khác để tăng hiệu quả. Ví dụ, hệ thống có 2 CPU và chúng có thể thực hiện các chuỗi lệnh đồng thời, cách này được biết đến như là xử lý song song theo luồng để tăng cường hiệu năng, tuy nhiên lại rất tốn kém

+Kỹ thuật siêu phân luồng

Các đơn vị thực thi không được sử dụng thường xuyên là do CPU không thể lấy dữ liệu nhanh như nó mong muốn do tắc nghẽn đường truyền (memory bus và front-side-bus), dẫn đến sự giảm sút hoạt động của các đơn vị thực thi. Ngoài ra, một nguyên nhân khác đã được đề cập là có quá ít ILP trong hầu hết các chuỗi lệnh thực thi.

Hiện thời đa số các phương pháp dùng để cải thiện hiệu năng trong các thế hệ CPU là tăng tốc độ xung clock và tăng độ lớn của bộ nhớ đệm (cache). Nhưng cho dù cả hai cách này cùng được sử dụng thì vẫn không thực sự sử dụng hết được tài nguyên sẵn có của CPU. Nếu có cách nào đó cho phép thực thi được nhiều chuỗi lệnh đồng thời mới có thể tăng hiệu quả sử dụng tài nguyên của CPU. Đó chính là cách mà kỹ thuật siêu phân luồng của Intel đã làm được, bản chất của nó là chia sẻ tài nguyên để sử dụng hiệu quả hơn các đơn vị thực thi lệnh đã có sẵn trên CPU.

Siêu phân luồng là một kỹ thuật nằm ngoài x86, là một phần nhỏ của SMT. Ý tưởng của SMT rất đơn giản: một CPU vật lý sẽ xuất hiện trên hệ điều hành như là hai CPU logic và hệ điều hành không thể phân biệt được. Nhiệm vụ của hệ điều hành là gửi 2 chuỗi lệnh tới 2 CPU và phần cứng sẽ đảm nhiệm những công việc còn lại.

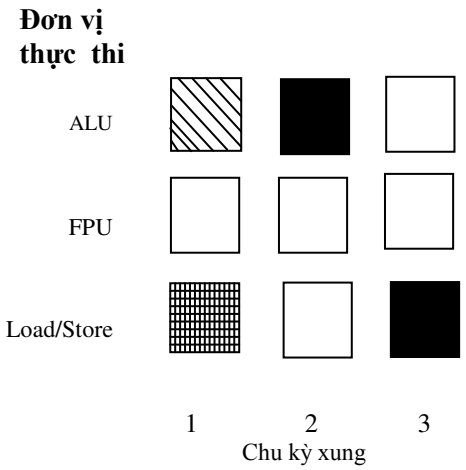
Trong các CPU sử dụng kỹ thuật siêu phân luồng, mỗi CPU logic sở hữu một tập các thanh ghi, kể cả thanh ghi bộ đếm chương trình riêng (separate program counter), CPU vật lý sẽ luân phiên các giai đoạn tìm/giải mã lệnh giữa hai CPU logic và thực thi những thao tác từ hai chuỗi lệnh đồng thời theo cách hướng tới những đơn vị thực thi ít được sử dụng

+Hạn chế của siêu phân luồng

Giả sử rằng CPU đơn giản trước đây cũng có các đặc tính của siêu phân luồng và thực thi 2 chuỗi lệnh đồng thời:

Chuỗi lệnh 1: add, add, store

Chuỗi lệnh 2: Load, add, store



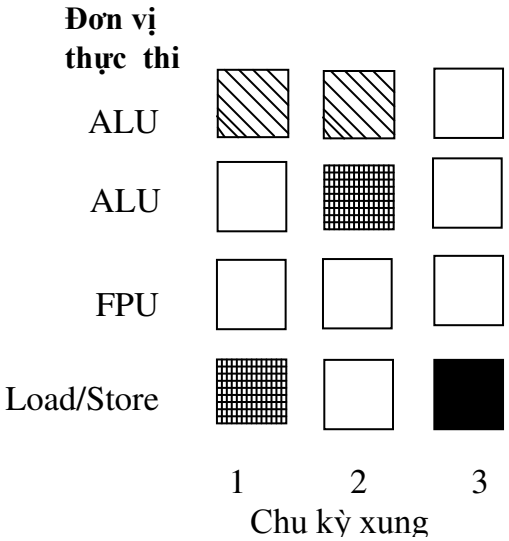
Các ô gạch chéo biểu diễn một chỉ dẫn từ chuỗi lệnh thứ nhất đang được thực hiện, những ô kẻ ca rô biểu diễn một chỉ dẫn từ chuỗi lệnh thứ hai đang được thực hiện. Các ô màu trắng biểu diễn những đơn vị thực hiện không được sử dụng, các ô màu đen biểu diễn sự xung đột khi mà cả hai chỉ dẫn đều sử dụng cùng một đơn vị thực thi. Rõ ràng là việc thực thi song song hai chuỗi lệnh với kỹ thuật siêu phân luồng lại thực hiện chậm hơn so với một CPU thông thường. Nguyên nhân thật ra rất đơn giản: CPU đồng thời thực hiện hai chuỗi lệnh quá đơn giản, tất cả đều là trùng lặp với lệnh add, load, store. Nếu thực thi các ứng dụng đòi hỏi nhiều phép toán động cùng với các ứng dụng số nguyên thì kết quả sẽ khác đi. Hiện tại các ứng dụng văn phòng trên máy tính để bàn hầu như chỉ sử dụng số nguyên (và trong tương lai chắc cũng vẫn chỉ sử dụng số nguyên). Vì vậy lợi ích mà công nghệ siêu phân luồng đem lại thấp (và đôi khi còn kém hơn không dùng công nghệ siêu phân luồng). Trên thực tế, nếu kích hoạt tính năng siêu phân luồng trên desktop, có thể giảm tốc độ tới 10%. Tuy nhiên người dùng các ứng dụng tính toán phức tạp thì sẽ được hưởng lợi rất nhiều từ kỹ thuật này. Ngoài ra kỹ thuật này cũng tăng tốc đáng kể cho các máy chủ, nhất là các máy chủ web server.

+Lợi ích của siêu phân luồng

Intel đã tạo ra siêu phân luồng không chỉ để cho các CPU máy chủ. Thực ra kiến trúc NetBurst của P4 và Xeon hiện nay hoàn chỉnh với lõi SMT. Xét ví dụ ở trên, ta cho thêm một ALU thứ 2 và thực hiện hai chuỗi lệnh trên.

Với một ALU thứ 2, xung đột duy nhất gặp phải là lần lưu trữ cuối cùng. Ta biết rằng CPU P4 được thiết kế với ba đơn vị số nguyên (hai ALU và một đơn vị xử

lý số nguyên khác chậm hơn cho phép dịch/quay). Quan trọng hơn nữa là mỗi ALU của P4 có thể thực hiện hai vi lệnh trong cùng một xung clock, nghĩa là trong hai chỉ dẫn add (phép cộng) mỗi chỉ dẫn có thể từ hai chuỗi lệnh khác nhau, được thực hiện đồng thời trong một xung clock duy nhất trên P4/Xeon.



Nhưng điều đó vẫn chưa giải quyết được vấn đề, do việc tăng thêm các đơn vị xử lý để tăng hiệu quả với kỹ thuật siêu phân luồng lại tốn kém đứng từ quan điểm vật lý (làm cho CPU có nhiều transistor hơn, tiêu tốn nhiều điện năng hơn; hoặc phải giảm kích thước CPU với các công nghệ chế tạo mới). Thay vào đó, Intel đang khuyến khích các nhà phát triển tối ưu hoá kỹ thuật siêu phân luồng. Chẳng hạn sử dụng lệnh dừng (HALT) một trong các bộ xử lý logic sẽ tối đa được tốc độ cho các ứng dụng không sử dụng được kỹ thuật siêu phân luồng, CPU còn lại chỉ hoạt động như là hệ thống một CPU. Khi một ứng dụng có thể sử dụng lợi ích từ siêu phân luồng, bộ xử lý logic thứ hai lại tiếp tục được hoạt động.

Chương 4. BỘ NHỚ CHÍNH

4.1 Các khái niệm cơ bản

Bộ nhớ chứa chương trình, nghĩa là chứa lệnh và số liệu. Người ta phân biệt các loại bộ nhớ: Bộ nhớ trong (RAM-Bộ nhớ vào ra ngẫu nhiên), bộ nhớ chỉ đọc (ROM); bộ nhớ ngoài bao gồm: đĩa cứng, đĩa mềm, băng từ, trống từ, các loại đĩa quang, các loại thẻ nhớ,...

Bộ nhớ RAM có đặc tính là các ô nhớ có thể được đọc hoặc viết vào trong khoảng thời gian bằng nhau cho dù chúng ở bất kỳ vị trí nào trong bộ nhớ. Mỗi ô nhớ có một địa chỉ, thông thường, mỗi ô nhớ là một byte (8 bit), nhưng hệ thống có thể đọc ra hay viết vào nhiều byte (2,4, hay 8 byte). Bộ nhớ trong (RAM) được đặc trưng bằng dung lượng và tổ chức của nó (số ô nhớ và số bit cho mỗi ô nhớ), thời gian thâm nhập (thời gian từ lúc đua ra địa chỉ ô nhớ đến lúc đọc được nội dung ô nhớ đó) và chu kỳ bộ nhớ (thời gian giữa hai lần liên tiếp thâm nhập bộ nhớ).

-*Dung lượng*: khả năng lưu trữ thông tin của thiết bị. Hiện nay người ta thường sử dụng bội của bit là MB để chỉ dung lượng của thiết bị

-*Tốc độ truy nhập*: Là tốc độ truyền dữ liệu của thiết bị. Người ta thường sử dụng MHz (Mêga Hec) để chỉ tốc dộ của thiết bị

-*Giao tiếp*: cấu trúc bên ngoài của bộ nhớ. Ví dụ, các RAM có số chân cắm và đặc tính khác nhau.

4.2 Mô hình phân cấp hệ thống nhớ

Giao tiếp của bộ vi xử lý với bộ nhớ đóng vai trò quan trọng để tạo nên hiệu suất cao cho bất kỳ hệ thống máy tính nào. Bộ nhớ bên ngoài bộ vi xử lý luôn có tốc độ chậm hơn bộ vi xử lý. Để nâng cao tốc độ của hệ thống máy tính với ứng dụng các bộ vi xử lý hiện đại, bộ nhớ được xây dựng theo 5 cấp, từ cấp 0 đến cấp 4 như sau:

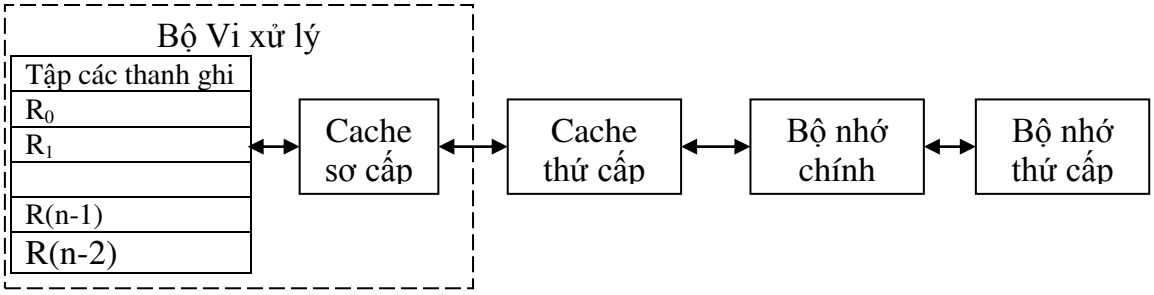
Cấp 0: Tập các thanh ghi bên trong bộ vi xử lý

Cấp 1: Cache sơ cấp (Primary cache): Là bộ nhớ có tốc độ truy cập nhanh nhất, nhưng thường có dung lượng nhỏ. Nó thường nằm bên trong bộ vi xử lý. Trong nhiều bộ vi xử lý cache sơ cấp được tổ chức thành 2 loại (Dual cache): Cache cho lệnh (Icache) và cache cho dữ liệu (Dcache)

Cấp 2: Cache thứ cấp (Second Cache): Cũng là bộ nhớ có tốc độ truy cập nhanh, nhưng dung lượng nhỏ hơn bộ nhớ chính, nó thường nằm ngoài bộ vi xử lý.

Cấp 3: Bộ nhớ chính (Main Memory): Bộ nhớ này được bộ vi xử lý đánh địa chỉ trực tiếp. Nó chứa dữ liệu và các chương trình hoạt động của máy tính. Một phần của chương trình đang chạy có thể được chứa trong cache nhằm tăng tốc độ xử lý. Bộ nhớ chính bao giờ cũng có dung lương lớn hơn bộ nhớ cache. Bộ nhớ chính không thể chứa hết các chương trình và dữ liệu cần thiết vì vậy cần phải có bộ nhớ thứ cấp.

Cấp 4: Bộ nhớ thứ cấp (Second Memory): Bộ nhớ này có dung lượng rất lớn, nhưng tốc độ truy cập dữ liệu chậm. Nó thường được gọi là bộ nhớ ngoài như đĩa từ, băng từ, đĩa CD…



Hình 4.1 - Tổ chức phân cấp bộ nhớ

4.3 Phân loại bộ nhớ và cấu trúc mạch nhớ

4.3.1 Phân loại bộ nhớ

**** Bộ nhớ chỉ đọc (ROM-Read Only Memory):***

Thông tin trong ROM do các nhà sản xuất lưu trữ trên nó. Thông tin trên ROM không thể xoá được và không sửa được, thông tin trong nó sẽ tồn tại mãi mãi khi ROM hoạt động. Như vậy một chip ROM sẽ không có tính mềm dẽo, tính khả dụng không cao, khi cần thay đổi thông tin phải thiết kế, chế tạo lại. Để tăng tính khả dụng và giảm giá thành, chip PROM (Programnable ROM) được chế tạo để thay thế chip ROM. Ban đầu thông tin trong PROM chưa được thiết lập. Thông tin trong PROM sẽ được ghi bằng một thiết bị bán kèm. Như vậy PROM vẫn chưa đáp ứng được nhu cầu, thông tin ghi lên nó chỉ được 1 lần. Chip EPROM (Erasable Programmable ROM) ra đời, chương trình trong EPROM có thể ghi/xóa nhiều lần bằng thiết bị bán kèm. Tuy EPROM đã có tính mềm dẽo hơn nhưng thông tin của

người sử dụng không thể lưu trữ. Do vậy chip EEPROM (Electrically Erasable Programmable ROM) ra đời. Chip này trong máy tính còn gọi là ROM CMOS. Trong EEPROM ngòai chương trình được thiết lập vĩnh viễn người sử dụng còn có thể thiết lập 1 số tham số khi cấu hình máy thay đổi.

Bộ nhớ RAM (Random Access Memory)

RAM và ROM đều là bộ nhớ truy xuất ngẫu nhiên, tức là dữ liệu được truy xuất không cần theo thứ tự. Tuy nhiên ROM có tốc độ chậm hơn RAM rất nhiều. Thông thường ROM cần trên 50ns để xử lý dữ liệu, trong khi đó RAM cần dưới 10ns. RAM có 2 lọai là RAM tĩnh (SRAM - Static RAM) và RAM động (Dynamic RAM-DRAM)

SRAM là loại RAM lưu trữ dữ liệu không cần cập nhật thường xuyên trong khi DRAM là loại RAM cần cập nhật dữ liệu thường xuyên. Thông thường dữ liệu trong DRAM sẽ được làm tươi (refresh) nhiều lần trong một giây để giữ lại những thông tin đang lưu trữ, nếu không thì dữ liệu trong DRAM cũng sẽ bị mất do hiện tượng rò rỉ điện tích của các tụ điện.

** Các khác biệt của SRAM so với DRAM:*

-Tốc độ của SRAM lớn hơn DRAM do không phải tốn thời gian refresh..

-Chế tạo SRAM tốn kém hơn DRAM nên thông thường sử dụng DRAM để hạ giá thành sản phẩm.

** Một số lọai RAM:*

+FPM - DRAM (Fast Page Mode DRAM)

Là một dạng cải tiến của DRAM, về nguyên lý thì FPM - DRAM sẽ chạy nhanh hơn DRAM do cải tiến cách dò địa chỉ trước khi truy xuất dữ liệu. FPM - DRAM hầu như không còn sản xuất trên thị trường hiện nay nữa.

+ EDO - DRAM (Extended Data Out DRAM)

Là một dạng cải tiến của FPM - DRAM, nó truy xuất nhanh hơn FPM - DRAM nhờ một số cải tiến cách dò địa chỉ trước khi truy cập dữ liệu. Tuy nhiên, EDO - DRAM là cần hỗ của chipset hệ thống. Loại bộ nhớ nầy chạy với máy 486 trở lên (tốc độ dưới 75MHz). EDO DRAM cũng đã quá cũ so với kỹ thuật hiện nay, tốc độ của EDO-DRAM nhanh hơn FPM-DRAM từ 10 - 15%.

+ BDEO-DRAM (Burst Extended Data Out DRAM)

Là thế hệ sau của EDO DRAM, dùng kỹ thuật đường ống (pipeline) để rút ngắn thời gian dò địa chỉ.

+ SDRAM (Synchronous DRAM)

Đây là một loại RAM có nguyên lý chế tạo khác hẳn với các loại RAM trước. Đồng bộ (synchronous) là một khái niệm rất quan trọng trong lĩnh vực số. RAM hoạt động do một bộ điều khiển xung nhịp (clock memory), dữ liệu sẽ được truy xuất hay cập nhật mỗi khi clock chuyển từ logic 0 sang 1, đồng bộ có nghĩa là ngay lúc clock nhảy từ logic 0 sang 1 chứ không hẳn là chuyển sang logic 1 hoàn toàn (tác động bằng cạnh xung). Do kỹ thuật này, SDRAM và các thế hệ sau có tốc độ cao hơn hẳn các loại DRAM trước, đạt tốc độ 66, 100, 133 MHz.

+DDR SDRAM (Double Data Rate SDRAM)

Đây là loại bộ nhớ cải tiến từ SDRAM. Nó nhân đôi tốc độ truy cập của SDRAM bằng cách dùng cả hai quá trình đồng bộ khi clock chuyển từ logic 0 sang 1 và từ logic 1 sang 0 (dùng cả cạnh âm và cạnh dương). Loại RAM này được CPU Intel và AMD hỗ trợ, tốc độ vào khoảng 266 MHz. (DDR-SDRAM đã ra đời trong năm 2000)

+ DRDRAM (Direct Rambus DRAM)

Hệ thống Rambus (tên hãng chế tạo) có nguyên lý và cấu trúc chế tạo hoàn toàn khác loại SDRAM truyền thống. Bộ nhớ sẽ được vận hành bởi một hệ thống phụ gọi là kênh truyền Rambus trực tiếp (direct Rambus channel) có độ rộng bus 16 bit và một xung clock 400MHz (có thể lên tới 800MHz). Theo lý thuyết thì cấu trúc mới nầy sẽ có thể trao đổi dữ liệu với tốc độ 400MHz x 16 bit = 400MHz x 2 bytes = 800 MBps. Hệ thống Rambus DRAM cần một chip serial presence detect (SPD) để trao đổi với motherboard. Kỹ thuật mới nầy dùng giao tiếp 16 bit, khác hẳn với cách chế tạo truyền thống là dùng 64 bit cho bộ nhớ nên kỹ thuật Rambus cho ra đời loại chân RIMM (Rambus Inline Memory Module), khác so với bộ nhớ truyền thống. Loại RAM này chỉ được hỗ trợ bởi CPU Intel Pentum IV, tốc độ vào khoảng 400 – 800 MHz

+ SLDRAM (Synchronous - Link DRAM)

Là thế hệ sau của DRDRAM, thay vì dùng kênh Rambus trực tiếp 16 bit và tốc độ 400MHz, SLDRAM dùng bus 64 bit chạy với tốc độ 200MHz. Theo lý thuyết thì hệ thống mới có thể đạt được tốc độ 200MHz x 64 bit = 200MHz x 8 bytes = 1600 MBps, tức là gấp đôi DRDRAM. Điều thuận tiện là là SLDRAM được phát triển bởi một nhóm 20 công ty hàng đầu về vi tính cho nên nó rất da dụng và phù hợp nhiều hệ thống khác nhau.

+ VRAM (Video RAM)

Khác với bộ nhớ trong hệ thống, do nhu cầu về đồ hoạ ngày càng cao, các hãng chế tạo card đồ họa đã chế tạo VRAM riêng cho video card của họ mà không cần dùng bộ nhớ của hệ thống chính. VRAM chạy nhanh hơn vì ứng dụng kỹ thuật Dual Port nhưng đồng thời cũng đắt hơn rất nhiều.

+SGRAM (Synchronous Graphic RAM)

Là sản phẩm cải tiến của VRAM, nó sẽ đọc và viết từng block thay vì từng mảng nhỏ.

+ *Flash Memory*

Là sản phẩm kết hợp giữa RAM và đĩa cứng, bộ nhớ flash có thể chạy nhanh như SDRAM mà và vẫn lưu trữ được dữ liệu khi không có nguồn cung cấp.

4.3.2 Cấu trúc mạch nhớ

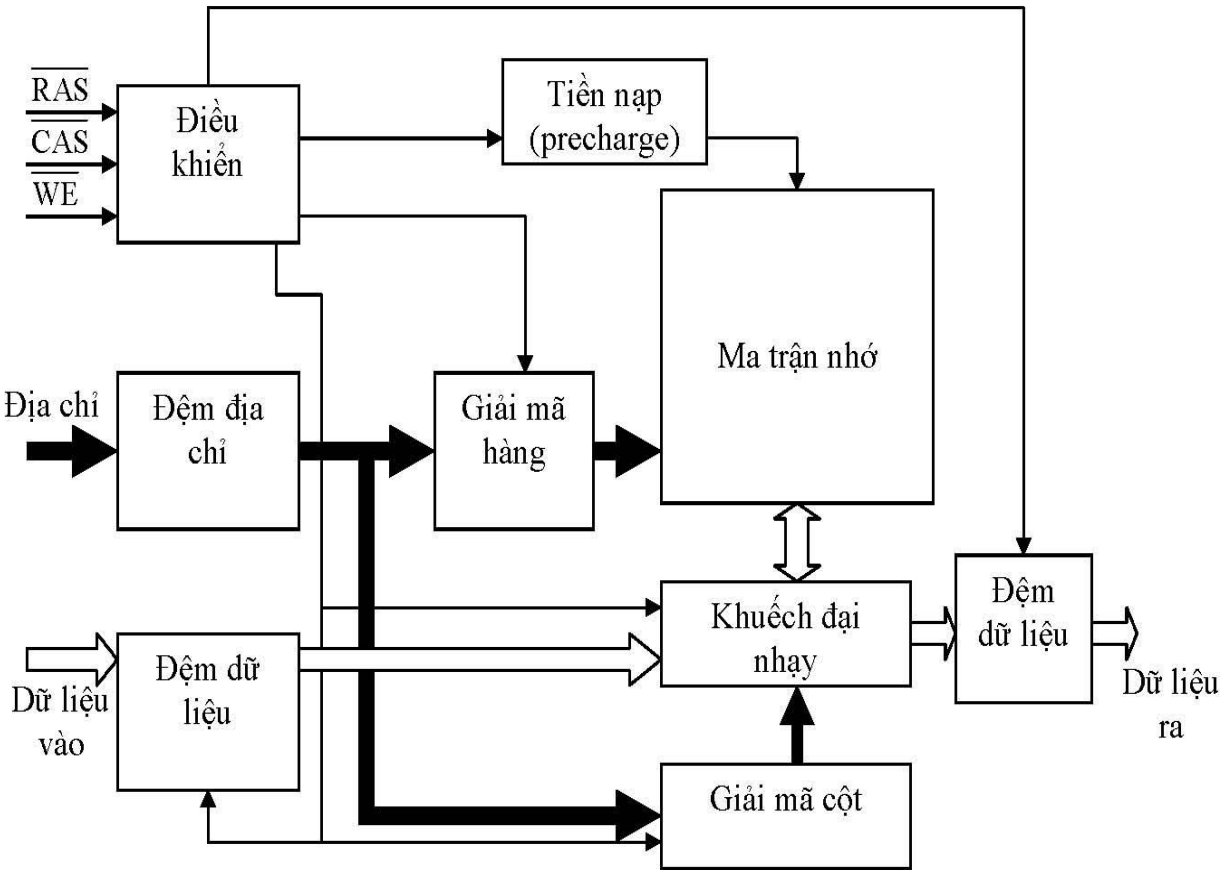
Mỗi loại thiết bị của bộ nhớ trong, thường có cấu trúc khác nhau, ta xét các thiết bị nhớ như sau:

DRAM

+*Cấu tạo của DRAM*

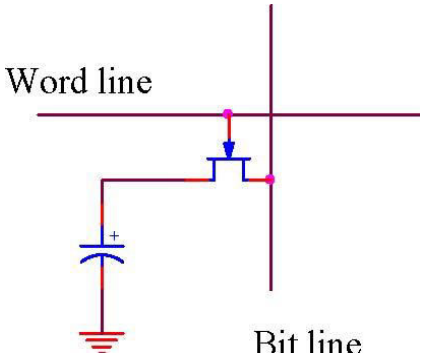
Địa chỉ xác định ô nhớ chia thành 2 phần: địa chỉ hàng và cột. Hai địa chỉ này được đưa lần lượt vào bộ đệm. Quá trình dồn kênh địa chỉ điều khiển bằng các tín hiệu RAS (Row Access Strobe), CAS (Column Access Strobe) và WE (Write Enable). Bộ điều khiển nhớ của CPU phải thực hiện 3 công việc sau: chia địa chỉ từ CPU thành các địa chỉ hàng và cột; tích cực các chân RAS, CAS và WE một cách chính xác; truyền và nhận các dữ liệu đọc, ghi.

Sơ đồ tổ chức của một DRAM như hình 4.2:



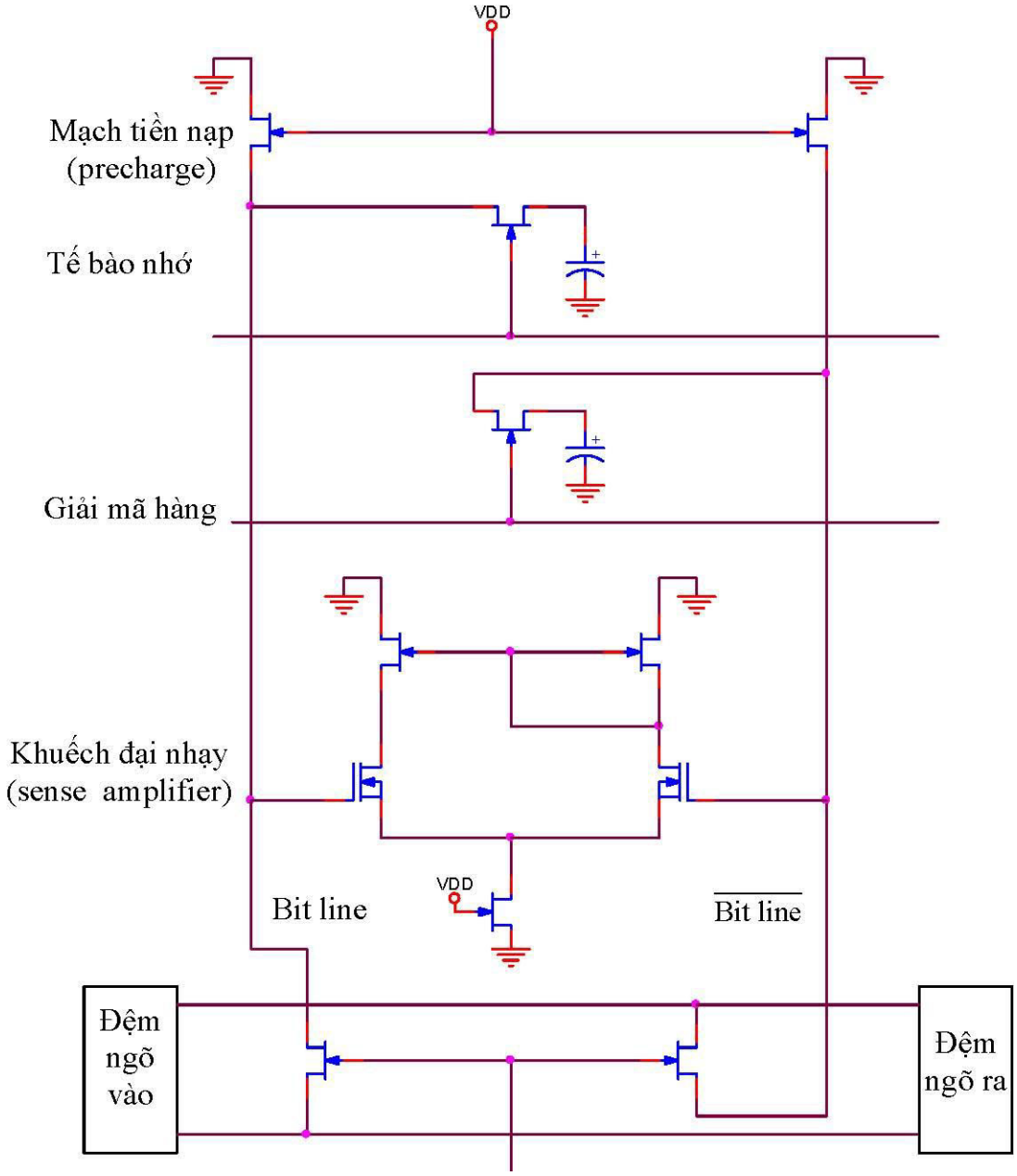
Hình 4.2 – Sơ đồ cấu tạo DRAM

Một ô nhớ của DRAM có thể biểu diễn như hình 4.3:



Hình 4.3 – Cấu tạo một tế bào nhớ DRAM

FET hoạt động như một công tắc, khi đường word tích cực thì cho phép mở FET. Do hiện tượng rò rỉ trên FET, nên sau một thời gian, điện áp trên tụ sẽ giảm dần.



Hình 4.4 – Sơ đồ nguyên lý của DRAM

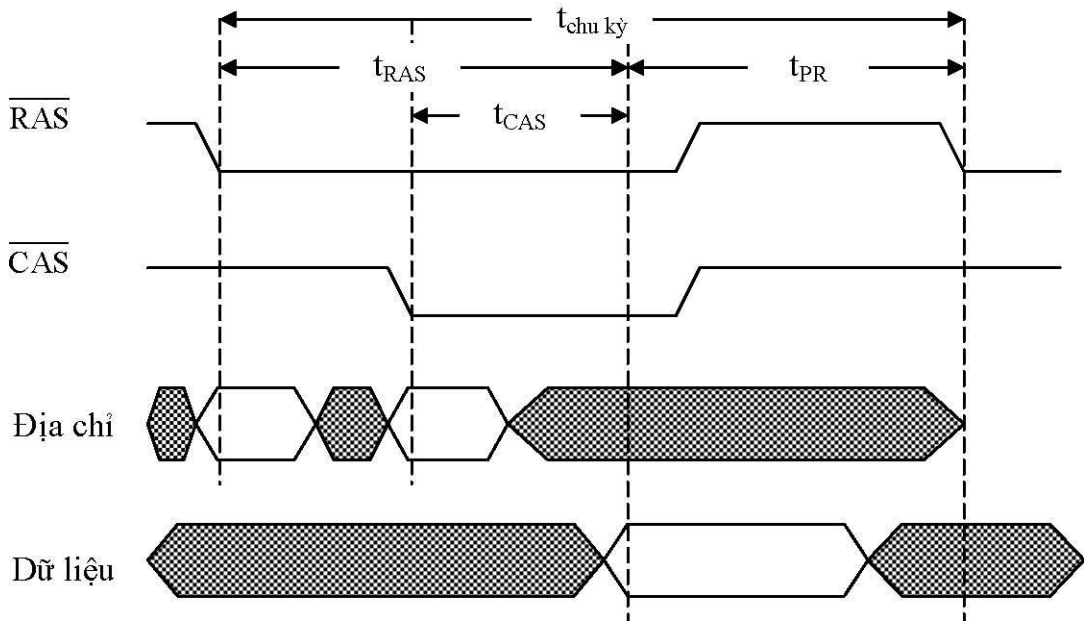
+Quá trình đọc / ghi bộ nhớ

Mạch tiền nạp cho phép xâm nhập và kích hoạt word line khi nạp tất cả các bit line bằng $V_{cc}/2$. Quá trình tiền nạp sẽ dừng khi các cặp dây này có giá trị bằng nhau về điện áp. Thời gian đòi hỏi cho quá trình này gọi là thời gian tiền nạp RAS. Khi quá trình này kết thúc thì mới có thể thực hiện truy xuất ô nhớ. Mạch tiền nạp làm tăng tính ổn định của điện áp ra.

Do điện dung của tụ điện rất nhỏ so với điện dung ghép giữa các bit line nên

điện áp trên các bit line thay đổi nhỏ, khoảng 100 mV. Nếu điện tích của tụ điện ban đầu bằng 0 thì điện thế trên bit line sẽ giảm xuống và kéo điện áp này xuống mức 0. Ngược lại, nếu điện tích khác 0 thì điện thế trên bit line sẽ tăng lên và nâng điện áp này lên V_{cc}. Tín hiệu giải mã cột được cấp sau tín hiệu giải mã hàng để cho phép chọn chính xác cột.

Giản đồ thời gian đọc dữ liệu của DRAM có thể biểu diễn như sau:



Hình 4.5 – Giản đồ thời gian đọc DRAM

t_{RAS} : Thời gian thâm nhập RAS – là thời gian từ lúc đặt địa chỉ hàng cho tới khi dữ liệu ra khỏi bộ đệm.

t_{CAS} : Thời gian thâm nhập CAS – là thời gian từ lúc đặt địa chỉ cột cho tới khi dữ liệu ra khỏi bộ đệm ($t_{CAS} < t_{RAS}$).

t_{PR} : thời gian hồi phục (thời gian tiền nạp RAS) – thời gian cần thiết từ lúc ngõ ra ổn định cho đến khi có thể cung cấp một địa chỉ mới

+ Làm tươi DRAM

Điện tích trên tụ điện bị giảm theo thời gian do chúng rò rỉ qua FET và lớp điện môi oxide làm cho dữ liệu có thể bị mất. Do đó, tụ điện phải được nạp lại một cách tuần hoàn (refresh), thông thường khoảng từ 1 ÷ 16 ms tùy theo loại RAM. Có 3 phương pháp refresh:

-***Chỉ tác động RAS***: tạo chu kỳ đọc giả (dummy read) để làm tươi ô nhớ. Trong chu kỳ giả này, tín hiệu RAS tích cực và địa chỉ hàng được đưa vào DRAM nhưng

CAS bị cấm nên không thể truyền dữ liệu ra ngoài được. Để làm tươi toàn bộ bộ nhớ thì tất cả các địa chỉ phải được cấp lần lượt. Nhược điểm của phương pháp này là cần phải dùng mạch logic hay một chương trình để làm tươi. Trong máy tính, điều này thực hiện bằng kênh 0 của DMAC 8237, tác động định kỳ bằng bộ đếm 1 của PIT 8253.

-***Tác động CAS trước RAS***: DRAM có một mạch logic làm tươi của riêng nó với một bộ đếm địa chỉ. Tín hiệu CAS sẽ tích cực trong một khoảng thời gian trước khi RAS tích cực. Địa chỉ làm tươi được phát ra bên trong bằng bộ đếm địa chỉ mà không cần mạch logic bên ngoài. Sau mỗi chu kỳ làm tươi, bộ đếm địa chỉ sẽ tự động tăng lên 1 để chỉ địa chỉ kế tiếp.

-***Ẩn***: Chu kỳ làm tươi được chứa sau chu kỳ đọc bộ nhớ. Tín hiệu CAS giữ nguyên mức thấp trong khi chỉ có RAS lên mức cao. Ở đây, bộ đếm địa chỉ cũng tự phát ra địa chỉ làm tươi.Việc đọc dữ liệu trong chu kỳ đọc cũng có thể thực hiện ngay cả khi đang thực hiện chu kỳ làm tươi. Phương pháp này sẽ tiết kiệm được thời gian do thời gian làm tươi thường ngắn hơn so với thời gian đọc.

+ Các chế độ hoạt động nhanh của DRAM

DRAM có khả năng thực hiện một hay nhiều chế độ cột để giảm thời gian t_{CAS}.

-***Chế độ trang***: quá trình truy xuất ô nhớ chỉ thay đổi địa chỉ cột. Như vậy, một trang sẽ tương ứng với một hàng trong ma trận nhớ. Để khởi động quá trình đọc, mạch điều khiển nhớ tác động tín hiệu RAS như thông thường nhưng lại bỏ qua địa chỉ hàng. Trong chế độ này, nếu ô nhớ tiếp theo trong cùng một trang thì tín hiệu RAS vẫn giữ liên tục ở mức tích cực. Do đó, thời gian truy xuất có thể giảm xuống 50%.

-***Chế độ cột tĩnh***: tương tự như chế độ trang nhưng tín hiệu CAS giữ nguyên không đổi. DRAM sẽ tự phát hiện sự thay đổi địa chỉ sau một khoảng thời gian CAS không đổi.

-***Chế độ nibble***: thay đổi tín hiệu CAS 4 lần để chuyển 1 nibble dữ liệu.

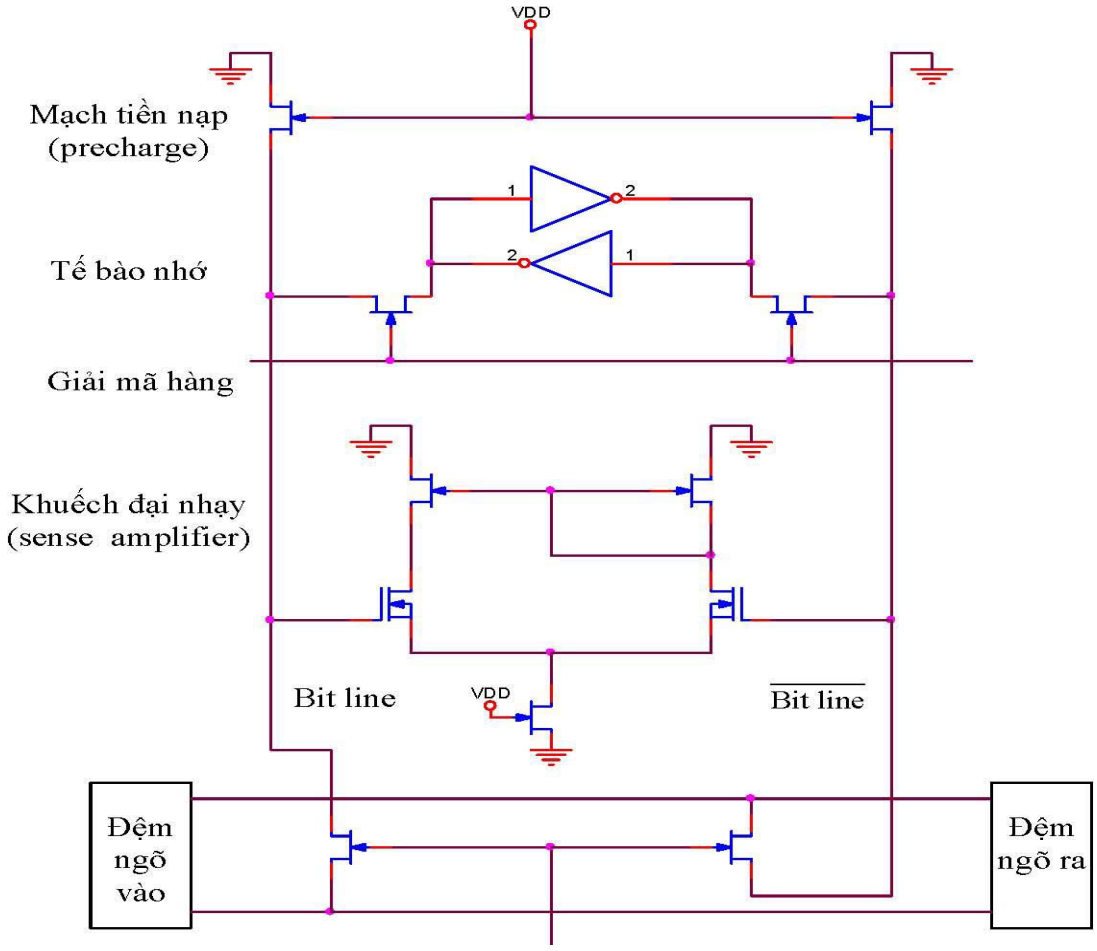
-***Chế độ nối tiếp***: tương tự như chế độ nibble nhưng không phải chỉ chuyển 4 lần trạng thái của tín hiệu CAS. Về nguyên tắc, toàn bộ hàng có thể đưa ra tuần tự.

-***Chế độ xen kẽ***: chế độ này là phương pháp hạn chế trễ do thời gian tiền nạp

RAS. Bộ nhớ chia thành vài bank xen kẽ nhau theo một tỷ số xác định. Dữ liệu có địa chỉ chẵn đặt ở bank 0 và địa chỉ lẻ đặt ở bank 1. Khi đó, thời gian tiền nạp của bank 0 là thời gian truy xuất của bank 1 và ngược lại.

***SRAM**

Đối với SRAM, nội dung ô nhớ vẫn giữ nguyên khi chưa mất nguồn cung cấp mà không cần phải tốn thời gian làm tươi ô nhớ. Do điện áp chênh lệch lớn nên thời gian xử lý khuếch đại sẽ nhỏ hơn trong DRAM (thời gian truy xuất của DRAM là khoảng 1ns trong khi của DRAM khoảng 40ns). Từ đó, SRAM không thực hiện phân kênh địa chỉ hàng và cột (điều này sẽ làm giảm thời gian truy xuất). Sau khi dữ liệu ổn định, bộ giải mã cột chọn cột phù hợp và đưa dữ liệu đến bộ đệm ngõ ra.

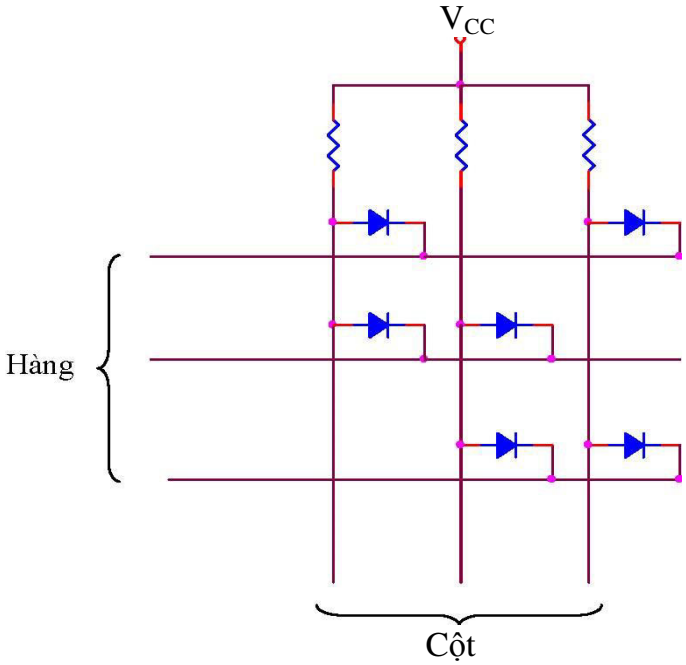


Hình 4.6 – Sơ đồ nguyên lý của SRA

**** ROM***

RAM không thích hợp cho các chương trình khởi động do dữ liệu trên RAM sẽ mất khi không cung cấp điện. Do đó, phải dùng đến ROM, là chip nhớ trong đó dữ liệu sẽ được lưu trữ mãi không cần duy trì nguồn cung cấp.

ROM được chế tạo trên một phiến Silic theo một số bước xử lý như quang khắc và khuếch tán để tạo các tiếp xúc bán dẫn dẫn điện một chiều (diode, FET,…). Người thiết kế xác định chương trình muốn ghi vào ROM và thông tin này dùng để điều khiển trong quá trình tạo ROM. Sơ đồ đơn giản về ROM có thể biểu diễn như hình 4.7



Hình 4.7: ROM đơn giản

Giao giữa một hàng và một cột là một vị trí ô nhớ. Nếu vị trí này tồn tại một diode thì sẽ lưu trữ dữ liệu 0, ngược lại thì lưu trữ dữ liệu 1. Khi đọc một ô nhớ nào đó trên một hàng thì bộ giải mã sẽ đặt hàng đó xuống mức 0, các hàng còn lại ở mức logic 1. Nếu giao giữa hàng và cột không có diode (nghĩa là lưu trữ dữ liệu 1) thì giá trị tương ứng trên cột là 1 (do cột nối với Vcc thông qua điện trở R). Nếu có diode (lưu trữ dữ liệu 0) thì diode sẽ phân cực thuận nên điện áp rơi trên diode khoảng 0.7V điện áp trên cột cũng là 0.7V tương ứng với mức logic 0.

Công nghệ chế tạo ROM được sử dụng là lưỡng cực và MOS. Thời gian truy xuất của bộ nhớ lưỡng cực khoảng từ 50 ÷ 90 ns còn của MOS lớn hơn khoảng 10

lần nhưng bộ nhớ MOS có kích thước nhỏ hơn và tiêu thụ năng lượng ít hơn.

+PROM

Cũng tượng tự như ROM nhưng tất cả các điểm giao giữa hàng và cột đều có một diode mắc nối tiếp với một cầu chì. Khi chưa lập trình, các cầu chì vẫn còn nguyên thì nội dung trên ROM là 0. Khi lập trình, nếu cần bit nào bằng 1 thì ta đặt một xung điện ứng với bit đó, cầu chì sẽ bị đứt và xem như tại vị trí này không có diode (ứng với mức logic)

+EPROM

Đối với EPROM, dữ liệu có thể ghi vào bằng điện và có thể xóa được. EPROM sử dụng một transistor có cấu trúc FAMOST (Floating gate avalanche injection MOS transistor). Đối với transistor loại này, cấu tạo cũng giống như dạng FET thông thường nhưng trong cực gate tồn tại thêm một cực, gọi là cực nổi (floating gate). Nếu cực nổi không có điện tích thì transistor này hoạt động như một FET thông thường, nghĩa là nếu cực Gate có điện áp dương thì FET dẫn, cực drain nối với cực source, cực drain có mức logic 1. Nếu cực nổi có chứa điện tích thì nó sẽ tạo ra trường điện từ đủ sức ngăn chặn không cho FET dẫn cực drain có mức logic 0.

Quá trình nạp điện tử vào cửa nổi được thực hiện bằng các xung điện có độ rộng khoảng 50 ms và biên độ 20V đặt vào cực gate và cực drain. Điện tử được lưu trữ tại vùng cửa nổi khi xung điện tắt (thời gian lưu trữ ít nhất là 10 năm). Để xóa dữ liệu trên EPROM, ta phải chiếu ánh sáng tử ngoại vào chip nhớ. Các điện tử sẽ hấp thu năng lượng đủ để có thể vượt ra khỏi cực nổi làm cho cực nổi không chứa điện tử, toàn bộ EPROM chứa giá trị 1. Do đó, trên các chip EPROM sẽ có một cửa số bằng thạch anh cho phép ánh sáng tử ngoại đi qua.

+EEPROM

Do việc sử dụng cửa sổ thạch anh không tiện lợi nên những năm gần đây đã xuất hiện them chip ROM có thể xóa bằng điện. Cấu tạo của EEPROM cũng giống như EPROM nhưng lúc này có thêm một lớp màng mỏng oxide giữa vùng cực nổi và cực drain cho phép các điện tử di chuyển từ vùng cực nổi sang cực drain khi đặt một điện áp âm. Như vậy, quá trình lập trình cho EEPROM tương tự như EPROM trong khi đó để xóa dữ liệu thì chỉ cần đặt một điện áp -20V vào cực gate và cực

drain với thời gian thích hợp (vì nếu thời gian quá dài thì các điện tử sẽ di chuyển thêm sang cực drain làm cho cực nổi sẽ có điện tích dương FET sẽ không hoạt động như bình thường).

+Bộ nhớ Flash

Bộ nhớ flash có cấu trúc giống như EEPROM nhưng lớp oxide mỏng hơn nên chỉ cần điện áp 12V là có thể xóa được dữ liệu. Bộ nhớ flash có thể hoạt động như RAM nhưng có thể lưu trữ được dữ liệu khi mất nguồn cung cấp. Thành phần chính của flash là ma trận nhớ gồm các ô nhớ FAMOST và không thực hiện phân kênh địa chỉ. Quá trình đọc dữ liệu thực hiện bằng điện áp 5V và lập trình dùng điện thế 12V. Thời gian lưu trữ của flash ít nhất là 10 năm.

4.4 Thiết kê modul nhớ

Bộ nhớ thường được tổ chức từ nhiều vi mạch nhớ ghép lại để có độ rộng bus địa chỉ và dữ liệu cần thiết. Các chip nhớ có đầy đủ chức năng của một bộ nhớ bao gồm:

-Ma trận nhớ: gồm các ô nhớ, mỗi ô nhớ tương ứng với một bit nhớ.

-Mạch giải mã địa chỉ cho bộ nhớ.

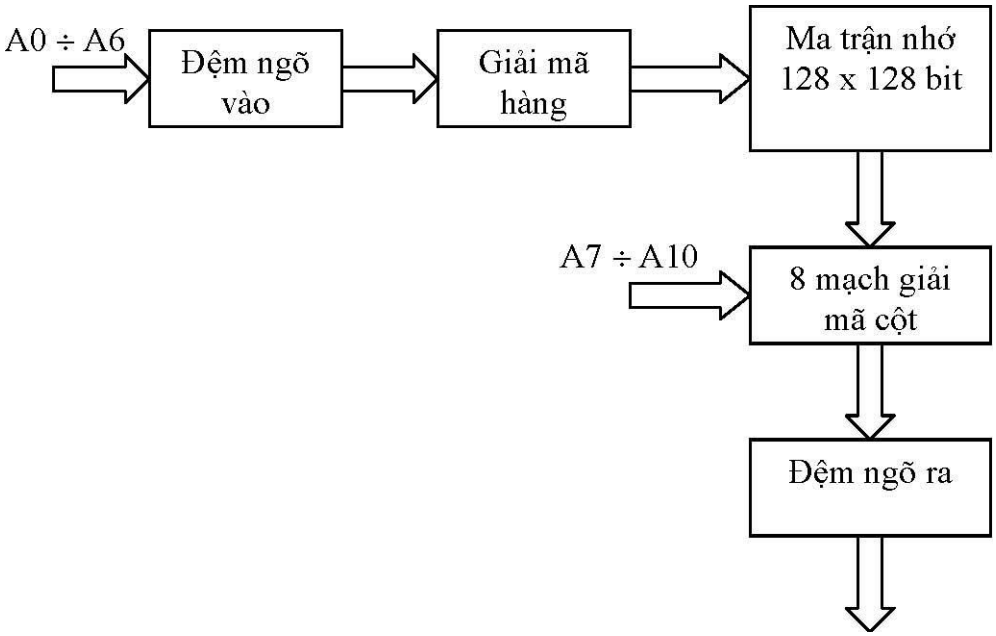
-Mạch logic cho phép đọc.

-Mạch logic cho phép ghi.

-Các mạch đệm vào, ra.

Cách tổ chức đơn giản nhất là tổ chức theo word. Một ma trận nhớ có độ dài của cột bằng số lượng word W và độ dài hàng bằng số lượng bit B của một word. Phương pháp này có thời gian truy xuất ngắn nhưng đòi hỏi bộ giải mã lớn khi tổng số word lớn.

Phương pháp giải mã hai bước cho phép giảm kích thước của phần giải mã địa chỉ bằng cách sử dụng khái niệm word logic và word vật lý. Word vật lý bao gồm tất cả các bit trong một hàng của ma trận và word logic là số bit tương ứng được gởi ra đồng thời. Lúc này, bộ nhớ cần hai mạch giải mã: giải mã hàng để chọn word vật lý và mạch giải mã cột có các mạch dồn kênh (multiplexer) chọn một word logic từ một word vật lý. Ví dụ như: 1 RAM dung lượng 2048 x 8 được tổ chức giải mã 2 bước như hình 4.8:



Hình 4.8: Giải mã hai bước cho bộ nhớ

Ma trận nhớ là 128 x 128 bit, có 128 = 2^7 word vật lý. Một word vật lý được chọn bởi 7 đường địa chỉ từ A0 ÷A6. Bộ giải mã hàng chọn 1 hàng từ 128 hàng.

Một word vật lý được chia làm 16 nhóm 8 bit. Nhóm thứ nhất chứa bit cao nhất của 16 word logic. Nhóm thứ 2 chứa các bit tiếp theo và nhóm cuối cùng chứa các bit thấp nhất. Như vậy mạch giải mã cột gồm 8 bộ dồn kênh: 1 → 16 để cung cấp 1 word logic 8 bit. Các bit địa chỉ từ A7÷A10 đều khiển mạch giải mã cột. Trong trường hợp đặc biệt khi số phần tử trong 1 word vật lý bằng số bit trong 1word vật lý thì đó là bộ nhớ tổ chức theo bit nghĩa là mỗi word logic có độ dài 1 bit.

Các mạch điện ngõ ra đảm bảo không những mức logic mong muốn và cung cấp đủ dòng mà còn có ngõ ra cực thu hở hay ba trạng thái cho phép kết nối chúng với môt vài bộ nhớ khác. Mạch đệm ngõ ra được điều khiển bằng các tín hiệu chọn mạch CS, cho phép bộ nhớ CE, cho phép ngõ ra OE.

Chương 5. QUÁ TRÌNH VÀO RA VÀ THIẾT BỊ NGOẠI VI

5.1 Quá trình vào ra

Trong kỹ thuật máy tính, thuật ngữ giao diện (interfacing) có nghĩa là ghép nối giữa các thiết bị vốn khác nhau về tốc độ và tín hiệu (mức logic) sao cho chúng có thể hiểu được nhau hay nói cách khác chúng có thể trao đổi được số liệu với nhau. Hiểu theo nghĩa hẹp, interfacing là mạch điện tử thực hiện việc ghép nối các IO (Input Output) với bus cuả hệ vi xử lý. Ví dụ để ghép nối máy in với CPU cần phải có bộ giao diện gắn vào bus máy tính để chuyển đổi các thông tin từ CPU thành các ký tự và hình ảnh trên máy in.

Để nối bộ nhớ với các ngoại vi hoặc bộ nhớ và các phần khác cuả máy tính người ta sử dụng tập hợp các đường dây gọi là bus. Khi một IO hoặc một module nhớ được nối với bus, một giao diện phải được sử dụng. Mạch giao tiếp (adapter) bao gồm các mạch chuyển đổi logic và tín hiệu để các thiết bị không tương thích nhau có thể ghép nối với nhau. Như trên đã nói, các thành phần cuả hệ thống máy tính như bộ nhớ, các IO và các thành phần khác phải nối với nhau. Bộ ALU và bộ điều khiển nằm chung với nhau trong một chip gọi là CPU. Trong các máy tính loại cũ, CPU nối trực tiếp với bộ nhớ và IO thông qua các bộ giao tiếp logic (IL = interface logic) dùng cable . Trong các máy tính hiện đại người ta dùng một bus chung duy nhất. Bus này bao gồm một số dây nối và đầu nối.

Trong các hệ thống đơn giản, CPU điều hành toàn bộ quá trình trao đổi thông tin trên bus. Bus là một tập hợp các đường dây để truyền số liệu. Trong cấu hình máy tính chỉ có một bus thì bus bao gồm 3 loại:

* Bus số liệu (data bus): đây là các đường dây mang số liệu. Hiện nay bus số liệu có thể có độ rộng 8,16, 32 hoặc 64 bit.

* Bus địa chỉ (address bus): đây là tập hợp các đường dây dùng để xác định địa chỉ bộ nhớ và các IO. Tuỳ theo máy mà bus địa chỉ có thể là 16, 20, 24, 32, 36 bit.

* Bus điều khiển (control bus): Các đường dây này bao gồm các đường dây để điều khiển thiết bị như đọc, viết, nguồn nuôi ...

Cùng với quá trình phát triển cuả kỹ thuật chế tạo máy tính đã có các chuẩn bus sau đây.

- Bus mở rộng ISA (Industry Standard Architectute) = AT bus. Bus mở rộng AT gồm 2 đoạn khe cắm cách rời nhau, một đoạn 62 chân (XT bus) và một đoạn thêm 36 chân (AT). Bus mở rộng này có 15 đường ngắt, 16 đường số liệu, 24 đường địa chỉ và một số đường điều khiển khác. Năm 1987 Uỷ ban chuẩn quốc gia cuả Mỹ cùng với Viện kỹ thuật điện - điện tử (IEEE) đã đưa ra bộ tiêu chuẩn gọi là ISA bao gồm tất cả các thông tin kỹ thuật cần thiết để tạo ra các loại bus và card mở rộng trên AT. Từ đó AT bus gọi là ISA bus. Bus ISA có tốc độ chậm (5MBps) .

- Bus MCA (Micro Channel Architecture). Bus này do IBM giới thiệu năm 1987 có độ rộng bus số liệu 32 bit. MCA dùng cho các máy tính PS/2 cuả IBM. Đây là bus có tốc độ truyền dữ liệu nhanh hơn ISA (45MBps), khả năng địa chỉ 32 bit. Tuy nhiên MCA không tương thích ngược với ISA.

- Bus EISA: đây là kiểu bus mở rộng ISA được nâng cao (Enhanced ISA) do nhiều công ty máy tính cùng hợp tác xây dựng. EISA tương thích ngược với ISA nhưng có tốc độ nhanh hơn (33MBps).

- Local bus: đây là bus kéo dài trực tiếp bus cuả bộ vi xử lý ra ngoài nhằm đạt được tốc độ cao cho các ứng dụng đồ họa. Vào đầu năm 1990 do hiện tượng thắt cổ chai cuả ISA bus đã thúc đẩy các nhà thiết kế máy tính cho ra đời local bus và ghép nối card màn hình vào bus địa phương này nhằm nâng cao khả năng thể hiện đồ họa. Năm 1992 Hiệp hội chuẩn điện tử và video (Video Electronics Standard Association) đã đưa ra chuẩn VESA nhằm thống nhất các phương pháp ghép nối card video và net card vào local bus. Chuẩn này còn gọi là VL bus (VESA local bus). Tốc độ cuả VESA local bus là 130 - 260MBps.

- Bus PCI (Peripheral Component Interconnect): đây là loại bus mở rộng 32 hoặc 64 bit do Intel xây dựng năm 1992. PCI không phải là local bus nhưng có các đặc trưng cao. PCI tương thích với chuẩn Plug and Play. Tốc độ của PCI là 568MB/s.

- Plus and Play (PnP): là một tiêu chuẩn kỹ thuật cho bus và các ngoại vi. Chuẩn PnP yêu cầu hệ điều hành Windows9X, BIOS thích hợp (flash BIOS) và card mở rộng tương thích (card PCI). Nếu thỏa mãn các yêu cầu trên, việc gắn một ngoại vi vào máy tính sẽ hoàn toàn tự động: máy tính tự động tìm thấy phần cứng

và sẽ hướng dẫn người sử dụng cài đặt phần mềm điều khiển thiết bị mà không sợ bị tranh chấp (conflict).

- Bus SCSI (Small Computer System Interface): Đây là chuẩn bus được đề nghị để dùng cho các ngoại vi có dạng card như modem, sound card và cả hard disk trong các máy tính để bàn và xách tay.

- Bus USB - Hiện nay trên các máy tính đang sử dụng rộng rãi chuẩn USB (Universal Serial Bus) gồm 4 đường dây, tốc độ truyền cỡ 60MBps. Số thiết bị ngoại vi nối với USB có thể đến 127.

5.2 Tổ chức điều khiển vào/ra trong máy tính

Để truy xuất thông tin giữa CPU và các ngoại vi trong cấu hình 1 bus, các thiết bị ngoại vi phải được gán 1 địa chỉ duy nhất. Việc trao đổi số liệu giữa 1 thiết bị IO với CPU bao gồm:

- CPU xác định địa chỉ cuả thiết bị muốn truy xuất

- Thiết bị IO có địa chỉ đã được xác định sẽ tiến hành trao đổi số liệu với CPU thông qua bus hệ thống.

Có 2 phương pháp chủ yếu được sử dụng để chỉ định các thiết bị vào ra. Phương pháp thứ nhất là dùng địa chỉ IO khác với địa chỉ bộ nhớ, phương pháp này gọi là ngoại vi tách biệt (isolated IO). Phương pháp thứ hai gọi là địa chỉ IO dùng chung với bộ nhớ (memory mapped IO).

Trong kỹ thuật IO tách biệt, mỗi thiết bị IO có 1 địa chỉ riêng biệt trên bus. Ví dụ máy in được gán địa chỉ số 3, bàn phím được gán địa chỉ số 4 ... Địa chỉ tương ứng cuả thiết bị sẽ được chuyển lên bus khi CPU trao đổi số liệu với IO. Trong kiểu địa chỉ IO này, CPU phải tạo ra tín hiệu địa chỉ riêng cho IO (input/ouput address bus) và tín hiệu địa chỉ riêng cho bộ nhớ (memory address bus).

Kỹ thuật địa chỉ IO dùng chung với bộ nhớ bao gồm việc các thiết bị IO chia sẽ (share) địa chỉ cùng với địa chỉ bộ nhớ. Nói cách khác các thiết bị IO dùng cùng địa chỉ với địa chỉ bộ nhớ. Trong trường hợp này CPU phải có tín hiệu điều khiển để chỉ ra rằng địa chỉ mà nó phát ra là dành cho bộ nhớ hay cho thiết bị ngoại vi. Đó là các tín hiệu MEMR/ MEMW và IOR/IOW.

5.3 Các phương pháp vào ra số liệu

5.3.1 Vào ra điều khiển bằng ngắt

Việc trao đổi số liệu giữa CPU và các IO thường được thực hiện theo 2 phương pháp: Ngắt và DMA (Direct Memory Access).

Ngắt (interrupt) là một phương pháp trao đổi số liệu giữa các IO có tốc độ chậm với hệ vi xử lý có tốc độ cao. Một ví dụ điển hình cho ngắt là ngắt Keyboard. Bình thường CPU tiến hành công việc cuả mình. Khi một phím được nhấn, một tín hiệu ngắt được gửi tới CPU. Trong chế độ ngắt, CPU tự động dùng chương trình tại vào cuối mỗi lệnh để xem có yêu cầu ngắt không, nếu có CPU sẽ dừng chương trình đang chạy để xử lý ngắt (chạy chương trình con phục vụ ngắt) sau đó quay trở lại chương trình chính.

Nhược điểm cuả ngắt là: CPU có thể bị ngắt bất cứ lúc nào do đó việc phát hiện lỗi trong một chương trình có ngắt trở nên khó khăn. Các chương trình con phục vụ ngắt phải cất giữ trạng thái cuả chương trình chính (nội dung các thanh ghi đang sử dụng, các cờ ...) và phải phục hồi trạng thái chương trình chính sau khi ngắt. Điều này buộc CPU phải thực hiện thêm một số lệnh và làm tăng đáng kể thời gian phục vụ ngắt.

Khi ngắt xảy ra, phải thực hiện các công việc sau :

- *Yêu cầu ngắt* : Yêu cầu ngắt (từ ngoại vi) thường được ghi nhận và chờ đến cuối mỗi chu kỳ lệnh để được xử lý.

- *Chuyển điều khiển đến phục vụ ngắt*: Khi có một ngắt thì CPU sẽ gọi một chương trình con phục vụ ngắt tương ứng để phục vụ .

- *Cất giữ và khôi phục trạng thái chương trình chính*: Tất cả CPU khi trả lời ngắt đều phải cất giữ trạng thái cuả chương trình chính bằng cách dùng ngăn xếp (satck). Địa chỉ đỉnh cuả ngăn xếp được cất trong thanh ghi SP. Quá trình khôi phục trạng thái chương trình chính được thực hiện ngay sau khi đã phục vụ ngắt.

- *Xác định nguồn ngắt*: Cần phải tiến hành xác định nguồn ngắt khi số đầu vào ngắt nhiều hơn số đầu vào ngắt mà CPU có được. Thường người ta dùng cách tạo vector (vectoring) để xác định nguồn ngắt, theo đó tất cả các nguồn ngắt được đưa vào mạch mã hoá để tạo vec tor ngắt (interrupt vector). Vector ngắt chứa địa chỉ cuả chương trình con phục vụ ngắt.

- *Vấn đề ưu tiên (priority)*: Khi có nhiều yêu cầu ngắt được gửi tới CPU cùng một lúc thì phải phân biệt độ ưu tiên. Các IO được gán thứ tự ưu tiên từ cao đến thấp (mức 0 đến mức n).

- *Cho phép và cấm ngắt (EI và DI)*: Có thể cấm ngắt bằng lệnh DI hoặc do CPU tự động cấm ngắt (khi RESET hoặc khi một ngắt đã được ghi nhận). Ngắt bị cấm cho đến khi CPU thực hiện xong lệnh trước lệnh cho phép ngắt (EI). Lệnh DI không tác dụng đối với đầu vào NMI .

Máy tính có 256 ngắt khác nhau và được chia làm 2 loại: Ngắt cứng (hardware interrupt) phát sinh từ các thiết bị phần cứng và ngắt mềm (software interrupt) do lệnh ngắt từ các chương trình.

Người ta còn phân loại ngắt theo nguồn gốc mà chúng được cung cấp. Cách thức thi hành ngắt mềm và ngắt cứng là như nhau. Khi 1 ngắt xảy ra, hệ thống sẽ phải truy cập bảng vector ngắt để lấy địa chỉ cuả chương trình con phục vụ ngắt. Trong PC có 256 ngắt khác nhau. Vùng thấp RAM cuả PC (1024 byte đầu tiên cuả RAM) dùng để lưu giữ địa chỉ cuả 256 chương trình con phục vụ ngắt khác nhau. Mỗi địa chỉ gồm IP và CS (4 byte). Khi một ngắt xảy ra bảng vector ngắt được truy xuất và giá trị thanh ghi CS:IP được dùng để chỉ tới chương trình con phục vụ ngắt. Có 16 ngắt cứng trên PC-AT. Chúng được các IO tạo ra khi có yêu cầu trao đổi số liệu. Trong PC - AT sử dụng 2 chip 8259 để điều khiển ngắt cứng. Chip 8259 là bộ điều khiển ngắt (Interrupt Controller). Chip 8259-1 điều khiển các ngắt INT8 đến INTF, địa chỉ từ 20h đến 3Fh. Chip 8259-2 điều khiển các ngắt INT 70 đến INT77, địa chỉ từ 1C0h đến 1DFh.

Ngắt thường dùng để thực hiện các dịch vụ vào ra cơ sở. Ngắt cũng dùng để điều khiển thiết bị.

5.3.2 Vào ra điều khiển theo DMA

DMA (Direct Memory Access) là một kỹ thuật chuyển dữ liệu nhanh từ một card thiết bị tới bộ nhớ hoặc từ bộ nhớ ra card thiết bị. Trong chu trình DMA, CPU làm công việc cuả mình đồng thời có một chip điều khiển việc truyền số liệu trực tiếp giữa bộ nhớ và IO. Chip này gọi là DMA controller (DMAC).

Trong PC- XT dùng 1 chip 8237 cho DMAC còn trong PC-AT, DMAC gồm 2 chip 8237. Việc lập trình cho DMAC thường được hoàn thành bởi 1 chương trình chạy trên máy tính (ví dụ chương trình chép đĩa, hoặc một thủ tục lưu trong ROM cuả một card giao tiếp). Trong khi DMAC làm việc, CPU được tự do và làm việc khác.

Để chuyển dữ liệu từ thiết bị IO đến bộ nhớ bằng DMA, trước tiên DMAC kích hoạt bus rồi cho lệnh đọc thiết bị (hoạt động đọc cổng IO) và đồng thời tạo ra lệnh ghi bộ nhớ. Việc kích hoạt đọc từ cổng IO sẽ làm cho thiết bị đặt 1 đơn vị thông tin (byte hoặc word) lên bus sau đó dữ liệu này được sao chép trực tiếp vào bộ nhớ. Quá trình diễn ra tương tự khi chuyển số liệu từ bộ nhớ ra IO bằng DMA.

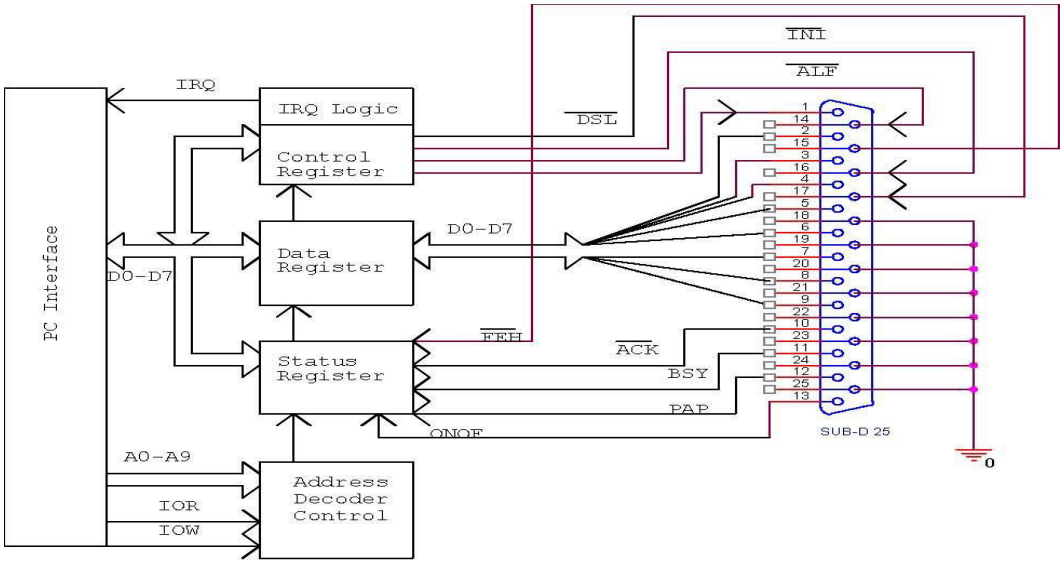
Giống như bộ điều khiển ngắt mà ở đó các yêu cầu ngắt được gửi tới bằng đường IRQ, một DMAC nhận yêu cầu DMA thông qua đường DMA request (DREQ). DREQ được gửi tới CPU, sau khi nhận DREQ, CPU trao quyền điều khiển bus cho DMAC và quá trình DMA được bắt đầu. Tất cả các bộ điều khiển đĩa mềm trên PC đều dùng DMA (kênh 2) để chuyển dữ liệu giữa đĩa mềm và bộ nhớ. Dữ liệu trên đĩa mềm được đọc theo từng đơn vị 512byte (1sector) thông qua DMAC từng 512 byte được chuyển từ đĩa mềm vào địa chỉ xác định trong bộ nhớ.

Trên PC-XT sử dụng 1 chip DMAC 8237. Chip này cung cấp 4 kênh DMA loại 8 bit, chúng được đánh số từ DMA0 đếm DMA3. Đến PC-AT do yêu cầu mở rộng khả năng trao đổi số liệu người ta thêm chip 8237 thứ hai. Có tổng cộng 8 kênh DMA trên PC-AT. Bảng 8-16 mô tả các kênh DMA trên PC.

5.4 Các mạch ghép nối và điều khiển thiết bị vào ra

5.4.1 Cổng song song (Parallel)

Các máy tính PC được trang bị ít nhất là 1 cổng song song và 1 cổng nối tiếp. Khác với ghép nối nối tiếp có nhiều ứng dụng, ghép nối song song thường chỉ phục vụ cho máy in. Sơ đồ ghép nối song song như hình 5.1



Hình 5.1: Sơ đồ cổng song song

Dữ liệu có thể trao đổi trực tiếp giữa 2 PC qua các cổng song song với nhau. Muốn vậy, các đường điều khiển bên này phải được kết nối với các đường trạng thái bên kia.

5.4.2 Cổng nối tiếp (Serial)

** Truyền nối tiếp đồng bộ và bất đồng bộ*

Ghép nối tiếp cho phép trao đổi giữa các thiết bị từng bit một. Dữ liệu thường được gửi theo các nhóm bit SDU (serial data unit) mà mỗi nhóm tạo thành 1 byte hay 1 word. Các thiết bị ngoại vi như: máy vẽ, modem, chuột có thể được nối với PC qua cổng nối tiếp COM.

Sự khác nhau giữa truyền nối tiếp đồng bộ và bất đồng bộ là: trong kỹ thuật truyền đồng bộ, ngoài đường dây dữ liệu phải đưa thêm vào một hoặc vài đường tín hiệu đồng bộ để cho biết rằng khi nào bit tiếp theo ổn định trên đường truyền. Ngược lại trong truyền bất đồng bộ, các bit dữ liệu tự nó chứa các thông tin để đồng bộ; phần phát và phần thu phải hoạt động với cùng 1 tần số xung clock. Thông tin đồng bộ (trong truyền bất đồng bộ) gồm có các bit start (cho biết bắt đầu của khối dữ liệu được truyền) và một bit stop (cho biết kết thúc khối dữ liệu).

** Kiểm tra chẵn lẻ và tốc độ truyền*

Bit chẵn lẻ (parity bit) được đưa vào khung SDU dùng để phát hiện lỗi trên đường truyền. Việc truyền bit chẵn lẻ chỉ kiểm soát được các lỗi trên đường truyền ngắn và các lỗi bit đơn nên trong một số ứng dụng đặc biệt người ta phải dùng mã

CRC mặc dù phức tạp hơn. Tuy nhiên, gần như tất cả các chip hỗ trợ cho ghép nối nối tiếp ngày nay đều được thiết kế phần cứng kiểm tra chẵn lẻ.

Một thông số khác liên quan tới truyền dữ liệu nối tiếp là tốc độ truyền dữ liệu được gọi là tốc độ baud. Trong việc truyền mã nhị phân, đó là số bit được truyền trong một giây (bps - bit per second).

5.5 Các thiết bị ngoại vi thông dụng trong hệ máy vi tính

5.5.1 Bộ nhớ ngoài:

** Đĩa từ mềm và ổ đĩa mềm:*



Hình 5.2 (đĩa mềm và ổ đĩa mềm)

Cấu tạo đĩa mềm: Đĩa mềm (floppy disk) thường là 1 đĩa nhựa dẻo trên mặt tráng 1 lớp oxyt sắt từ, được bảo vệ bởi 1 bao giấy hoặc nhựa cứng. Trên bao có 1 khoảng trống để đầu đọc/ghi của ổ đĩa có thể đọc/ghi lên đĩa.

Một đĩa mềm được tổ chức như sau:

Track (rãnh từ): Là các đường tròn đồng tâm mà dữ liệu được ghi lên đó. Được đánh số từ 0 kể từ ngoài vào.

Sector (cung từ): mỗi track được chia thành nhiều sector, mỗi sector chứa 512 byte dữ liệu. Số sector trên 1 track tùy thuộc vào loại đĩa (có từ 8 đến 36 sector). Được đánh số từ 1

Cluster (liên cung): là 1 nhóm gồm 2, 4, hoặc 8 sector

Đọc và ghi lên đĩa mềm: Trên mặt đĩa là lớp oxyt sắt từ và người ta dùng hiện tượng cảm ứng điện từ để đọc và ghi lên đỉa từ. Trong lệnh định dạng đĩa từ hệ điều hành đã chia đĩa từ thành track, sector và cluster.

Ổ đĩa mềm (FDD- Floppy disk drive): Ổ đĩa mềm là nơi để đặt đĩa mềm. Bên

trong chứa 1 môtơ làm quay đĩa. Tùy theo loại ổ đĩa tốc độ môtơ có thể thay đổi từ 300 – 360 vòng/phút. Một mô tơ bước để định vị đầu đọc/ghi. Các đầu đọc/ghi, mạch điều khiển ổ đĩa mềm thường được cắm trên khe cắm mở rộng hoặc được kết nối trên bản mạch chính. Để nối ổ đĩa mềm vào hệ thống người ta dùng 1 sợi cáp đĩa mềm, 1 đầu nối vào mainboard và 1 đầu nối vào ổ đĩa. Nguồn cung cấp cho ổ đĩa là nguồn hệ thống, một dây cắm cung cấp nguồn cho ổ đĩa được thiết kế sẵn để cắm vào ổ đĩa.

** Đĩa từ cứng và ổ đĩa cứng:*

Cấu tạo vật lý:

Đĩa cứng (hard fixe disk) thường làm bằng kim loại hoặc nhựa cứng trên mặt tráng chất từ tính. Một ổ đĩa cứng thường chứa ít nhất là 1 đĩa. Các đĩa được xếp chồng đồng trục. Trong ổ đĩa còn có các đầu đọc/ghi (mỗi mặt đĩa thường có 1 đầu đọc/ghi), môtơ quay đĩa, môtơ bước, mạch điều khiển ổ đĩa… Trên đĩa có các jumpe dùng thiết đặt điện áp, chức năng của ổ đĩa (là ổ đĩa chủ (master) hay ổ đĩa tớ (slaver)). Tốc độ quay của đĩa cứng rất cao. Sự tiến bộ của kỹ thuật chế tạo làm dung lượng của đĩa cứng ngày càng tăng, kích thước không gian giảm.



Hình 5.3: Đĩa cứng

Một số khái niệm cơ bản của ổ cứng:

+ Tốc độ quay: Là số vòng quay của đĩa trên 1 phút. Các đĩa cứng có thể có tốc độ quay từ 3600 đến 7200 vòng/phút. Tốc đô quay càng lớn thì tốc độ đọc/ghi dữ liệu lên đĩa càng cao.

+ Từ đạo (Track),cung từ (Sector), từ trụ (Cylinder):

- Từ đạo (Track): Mỗi mặt đĩa dược chia thành các rãnh tròn đồng tâm gọi là các từ đạo và được đánh số từ ngoài vào, bắt đầu là track 0.

- Cung từ (sector): Mỗi từ đạo được chia thành các cung từ. Cung từ là đơn vị nhỏ nhất trên đĩa. Các sector được đánh số bắt đầu từ 1.

- Từ trụ (Cylinder): Từ trụ là tập hợp tất cả các từ đạo có cùng bán kính trên các mặt đĩa. Có thể hình dung các từ trụ tạo thành 1 hình trụ (nên gọi là cylinder)

+ Thời gian truy cập dữ liệu:

Thời gian truy cập dữ liệu được tính bằng tổng thời gian của việc tìm kiếm, thời gian chuyển đầu từ từ từ đạo này sang từ đạo cần đọc/ghi dữ liệu và thời gian góc quay trể (Thời gian góc quay trể là do khi đã xác định đúng từ đạo nhưng phải còn chờ đến đúng cung từ được yêu cầu). Tất cả các thông số thời gian càng ngắn thì đĩa hoạt động càng nhanh

Cổng giao tiếp EIDE và SCSI:

+ Giao tiếp EIDE (Enhance Intergrated Driver Electronics): Các mạch điều khiển có thể được tích hợp trên mainboard hoặc dùng 1 card mở rộng và cắm trên mainboard (thường thấy trên các máy 4.86 trở về trước). Giao tiếp EIDE có 2 kênh gọi là kênh sơ cấp (Primary chane) và kênh thứ cấp (Secondary chane). Mỗi kênh có thể nối với 2 thiết bị. Các thiết bị có thể là ổ đĩa cứng hoặc ổ đĩa CD ROM. Trong 1 kênh có 2 thiết bị thì sự điều khiển bus phải theo lượt nên có thể bị ảnh hưởng đến tốc độ. Giao tiếp EIDE thực ra là phiên bản mới của giao tiếp IDE (Integrated Driver Electronic) nên loại giao tiếp này cũng thường gọi là giao tiếp IDE.

+ Giao tiếp SCSI (Small Computer System Interface): Giao tiếp SCSI có thể nối đồng hời 7 thiết bị vào 1 bus. Tuy nhiên ít có mainboard có tích hợp bộ điều khiển SCSI. Do vậy phải có mạch điều khiển SCSI riêng. Giao tiếp SCSI có nhiều loại: Loại 8 bit có cable dữ liệu 50 dây. Loại 16 bit có cable dữ liệu 68 dây. Nhìn chung loại giao tiếp này thường rất đắt và chỉ hiệu quả ở những hệ thống cần sử dụng nhiều thiết bị dùng chung bus.

**Đĩa CD-ROM (Compact Disc – Read Only Memory):*



Hình 5.4: (ổ đĩa CD-ROM)

CD-ROM là công nghệ mới phát triển nhưng có nhiều ưu điểm, được sử dụng ngày càng nhiều. Đến ngày nay 1 bộ máy PC gần như không thể thiếu 1 ổ đĩa CD cho nó. Thông thường 1 đĩa CD chứa khoảng 650Mb dữ liệu. Trong ổ đĩa có 1 diode laser bán dẫn nhỏ phát ra 1 tia laser rất mảnh. Một gương quang học hoạt động một cách chính xác (nhờ sự điều khiển của mạch điện tử) sẽ hội tụ tia laser lên bề mặt của đĩa CD-ROM. Đĩa được ghi các thông tin ở dạng các lỗ lõm nhỏ trên bề mặt của đĩa theo các đường xoắn từ ngoài mép đĩa vào trong. Cường độ phản xạ của tia sẽ thay đổi khi đĩa quay. Một sensor cảm biến quang nhận tia phản xạ và mạch điện tử chuyển chúng thành các bit dữ liệu. Ổ đĩa CD-ROM là 1 hộp kín bên trong chứa các bộ phận điều khiển đĩa, mở đóng hợp đĩa, mạch điện tử số và mạch âm thanh. Ngày nay đã có các ổ đĩa CD-ROM đọc/ghi và có thể trang bị cho máy PC

Ổ đĩa CD-ROM:

Các thông số cần quan tâm của 1 ổ đĩa CD-ROM là: Tốc độ đọc của ổ đĩa: Được ký hiệu là X. Mỗi X được tính tương đương với 150KB được đọc trong 1 giây. Trên thị trường hiện nay phổ biến là loại 52X.

Các ổ đĩa CD-ROM và các đĩa CD-ROM là các thiết bị chỉ đọc thông tin.

Ổ đĩa CD-RW (Compact Disc – Read Write):

Là loại ổ đĩa có thể ghi dữ liệu lên dĩa CD và đọc dữ liệu trên đĩa CD. Nguyên tắc đọc giống như đọc trên ổ đĩa CD-ROM. Nguyên tắc ghi thì người ta dùng đầu ghi là 1 chùm sáng laze khoét các lổ lồi lõm trên bề mặt đĩa theo sự điều khiển của các Bit dữ liệu. Có 2 loại đĩa dùng để ghi dữ liệu đó là:

Loại chỉ ghi 1 lần (CD Recordable): Loại này cho phép ghi 1 lần sau đó không thể ghi và xóa thêm được nữa.

Loại ghi nhiều lần (CD ReWritable): Loại này cho phép ghi (ghi thêm)/xóa

nhiều lần. Số lần ghi/xóa tuỳ thuộc vào chất lượng đĩa. Khi dùng đĩa loại này thì các đầu CD và VCD bình thường không thể đọc loại đĩa này.



Hình 5.5: Ổ đĩa CD-RW

Các tham số cần chú ý trên đầu CD – RW: Tốc độ ghi, tốc độ đọc, tốc độ vừa đọc vừa ghi. Các tham số này thường được ghi trên mặt của đầu đĩa. Ví dụ 1 đầu có ghi 32X/8X/4X có nghĩa là: Tốc độ đọc là 32X, tốc độ ghi là 8X, tốc độ vừa đọc vừa ghi là 4X. Các tham số trên cũng có thể được ghi cụ thể là R32X/W8X/RW4X.

Ổ đĩa DVD – ROM và DVD RW:

- Ổ đĩa DVD – ROM (Digital Video Disc (Digital Vervatile Disc) – Read Only Memory): Là loại đĩa (ổ đĩa) chỉ dọc. Loại này là loại phát minh mới trong lĩnh vực lưu trữ hình ảnh, âm thanh bằng kỹ thuật số. Có 2 loại đó là Digital Video Disc (Đĩa phim ảnh kỹ thuật số) và loại Digital Versatile Disc (Đĩa đa năng kỹ thuật số).

- Các tham số của đầu DVD: Các tham số cần chú ý là tốc độ đọc của đầu đĩa. Tốc độ hiện nay có thể là 12X hoặc 16X (Mỗi 1X tương đương với 1,38MB/s).

- Đầu DVD - RW: Giống như CD-RW ngoài chức năng đọc còn có thêm chức năng ghi dữ liệu lên đĩa DVD.

- Hiện nay chưa có 1 chuẩn thống nhất cho các đĩa DVD

- Dung lượng của đĩa DVD khoảng 4,7GB (Lớn gấp 7 lần dung lượng đĩa CD).

** Thẻ nhớ (USB):*

Ngày nay thẻ nhớ trở thành thiết bị nhớ ngoài phổ biến. Dung lượng thẻ tăng theo công nghệ chế tạo từ 128MB đến nay đã lên đến 64GB. Thẻ nhớ giao tiếp với máy tính thông qua cổng thông minh (cổng USB).



Hình 5.6: Thẻ nhớ

***** *Màn hình và card màn hình:*



Hình 5.7: Card màn hình

+ Card màn hình (Graphics adaptor):

Là một bản mạch mở rộng được cắm vào bộ kết nối bus mở rộng của PC. Màn hình hiển thị thông tin của PC thông qua card này. Nhiệm vụ của nó là chuyển dữ liệu thô của PC thành dữ liệu hình ảnh để chuyển ra cho màn hình.

Phần trung tâm là chip điều khiển ống hình CRTC (cathode ray tube controler). CPU thâm nhập RAM video qua mạch ghép nối bus để viết thông tin xác định ký tự hoặc hình ảnh cần hiển thị. CRCT liên tục phát ra các địa chỉ để RAM video đọc các ký tự và truyền chúng tới máy phát ký tự (Character gerenerater). Trong chế độ văn bản (text mode), các ký tự được xác định bởi mã ASCII, trong đó có cả thông tin về thuộc tính của ký tự (như thuộc tính nhấp nháy, đảo màu …) ROM ký tự (charater ROM) lưu trữ các hình ảnh điểm mẫu của ký tự tương ứng để máy phát ký tự biến đổi các mã ký tự đó thành 1 chuỗi các bit điểm ảnh (pixe) và chuyển chúng tới thanh ghi dịch (Shift register). Máy phát tín hiệu sẽ sử dụng các bit điểm ảnh này cùng với các thông tin thuộc tính từ RAM video và các tín hiệu đồng bộ (Sync signal) từ CRTC để phát ra các tín hiệu cần thiết cho monitor.

Trong chế độ đồ họa (Graphics mode), thông tin trong RAM video được sử dụng trực tiếp cho việc phát ra các ký tự. Lúc này các thông tin về thuộc tính cũng không cần nữa. Chỉ từ các giá trị bit trong thanh ghi dịch, máy phát tín hiệu sẽ phát các tín hiệu về độ sáng và màu cho monitor.

- Theo sự phát triển của công nghệ chế tạo các loại card màn hình ra đời như sau:

MDA (card màn hình trắng đen-1981)

CGA (card màn hình màu- 1981)

EGA (card màn hình nâng cao – 1984)

PGA (card màn hình chuyên nghiệp- 1984)

MCGA (chip đồ họa nhiều màu – 1987)

VGA (Card đồ họa hình ảnh)

8514 (1987)

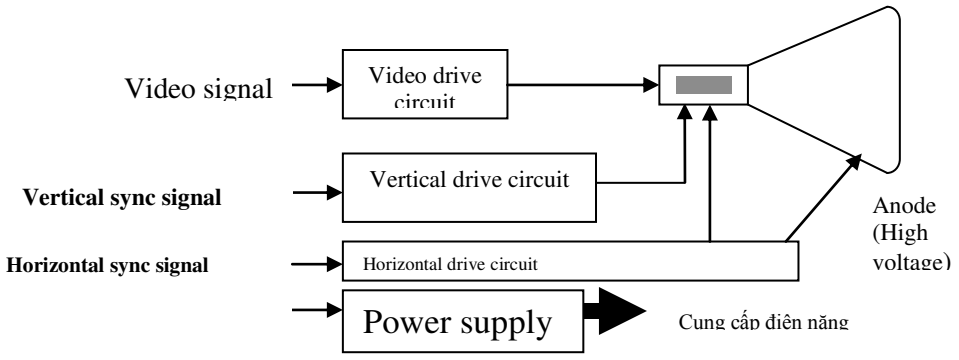
SVGA (Card màn hình siêu việt-1990)

XGA 90

+ Màn hình (Monitor):

- Màn hình đen trắng:

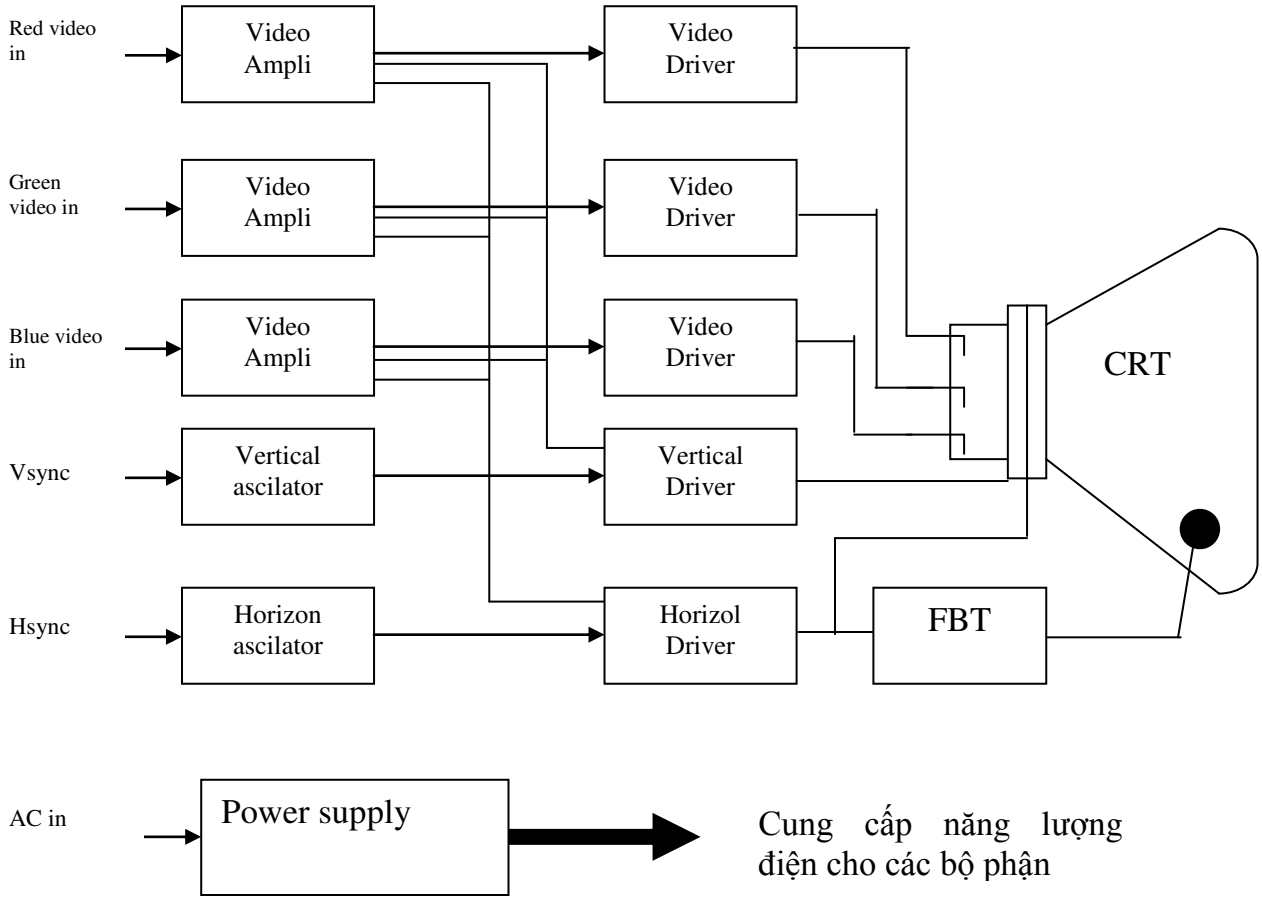
Đèn hình hay còn gọi là ống phóng tia âm cực CRT (cathode ray tube). Các tia điện tử phát xạ từ Catốt trong ống được hội tụ thành 1 chùm tia, sau đó được tăng tốc và nhờ bộ phận lái tia đến đập một cách chính xác vào màn hình có phủ lớp huỳnh quang, tạo thành điểm sáng trên màn hình. Độ chói (sáng/tối) được quyết định bởi cường độ chùm tia đập vào màn huỳnh quang. Sơ đồ khối hình 5.7 mô tả màn hình đen trắng.



Hình 5.7

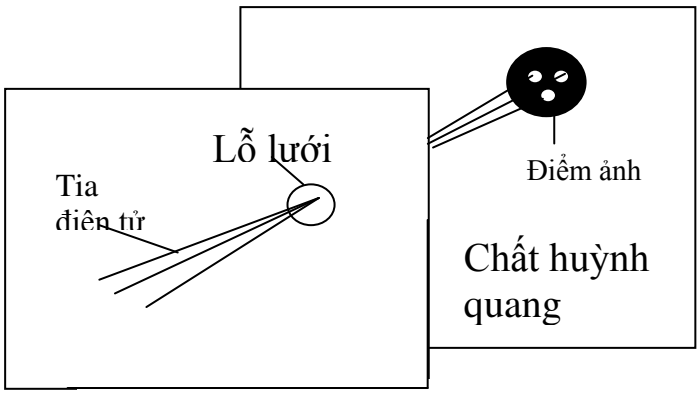
Màn hình màu:

Đèn hình màu hay còn gọi là ống phóng tia âm cực màu (The color CRT). Nguyên lý cơ bản của màn hình màu được mô tả ở sơ đồ khối như hình 5.8.



Hình 5.8: Sơ đồ khối màn hình màu

Các nguyên lý cơ bản hiện hình của màn hình màu gần giống như mành hình đen trắng. Các điện tử được bắn ra khỏi cathode và được tăng tốc bởi điện áp dương rất cao (High volta) hướng tới màn hình có phủ lớp huỳnh quang. CRT màu sử dụng 3 cathode và các lưới điều khiển hình tương ứng. Người ta đặt 1 lưới kim loại với hàng ngàn lỗ cực nhỏ với mỗi lỗ cho 1 ảnh điểm trên màn hình. Lưới kim loại này được đặt rất gần với bề mặt lớp huỳnh quang. Lớp huỳnh quang của màn hình màu là sự sắp xếp hợp lý của chất phát sáng khi có điện tử tương ứng đập vào đập vào. Người ta sắp xếp chất phát sáng màu đỏ, màu xanh dương và màu xanh lá cây thành các điểm tam như hình 5.9.



Hình 5.8

Các tia điện tử đỏ, xanh dương và xanh lá sẽ đập vào các chất phát sáng tương ứng trên màn hình. Các điểm phát sáng màu này đặt rất gần nhau nên mỗi ảnh điểm được hiển thị là do sự pha trộn của 3 màu cơ bản là màu đỏ (red), màu xanh dương (blue) và màu xanh lá cây (green). Các màu này có được là do chất huỳnh quang tương ứng phát ra các màu đó khi có tia điện tử đập vào.

-Hiện nay có nhiều hãng sản xuất màn hình như SONY, HITACH, LG…

Các loại màn hình màu:

+ Màn hình màu theo các chuẩn CGA (Color Graphics Adaptors), EGA (Enhanced Graphics Adaptor) và VGA (Video Graphics Array). Các loại màn hình trên liên quan mật thiết với các loại card màn hình. Trong loại VGA có 2 loại là Analog (tương tự) và loại Digital (số). Loại Analog rất ít thấy trên thị trường Việt Nam



Hình 5.9

+ Màn hình màu phẳng: Có 2 loại

Màn hình phẳng FLATRON: Được chế tạo dựa trên màn hình truyền thống (CRT), có cải tiến thêm như có phủ lớp chống tĩnh điện, hội tụ tự động… Hình dáng bên ngoài như màn hình thông thường. Ta phân biệt được là nhờ ở mặt màn

hình tương đối phẳng và có màu đen tuyền của nó.



Hình 5.10 (Màn hình phẳng)

Màn hình tinh thể lỏng LCD (Liquid Crystal Display): là một tiến bộ của kỹ thuật. Các điểm ảnh được hiển thị là do dữ liệu trong RAM video quyết định. Ưu điểm là gọn nhẹ, tiết kiệm nhiều năng lượng. Màn hình tinh thể lỏng là hướng phát triển của các thế hệ màn hình sau này.



Hình 5.11: Màn hình LCD

Độ lớn của màn hình:

Là số đo của đường chéo của mặt màn hình. Đơn vị thường dùng là Inches (1inches = 2,54 Cm)

Mạch điện tử phục vụ cho sự hiện ảnh là các môđun điện tử đã được mô tả. Tất cả các linh kiện điện tử được ghép nối trên 1 bản mạch.

**Bàn phím và cổng bàn phím:*

Bàn phím:

Bàn phím là thiết bị nhập chuẩn của máy tính PC. Bàn phím thực chất là 1 hệ thống con, vì nó có 1 bộ vi xử lý chuyên dụng riêng, chuyên xử lý trạng thái hoạt động của tất cả các phím. Các phím được sắp xếp theo 1 ma trận diện, mỗi phím được địa chỉ hóa theo cột và hàng. Bộ vi xử lý thực hiện quét liên tục ma trận phím

để xác định phím đang hoạt động (ấn hay nhả). Thông tin về phím hoạt động được đưa vào bộ đệm mã quét (Scan-code buffer). Từ bộ đệm này thông tin được gởi liên tiếp theo đường dây đến đơn vị giao tiếp với bàn phím nằm trên mainboard.

Bàn phím thông thường có từ 84 đến 104 phím (có loại còn thêm 1 số phím chức năng riêng), chia làm 3 nhóm phím là: Nhóm phím chức năng, nhóm phím điều khiển và nhóm phím ký tự. Khi một phím được ấn trên bản mạch của bàn phím có 1 công tắt được đóng (Mỗi phím coi như 1 công tắt thường hở). Bàn phím nối với máy tính bằng 1 sợi cáp, ở cuối sợi cáp là 1 Jack cắm để cắm vào main board ở phía sau máy. Có 2 loại jack thường thấy là loại jack lớn và loại jack nhỏ. Hai loại jack này có thể chuyền cho nhau được nhờ 1 đầu nối chuyển đổi để tương thích với lỗ cắm trên mainboard.

Cổng bàn phím:

Cổng bàn phím thường được bố trí phía sau máy và hàn cứng vào mainboard cùng với bộ phận ghép nối bàn phím để truyền dữ liệu vào CPU hoặc ngược lại. Ngày nay người ta còn sử dụng bàn phím không dây.

**Mouse, cổng mouse*

Mouse (Chuột)

Cấu tạo:

Chuột là thiết bị nhập của máy tính, được nối với PC qua cổng nối tiếp. Khi di chuyển chuột hoặc ấn phím chuột thì một gói dữ liệu sẽ được chuyển đến cổng và chuyển vào cho máy (1lệnh được phát ra), CPU sẽ thi hành lệnh đó. Phần mềm điều khiển chuột sẽ qui định các phím ấn của chuột và lệnh tương ứng được thi hành.

Về cấu tạo có nhiều kiểu chuột:

Chuột cơ học (Mechanical mouse): Trên chuột cơ học cử động của con trỏ chuột trên màn hình phụ thuộc vào sự chuyển động cơ học của 1 viên bi nằm phía dưới của chuột. Khi di chuyển chuột trên 1 mặt phẳng, viên bi lăn, làm chuyển động 2 bánh xe đặt vuông góc nhau trong chuột (một bánh cho trục X và 1 bánh cho trục Y). Hai bánh xe có các rãnh trống được đặt trong khe hở của 2 sensor cảm biến quang. Khi bánh xe quay bộ phận thu của sensor sẽ nhận hay không nhận được ánh sáng từ nguồn phát. Mức logic của đầu ra của sensor sẽ là 1 xung điện. Căn cứ vào

xung điện (tốc độ quay của bánh xe) CPU diều khiển tốc độ di chuyển của con trỏ chuột trên màn hình. Trên chuột còn có 2 hoặc 3 phím ấn, có khi còn được trang bị thêm phím chức năng khác. Khi 1 phím được ấn tùy vào phần mềm điều khiển mã lệnh tương ứng sẽ được thi hành.

Chuột quang học (Optical mouse): Thay vì chuyển động cơ học của viên bi người ta dùng các bộ sensor cảm biến quang. Ở đây nguồn phát là 1 LED (light emitting diode). Bộ phát hiện quang là nguồn thu ánh sáng phản xạ từ tấm bàn di chuyển chuột kèm theo với nó. Tấm bàn di chuyển có kẻ các đường phản quang tương ứng với bước sóng mà các LED phát ra. Khi di chuyển chuột trên tấm bàn này, các bộ thu sẻ nhận dược các bước sóng tương ứng và các xung điện được hình thành. Các xung điện này được gởi đến CPU để điều khiển con trỏ chuột trên màn hình. Các phím nhấn của chuột quang học cũng tương tự như phím nhấn của chuột cơ học.



Hình 5.12: Mouse quang

Chuột tĩnh (Trackball): Thay vì chuyển động chuột trên mặt bàn để làm chuyển động viên bi, người ta cố định chuột và dùng tay lăn viên bi để điều khiển sự chuyển động của con trỏ chuột trên màn hình.

Cổng mouse:

Chuột được nối và cổng nối tiếp của máy tính. Trong máy tính thường dành 4 cổng nối tiếp để nối các thiết bị nối tiếp. Các cổng đó có tên là COM. Thường thì COM1 và COM3 dùng ngắt (IRQ4), COM2 và COM4 dùng ngắt 3 (IRQ3). Chuột thường dùng COM1. Ngày nay người ta cò sử dụng mouse không dây.

**Máy in, cổng máy in*

Máy in:

Cấu tạo – hoạt động:

Máy in là thiết bị xuất ngoại vi của máy tính PC. Máy in thường được nối vào cổng song song của máy tính. Cổng thường dùng cho máy in là LPT1 và LPT2. Về nguyên tắc cấu tạo thì có thể chia máy in thành 2 loại: Máy in ma trận điểm và máy in điện quang.

- Máy in ma trận điểm:

Máy in ma trận điểm kiểu rời (máy in kim): Nguyên tắc máy in ma trận điểm kiểu rời là khi in sử dụng ma trận các kim thép đặt vuông góc với giấy in. Các kim thép được đặt trong 1 đầu in, thường thì một đầu in có 7 đến 9 kim hoặc 18 đến 24 kim. Số kim trong đầu in càng nhiều thì chất lượng in càng tốt. Mực in được thấm vào 1 băng in gọi là dây ru-băng

Ký tự để in được lưu trữ trong ROM, hệ thống xác định các kim trong ma trận gõ. Khi kim được tác động bởi lực từ (gõ) nó sẽ phóng qua dây ru-băng và mang theo vết mực lên giấy.



Hình 5.13: Máy in kim

Cơ cấu máy in ma trân điểm kiểu rời gồm 1 đầu in chứa các kim thép được vận hành bởi các cuộn dây, đầu in được gắn trên 1 thanh trượt cố định ngang qua vùng in. Nối đầu in với mạch điều khiển là các sợi cáp.Hai động cơ bước dẫn động quá trình in: Một dùng để di chuyển đầu in và 1 dùng chuyển động tiến của giấy in. Hai động cơ hoạt động đồng bộ và được điều khiển bởi bộ vi xử lý. Một hộp đựng dây ru-băng, dây ru-băng được mắc vuông góc với đầu in. Giấy in có thể là giấy cuộn hoặc giấy rời có kích thước phù hợp với máy in. Khổ giấy dùng cho máy in trang bị cho máy tính PC thường lới tối đa là khổ giấy A3.

In không va chạm: Máy in không va chạm có thể chia làm 2 loại: Loại máy sử

dụng giấy in có lớp tráng phủ và loại sử dụng giấy in bình thường. Thông thường người ta hay sử dụng loại máy in dùng loại giấy in bình thường, trong máy các hạt mực nhỏ xíu được phun mạnh lên giấy.

-Máy in điện quang: Là loại máy in hoạt động dựa vào hiện tượng điện quang. Các máy in laser sử dụng tia laser để hiển thị lại hình ảnh. Cũng có loại máy in không sử dụng tia laser mà sử dụng 1 thanh có chứa nhiều LED rất nhỏ để hiển thị lại hình ảnh. Ngày nay người ta thường sử dụng máy in laser.

***Modem, Scanner**

+Modem:

Modem là gì ?:

Modem là chữ viết tắt của Modulator-Demodulator (điều chế và giải điều chế). Modem là 1 thiết bị điện tử nối với cổng nối tiếp của máy tính dùng trong việc thông tin giữa 2 máy tính qua đường dây cáp điện thoại.

Modem kết hợp với các phần mềm truyền số liệu thích hợp sẽ cho phép người ta thực hiện các công việc trên máy tính như truyền các tập tin, gởi thư điện tử,fax, quay số điện thoại, truy cập thông tin trên mạng…

Các loại modem:

Về mặt vật lý có thể chia modem thành 2 loại:

Modem nội trú (Intenal modem): Là 1 bảng điều khiển mở rộng chuẩn, bảng được cắm vào khe cắm trong máy tính. Nguồn điện cung cấp cho bảng là nguồn của hệ thống và nó cung cấp cổng nối tiếp riêng của nó. Đường điện thoại được cắm vào bộ kết nối sau lưng bảng mạch.



Hình 5.14: Modem nội trú

-Modem ngoại trú (external modem): Là 1 hợp rời, nối với cổng nối tiếp của

máy tính nhờ 1 cáp tín hiệu. Nguồn điện cung cấp cho nó là 1 biến áp cắm rời. Modem kết nối với cáp điện thoại bằng 1 sợi cáp.



Hình 5.15: Modem ngoại trú



Hình 5.16: (Modem ADSL không dây

Tốc độ của modem:

Chỉ số Bits per second (bps) là số bit mà modem có thể truyền trong 1 giây

Chỉ số baud rate là nhịp truyền của modem (thông tin truyền trong 1 giây)

Ví dụ: modem 2400 bps, 9600 bps

Modem gởi và nhận tín hiệu:

- Khi gắn 1 modem vào máy tính là ta đã kết nối 1 cổng nối tiếp của máy tính vào 1 thiết bị nối tiếp. Luồng dữ liệu giữa modem và máy tính là luồng dữ liệu tại chổ (local flow control) có thể được điều khiển bởi bắt tay phần mềm hay bắt tay phần cứng.

- Việc điều khiển luồng giữa 2 modem được gọi là điều khiển luồng nối đầu nhau (end-to-end flow control) thường được tự động xử lý lỗi (chữa lỗi)

- Dữ liệu được chuyển từ máy tính đến modem là dữ liệu kỹ thuật số (0 và 1).

Đến modem dữ liệu được biến điệu thành 1 tín hiệu gọi là tín hiệu vật mang tin (sóng mang). Tín hiệu này được gởi lên đường cáp của điện thoại. Ở đầu kia modem nhận được tín hiệu vật mang tin gởi đến và tín hiệu đó được hoàn điệu (demulated – chuyển tín hiệu vật mang tin thành tín hiệu kỹ thuật số) và được đưa vào cổng nối tiếp của máy tính nhận.

-Để liên lạc với nhau 2 modem gởi các tín hiệu cho nhau và xác định 1 phương tiện truyền thông tin chung. Sau khi đã xác lập phương tiện truyền thông tin các modem sẽ kết nối với nhau với 1 mẫu chung cao nhất. Quá trình sử dụng để thương lượng và truyền gọi là quá trình biến điệu.

Lắp đặt modem:

Các bộ phận đi kèm theo modem: Một modem thường có các bộ phận đi kèm như sau:

Bộ nguồn nuôi (Adaptor)

Cáp nối với máy tính.

Cáp nối với đường dây điện thoại.

+ Scaner (Máy quét):

Máy quét để bàn là loại thông dụng bổ sung cho máy tính PC. Kết cấu và năng lực của máy quét để bàn đang là mục tiêu cải tiến của lĩnh vực điện tử. Các sản phẩm hiện nay phát triển nhanh chóng từ loại đen trắng đến loại màu. Một số loại chỉ quét được màu, một số loại chỉ quét được trắng đen, hiện nay máy scaner quét dược cả màu và trắng đen.



Hình 5.17: Máy quét

Chương 6. HỢP NGỮ

6.1. Hợp ngữ và các trình biên dịch hợp ngữ 16 bit

6.1.1 Ngôn ngữ máy và hợp ngữ

Bộ xử lý hoạt động dựa trên kỹ thuật số nên chỉ hiểu được những giá trị nhị phân. Do đó, ngôn ngữ mà CPU hiểu được gọi là ngôn ngữ máy (Machine language). Mỗi bộ xử lý có tập giá trị nhị phân, trong đó mỗi giá trị nhị phân sẽ điều khiển CPU thực hiện một tác vụ (gọi là mã lệnh), tập hợp các mã lệnh nhị phân gọi là tập lệnh. Những bộ xử lý khác nhau sẽ có tập lệnh khác nhau. Do đó, ngôn ngữ máy của CPU nào thì chỉ thực hiện được trên CPU đó.

Bảng 6.1 trình bày 1 đoạn chương trình ngôn ngữ máy viết cho họ Intel-8086.

Bảng 6.1

TT lệnh	Giá trị thập lục phân	Mã lệnh nhị phân
1	B4 09	10110100 00001001
2	BA 03 01	10111010 00000011 00000001
3	40	01000000

Đối với CPU có kiến trúc CISC, chiều dài các lệnh có thể khác nhau, do đó trong đoạn chương trình trên gồm 3 lệnh có chiều dài lần lượt là 2 byte, 3 byte và 1 byte. Byte đầu tiên của mỗi lệnh là Tác vụ (Op-Code) mà CPU phải thực hiện, các byte còn lại là Tác tử (Operand) xác định dữ liệu hoặc nơi chứa dữ liệu mà lệnh tác động vào. Để có thể lập trình với ngôn ngữ máy, người lập trình phải hiểu rõ tổ chức phần cứng của máy đang sử dụng. Vì là ngôn ngữ riêng của CPU và được CPU thực hiện ngay khi đọc được lệnh, nên chương trình được viết bằng ngôn ngữ máy thì sẽ được thực hiện rất nhanh và chiếm ít bộ nhớ trong. Tuy nhiên, dạng nhị phân của ngôn ngữ máy rất khó nhớ và dễ nhầm lẫn. Để khắc phục nhược điểm khó nhớ của dạng nhị phân, người ta dùng những từ gợi nhớ thay cho những mã lệnh nhị phân (gọi là từ gợi nhớ mã lệnh –memonic). Chương trình viết bằng từ gợi nhớ mã lệnh gọi là Hợp ngữ (Assembly).

Mỗi mã lệnh nhị phân có một lệnh tương ứng trong Hợp ngữ và mỗi CPU khác nhau sẽ có Hợp ngữ khác nhau. Các tài liệu tham khảo về tập lệnh của CPU thường được cung cấp cho các lập trình viên là tập lệnh của CPU ở dạng Hợp Ngữ.

Bảng 6.2 viết lại đoạn chương trình ở bảng trên ở dạng Hợp ngữ. Qua đó chúng ta có thể hiểu được phần nào chức năng của mỗi dòng lệnh trong đoạn chương trình đã nêu thông qua lệnh Hợp ngữ.

Bảng 6.2

Thứ tự lệnh	**Giá trị thập lục phân**	**lệnh Hợp ngữ**	**Ý nghĩa**
1	B4 09	MOV AH, 09h	AH ← 09h
2	BA 03 01	MOV DX, 0103h	DX ← 0103h
3	40	INC AX	AX ← AX + 1

Như vậy, Hợp ngữ sẽ làm cho lập trình viên dễ dàng hơn trong việc viết chương trình, nhưng CPU thì không hiểu được những từ gợi nhớ của mã lệnh đó. Do đó, từng lệnh trong chương trình viết bằng hợp ngữ phải được dịch sang ngôn ngữ máy tương ứng. Chương trình làm nhiệm vụ này gọi là: *Trình hợp dịch (Assembler)*

Khi mới ra đời, các chương trình hợp ngữ chỉ làm việc đơn thuần là dịch những từ gợi nhớ mã lệnh sang mã máy, nhưng dần dần các trình hợp dịch bổ sung thêm các lệnh riêng của mình vào để thuận lợi hơn cho các lập trình viên. Trình hợp dịch cho phép dùng các nhãn, các ký hiệu, biến đổi dạng lệnh, khai báo biến, phân phối bộ nhớ, macro… Các lệnh riêng đó gọi là lệnh giả (pseudo code). Các lệnh giả không có mã máy nhị phân tương ứng, nên bản thân các lệnh giả không điều khiển CPU thực hiện tác vụ mà chỉ dùng để hướng dẫn trình hợp dịch trong quá trình dịch chương trình. Do đó, khi gặp từ gợi nhớ mã lệnh, trình hợp dịch sẽ dịch sang mã máy nhị phân; còn khi gặp lệnh giả, trình hợp dịch không dịch thành mã nhị phân.

6.1.2 Hợp ngữ MASM (Hợp ngữ của CPU 8086)

MASM (Microsoft Macro Assembler) là trình hợp ngữ do hãng phần mềm Microsoft phát triển cùng với phiên bản hệ điều hành DOS. Ngoài ra, hãng Borland (chuyên xây dựng các chương trình dịch) cũng phát hành chương trình dịch hợp ngữ gọi là TASM (Turbo Assembler). Các phiên bản này chỉ cho phép viết chương trình trên hệ điều hành DOS, cho nên không viết được chương trình cho Windows

bằng các phiên bản này. Hiện nay, đã có các phiên bản cho phép viết chương trình trên Windows, như MASM32 (MASM 32 bits).

Người lập trình dùng các từ gợi nhớ mã lệnh, các lệnh giả, các ký hiệu ... do trình hợp ngữ MASM qui định để viết ra một chương trình mà ta gọi là chương trình hợp ngữ nguồn (source file). Chương trình nguồn được lưu trữ ở bộ nhớ ngoài dưới một tên tập tin có phần mở rộng là .ASM. MASM sẽ dịch chương trình nguồn thành chương trình đối tượng có phần mở rộng là .OBJ (Object file). Chương trình đối tượng sẽ được liên kết (LINK) thành chương trình có thể chạy được có phần mở rộng là .EXE (Executive file).

Thông thường các loại hợp ngữ khác nhau đều có cú pháp gần giống nhau, nhưng chúng cũng có những đặc điểm riêng thể hiện khả năng của nó.

6.1.3 Cấu trúc của một hàng lệnh

Một hàng lệnh có thể có 4 thành phần, trong đó các thành phần cách nhau ít nhất 1 khoảng trắng hoặc 1 ký tự Tab; các thành phần trong dấu ngoặc vuông là tuỳ chọn. Có tối đa 132 ký tự trên 1 hàng (512 đối với MASM 6.0):

[Identifier]	Operation	[Operad(s)]	[; Commend]
Định danh	Từ gợi nhớ mã lệnh hoặc lệnh giả	Toán hạng hoặc biểu thức	Chú thích

Ví dụ: 1 dòng lệnh như sau:

BDAU**:** MOV DX**,** 3F8h **;** 3F8H là địa chỉ cổng vào ra nối tiếp

Trong đó: BDAU – là Định danh với vai trò là nhãn

: – (Dấu 2 chấm) phân cách Định danh và Từ gợi nhớ

MOV – Từ gợi nhớ mã lệnh

DX, 3F8h – Hai toán hạng, phân cách bằng dấu phẩy

; – (Dấu chấm phẩy;) Phân cách Toán hạng và Chú thích

3F8H là địa chỉ cổng vào ra nối tiếp - là câu chú thích

- Định danh:

Định danh có thể là tên (Name) hoặc Nhãn (Label). Tên có thể là tên biến, tên hằng, ….Nhãn áp dụng cho tên của 1 chỉ thị

Tên có chiều dài tối đa là 31 ký tự và phải bắt đầu bằng một chữ cái. Các khoảng trống và ký hiệu toán học không được dùng để đặt tên.

Nhãn dùng để đánh dấu một địa chỉ mà các lệnh như: lệnh nhảy, gọi chương trình con, lệnh lặp cần đến. Nó cũng được dùng cho các lệnh giả LABEL hoặc PROC hoặc EXTRN

Ví dụ:

NH: MOV AX, DS ; NH là nhãn đánh dấu một địa chỉ ô nhớ

FOO LABEL near ; đặt tên cho địa chỉ ô nhớ sau lệnh giả này

CTCON PROC FAR ; địa chỉ bắt đầu của chương trình con

EXTRN **NH** near ; NH là nhãn ngoài chương trình gốc này

- Biến: Biến được dùng làm toán hạng cho lệnh hoặc biểu thức. Biến có thể chứa giá trị hoặc 1 địa chỉ mà ta cần.

Ví dụ: **TWO** DB 2 ; khai báo biến có tên là TWO và có giá trị là 2

- Tên hằng: Tên hằng là một tên được định nghĩa để thay cho một biểu thức, một từ gợi nhớ lệnh. Tên hằng có thể dùng làm toán hạng trong biểu thức, trong lệnh hay trong lệnh giả.

Ví dụ: **FOO** EQU 7h ; Hằng FOO thay thế giá trị 07h

Hang_so = 0Fh ; Hằng Hang_so thay thế giá trị 0Fh

- Từ gợi nhớ mã lệnh, lệnh giả:

Từ gợi nhớ mã lệnh sẽ được trình bày trong các mục sau về tập lệnh của bộ xử lý 8086. Trong phần này sẽ trình bày các lệnh giả thường dùng của MASM. Lệnh giả có thể được chia thành 5 nhóm: cấu trúc chương trình, khai báo dữ liệu, dịch có điều kiện, Macro và Liệt kê

A. Nhóm cấu trúc chương trình:

+ SEGMENT và **ENDS**: Khai báo đoạn.

Cú pháp:

<tên đoạn> SEGMENT [align] [combine] [‘class‘]

….. ; nội dung của đoạn

<tên đoạn> ENDS

Trong đó:

[align] xác định nơi bắt đầu của đoạn gồm các giá trị :

Byte : Đoạn có thể bắt đầu ở địa chỉ bất kỳ.

Word : Đoạn phải bắt đầu ở địa chỉ chẵn.

Para : Đoạn phải bắt đầu ở địa chỉ là bội số của 16.

Page : Đoạn phải bắt đầu ở địa chỉ là bội số của 256.

[combine] xác định cách kết hợp phân đoạn này với phân đoạn khác:

PUBLIC: các đoạn cùng tên và cùng class được ghép nối tiếp nhau khi liên kết.

COMMON: các đoạn cùng tên và cùng class được ghép phủ lấp lên nhau khi liên kết.

AT <biểu thức> : Đoạn được đặt tại một địa chỉ là bội số của 16 và được ghi trong biểu thức.

STACK : Giống như Public, tuy nhiên con trỏ ngăn xếp SP chỉ vào địa chỉ đầu tiên của ngăn xếp đầu tiên.

PRIVATE: các đoạn cùng tên và cùng class không được ghép vào nhau

Ví dụ: Khai báo hai đoạn có tên là DSEG và CSEG: Không phủ lấp lên nhau

```
DSEG SEGMENT
……   ; Khai báo dữ liệu
DSEG ENDS
CSEG SEGMENT
……   ; Các lệnh trong đoạn
CSEG ENDS
```

+ ASSUME: Chỉ định loại của một đoạn.

Cú pháp: ASSUME <SegReg>: <Tên1>, <SegReg>: <Tên2>

Ví dụ: Chỉ định đoạn có tên DATA là đoạn dữ liệu (DS) và đoạn có tên CODE là đoạn lệnh (CS):

```
ASSUME DS: DATA, CS: CODE
```

Lưu ý: Lệnh ASSUME chỉ được viết trong đoạn lệnh.

Ví dụ: Báo cho hợp ngữ biết là không có đoạn nào được chỉ định loại. Khi đó, mỗi lần liên hệ đến một nhãn (biến) phải dùng cả địa chỉ đoạn của chúng:

ASSUME NOTHING

+ COMMENT: ghi chú chương trình. Có thể viết trên nhiều dòng sau dấu chấm phảy (;)

+ EVEN : Làm cho thanh ghi đếm chương trình PC là giá trị chẳn

+ EXTRN: Cho biết một tên hay một ký hiệu đã được định nghĩa bên ngoài ở một module khác được sử dụng ở module chương trình hiện tại.

+ END: Điểm cuối chương trình nguồn và xác định điểm bắt đầu

Cú pháp: END <Nhãn>

Ví dụ: END begin ; điểm bắt đầu chạy CT là nhãn begin

+ GROUP: Nhóm các đoạn khác nhau có một tên nhóm dùng chung

Ví dụ: Nhóm 3 đoạn có tên DATA1, DATA2 và DATA3 có tên nhóm dùng chung là NHOM_CHUNG:

NHOM_CHUNG GROUP DATA1,DATA2,DATA3

+ INCLUDE : Xen một tập tin hợp ngữ khác vào tập tin hiện hành

Ví dụ: Xen tập tin THEM.ASM trên ổ đĩa C:\ASM vào tập tin hiện hành ngay tại vị trí của lệnh giả INCLUDE:

INCLUDE C:\ASM\THEM.ASM

+ LABEL : đánh dấu một địa chỉ là địa chỉ của lệnh hay số liệu kế

Ví dụ:

NHF LABEL FAR ; nhãn xa, đánh dấu vị trí NH của lệnh kế

NH: MOV AX, DATA

CH DW 100 DUP (0) ; chuỗi từng từ

+ NAME : đặt tên cho một module hợp ngữ

Ví dụ: NAME Cursor ; đặt tên cho modun là cursor

+ ORG: ấn định địa chỉ bắt đầu cho đoạn chương trình

Ví dụ: ORG 100h

MOV AX,code ; Lệnh này được đặt tại địa chỉ CS:100h

+ PROC và **ENDP**: Khai báo chương trình con

Cú pháp:

<Tênchươngtrìnhcon> **PROC** [Near/Far]

<Tênchươngtrình con> **ENDP**

Ví dụ:

CTCON PROC Near

MOV AX, 10 ; Bắt đầu chương trình con

ADD DX, AX

RET ; Trở về CT chính

CTCON ENDP

+ PUBLIC : khai báo các tên trong module hiện hành mà các module khác có thể sử dụng.

Ví dụ : PUBLIC GIATRI, NH, HIENTHOI

B. Nhóm khai báo dữ liệu:

Tất cả dữ liệu trong chương trình hợp ngữ được trình hợp dịch chuyển sang dạng nhị phân. Do đó giá trị của dữ liệu có thể được viết ở các dạng như: ký tự, thập phân, nhị phân, bát phân hay thập lục phân.

B.1. Cách viết số:

- Hệ thập phân: 10, 150, 1234
- Hệ bát phân: kết thúc bằng ký tự *O* (hay *o*), như: 10o, 35O, 1230O
- Hệ thập lục phân: phải kết thúc bằng ký tự *H* (hoặc *h*) và bắt đầu là số, như: 10h, 13H, 2Fh, 0D4E1h
- Hệ nhị phân: kết thúc bằng ký tự *B* (hoặc *b*), như: 10b, 01100100B

B.2. Chuỗi ký tự

Ký tự hay chuỗi ký tự phải được viết giữa cặp dấu nháy đơn (‘) hoặc cặp dấu nháy kép (“), như: ‘A’, “b”, ‘Hello Assembly’, “This is a string”

Trình hợp dịch chuyển ký tự sang dạng nhị phân tương ứng với mã ASCII, do đó ‘A’, “A”, 41h hay 65 đều có nghĩa như nhau khi lưu trữ.

B.3. Định nghĩa dữ liệu (Khai báo biến)

Cú pháp chung: [Tên Biến] <Loại Biến> <Giá trị>

[Tên biến] đặt theo quy cách của Tên. Tên biến (tên vùng nhớ), thực chất là địa chỉ tượng trưng của vùng nhớ và được chuyển thành địa chỉ thật sau khi dịch chương trình.

<Loại Biến> xác định kích thước của biến theo byte, bao gồm:

DB (Define Byte): khai báo biến 1 byte (1 ô nhớ)

DW (Define Word): khai báo biến 2 byte (1 word máy tính)

DD (Define Double word): khai báo biến 4 byte (word đôi)

DQ (Define Quad word): khai báo biến 8 byte (bốn word).

DT (Define Ten byte): khai báo biến 10 byte.

<Giá trị> bao gồm các dạng:

- Số hay biểu thức: *01100b, 0A1D3h, 15* hay *(50+10h)*9*

- Ký tự hay chuỗi: *'A', 'CHAO BAN'*

- Rỗng (không gán trước giá trị): ***?***

- Mảng có n giá trị khác nhau: *<trị 1>, <trị 2>, ..., <trị n>*

- Mảng có n giá trị giống nhau: n *DUP(<giá trị>)*

Lưu ý: - Độ lớn của giá trị không vượt quá khả năng lưu trữ của vùng nhớ đã định nghĩa bằng <Loại biến>

- Biến chuỗi ký tự phải khai báo bằng DB

Các ví dụ:

- Khai báo biến 1 byte, tên là SO và gán trước trị là 14:

```
SO DB 14
```

- Khai báo mảng M gồm 5 phần tử có trị lần lượt 1, 3, 5, 7, 9:

```
M  DB  1, 3, 5, 7, 9
```

- Khai báo biến 2 byte, lưu trữ giá trị 10, tên là WA:

```
WA   DW   10 ; 10 = 000Ah
```

- Khai báo biến lưu trữ giá trị 1234h, đặt tên là WB:

```
WB DW 1234h
```

- Khai báo biến, lưu trữ chuỗi "Hello", tên là str:

```
str DB 'Hello'    ; Lưu trong bộ nhớ bằng các mã ASCII
```

- Khai báo biến mảng Str2 lưu trữ lần lượt các trị 'ABC', 0Ah, 0Dh, '$':

```
Str2  DB  'ABC', 0Ah, 0Dh, '$'    ; Mãng 6 phần tử 1 byte
```

- Khai báo biến mảng 7 phần tử có cùng giá trị là '$', tên là Str3:

```
Str3 DB 7 DUP('$') ; Mãng 7 phần tử 1 byte
```

Trong hợp ngữ, kiểu của biến được hiểu đơn giản hơn thông qua số lượng ô nhớ của biến và được thể hiện bằng <*Loại Biến*>.

Các biến phải được khai báo trong đoạn dữ liệu (DS), một vài trường hợp đặc biệt vẫn được khai báo trong đoạn lệnh, nhưng phải luu ý đến tổ chức chương trình sao cho CPU không xem các biến như là lệnh. Các biến sẽ được phân phối bộ nhớ theo thứ tự được khai báo lần lượt từ ô nhớ có địa thấp đến cao. Trong một đoạn có nhiều biến được khai báo, biến được khai báo đầu tiên sẽ có địa chỉ độ dời trong đoạn dữ liệu bắt đầu là 0h và tiếp theo cho các biến được khai báo tiếp theo sau.

B.4. Dữ liệu có cấu trúc:

+ RECORD và **STRUC**: Khai báo biến kiểu có cấu trúc mẫu tin.

Cú pháp: [Tên Biến] <Kiểu cấu trúc> [trường: d] [,…]

d: Số nguyên dương, xác định độ lớn của *trường*.

<Kiểu cấu trúc>:

RECORD: d tính bằng bít. Cấu trúc dài tối đa là 16 bít

STRUC: d tính bằng byte.

Ví dụ:

MUC RECORD CAO : 7, VUA : 3, THAP : 4

DIEMSV STRUC TEN: 3, MON1: 1, MON2: 1, MON3:1

B.5. Khai báo hằng

EQU và **=** : gán trị cho 1 hằng

EQU để gán trị cho hằng chỉ một lần khi khai báo.

Muốn gán lại giá trị nhiều lần, ta dùng lệnh "**=**"

Ví dụ: HANG_1 EQU 2*10 ; gán trị 20 vào HANG_1

HANG_2 = 2*10 ; gán trị 20 vào HANG_2

C. Nhóm lệnh giả về dịch (compile) có điều kiện:

Các lệnh giả về dịch có điều kiện nhằm báo cho hợp ngữ tiến hành dịch một nhóm lệnh nếu một điều kiện được thỏa mãn (đúng). Ngược lại sẽ không dịch khi điều kiện không thoả mãn (sai)

Cú pháp chung:

<***Điều kiện***> ; Điều kiện dịch chương trình

Nhóm lệnh A

[***ELSE***]

Nhóm lệnh B

ENDIF ; Hết điều kiện dịch

<*Điều kiện*> có thể có những hình thức sau:

IFE <B-Thức> : Nếu <B-Thức> bằng 0 thì ***nhóm lệnh A*** được dịch.

Nếu <B-Thức> khác 0 thì ***nhóm lệnh B*** được dịch *(nếu có lệnh giả ELSE)*

IF1 : Nếu đang dịch lần 1 thì ***nhóm lệnh A*** được dịch

IF2 : Nếu đang dịch lần 2 thì ***nhóm lệnh A*** được dịch

IFDEF <ký hiệu>: Nếu ký hiệu đã định nghĩa thì dịch ***nhóm lệnh A.***

IFNDEF <ký hiệu>: Nếu ký hiệu không được định nghĩa thì dịch ***nhóm lệnh A***

IFB <ĐốiSố> : Nếu đối số là khoảng trống hoặc không có đối số thì dịch ***nhóm lệnh A.***

IFNB <ĐốiSố> : Nếu có đối số thì dịch ***nhóm lệnh A***

IFIDN <ĐốiSố1>, <ĐốiSố2>: Nếu ĐốiSố1=ĐốiSố2 thì dịch ***nhóm lệnh A***

IFDIF <ĐốiSố1>,<ĐốiSố2>: Nếu ĐốiSố1 ≠ ĐốiSố2 thì dịch ***nhóm lệnh A***

D. Nhóm lệnh giả về MACRO

Cú pháp:

<Tên> MACRO [tham số]

…. ; đoạn chương trình

ENDM ; chấm dứt Macro

Ví dụ : Viết MACRO tên **Agen** với 3 tham số vào X, Y, Z như sau :

Agen MACRO X, Y, Z

MOV AX, X ; AX ← X

ADD AX, Y ; AX ← AX + Y

ADD AX, Z ; AX ← AX + Z

ENDM

Trong chương trình, gọi macro **Agen** bằng lệnh:

Agen 10, 20, 30 ; *X=10, Y=20, Z=30*

Một đoạn chương trình sau đây được xen vào ngay lệnh gọi macro:

MOV AX, 10 ; AX ← 10

ADD AX, 20 ; AX ← AX + 20

ADD AX, 30 ; AX ← AX + 30

Ngoài ra, trong Macro ta có thể dùng lệnh giả **LOCAL, EXITM**:

EXITM : Thoát ra khỏi MACRO trước khi gặp lệnh ENDM.

LOCAL: Định nghĩa các nhãn địa phương trong các MACRO khi chương trinh muốn gọi macro nhiều lần.

Ví dụ: WAIT MACRO count

MOV CX, count

next: ADD AX, 1

LOOP next

ENDM

MACRO nầy chỉ được phép gọi một lần vì nhãn next chỉ có thể xuất hiện trong chương trình một lần. Nếu muốn gọi macro WAIT nhiếu lần thì ta phải thêm lệnh giả local như sau:

WAIT MACRO count

Local next ; *nhãn địa phương*

MOV CX, count

next: ADD AX,1

LOOP next

ENDM

E. Nhóm lệnh giả về liệt kê (listing)

Nhóm lệnh này dùng để quy định việc hiển thị hoặc in ấn file nguồn theo 1 định dạng văn bản. Nhóm lệnh này thường được viết ở đầu chương trình như: Số trang, số cột, tựa đề chương trình. Nhóm lệnh này không được dịch

PAGE số hàng, số cột

Ví dụ : PAGE 58, 60 ; mỗi trang có 58 hàng, 60 cột.

TITLE (Tiêu đề) : cho phép đặt tiêu đề cho chương trình

Ví dụ : TITLE *chương trình hợp ngữ thứ nhất.*

SUBTTL (Subtitle: tiêu đề con): liệt kê tiêu đề con ở mỗi đầu trang.

% OUT <văn bản> : văn bản được liệt kê khi hợp ngữ dịch chương trình. Bao gồm các chỉ thị loại văn bản sau:

.LIST : Liệt kê tất cả dòng lệnh với mã của nó (mặc nhiên)

.XLIST : Không cho liệt kê

.XALL : Liệt kê mã do MACRO tạo nên

.LALL : Liệt kê toàn bộ MACRO

.SALL : Không liệt kê MACRO

.CREF : Liệt kê bảng tham chiếu chéo

.XCREF : Không liệt kê bảng tham chiếu chéo

6.1.3 Toán hạng và toán tử

A. Toán hạng:

Hợp ngữ phân biệt 3 loại toán hạng: *tức thì, thanh ghi và ô nhớ*. Để phân biệt, mỗi loại toán hạng có cách viết khác nhau hoàn toàn.

A.1. Toán hạng tức thì: có thể là một số, biểu thức hay một hằng đã được định nghĩa (bằng lệnh EQU hay **=**).

Toán hạng tức thì có thể được sử dụng theo 4 dạng: thập phân, nhị phân, thập lục phân, bát phân và Ký tự

Cách viết: 10; 10b; 10h; "K"

Ký hiệu tổng quát đại diện cho toán hạng tức thì:

Immed : Toán hạng tức thì nói chung

Immed8 : Toán hạng tức thì 8 bit

Immed16 : Toán hạng tức thì 16 bit

Immed32 : Toán hạng tức thì 32 bit

A.2. Toán hạng thanh ghi: dùng để truy xuất đến nội dung thanh ghi.

Cách viết: AX, BX, CX, DX, AL, AH, SI, DS, ...

Ký hiệu tổng quát đại diện cho toán hạng thanh ghi:

Reg : Toán hạng thanh ghi đa dụng nói chung

Reg8 : Toán hạng thanh ghi đa dụng 8 bit

Reg16 : Toán hạng thanh ghi đa dụng 16 bit

SegReg : Toán hạng thanh ghi đoạn

A.3. Toán hạng ô nhớ: dùng để truy xuất đến nội dung ô nhớ.

Toán hạng ô nhớ thường được tượng trưng một địa chỉ của ô nhớ. Địa chỉ đó luôn luôn là địa chỉ độ dời trong một đoạn tương ứng.

Các cách viết cơ bản:

Tên_Biến : Truy xuất đến nội dung của Biến.

[Immed16] : Truy xuất nội dung của ô nhớ trong đoạn DS, ô nhớ đó có địa chỉ độ dời là *Immed16* . Còn gọi là trực tiếp ô nhớ.

[Reg16] : Truy xuất nội dung của ô nhớ trong đoạn DS, mà ô nhớ đó có địa chỉ độ dời lưu trong *Reg16*. Còn gọi là gián tiếp thanh ghi.

Ví dụ:

MOV AH, MOT ; *AH ← nội dung biến MOT*

MOV DH, [1234h] ; *DH ← nội dung ô nhớ có địa chỉ là 1234h*

MOV AL, [BX] ; *AL ← nội dung ô nhớ có địa chỉ trong BX*

Ký hiệu tổng quát đại diện cho toán hạng ô nhớ:

Mem : Toán hạng ô nhớ nói chung

Mem8 : Toán hạng ô nhớ 8 bit

Mem16 : Toán hạng ô nhớ 16 bit

Mem32 : Toán hạng ô nhớ 32 bit

Ví dụ các cách viết khác của toán hạng ô nhớ:

MOT+5 , MOT[5] , 5[MOT] : chỉ đến địa chỉ "MOT cộng 5" *(MOT là tên của 1 biến)*

B. Toán tử:

Có 4 loại toán tử: Thuộc Tính, Số Học, Quan Hệ và Logic.

Cú pháp chung: <Toán tử> <Toán hạng>

Toán tử tác động lên toán hạng để cho ra kết quả, kết quả này dùng làm toán hạng trong lệnh.

B.1. Toán tử thuộc tính (attribute):

PTR (pointer): dùng thay đổi kiểu của các địa chỉ, số liệu

Ví dụ : CALL WORD PTR [BX+SI] *; Gọi chương trình Con*

[BX+SI] mặc nhiên trỏ tới ô nhớ (dữ liệu) 1 byte, nhưng nếu muốn lấy dữ liệu 2 byte thì phải dùng *WORD PTR* để đổi.

: (Dấu hai chấm): dùng thay đổi đoạn mặc nhiên.

Ví dụ : MOV AH, ES **:** [BX+SI] ; *AH ← ES:[BX+SI]*

[BX+SI] mặc nhiên là địa chỉ độ dời của dữ liệu trong đoạn DS nhưng nếu dữ liệu trong đoạn ES thì phải viết *ES:*[BX+SI]

SHORT: dùng để thay đổi kiểu mặc nhiên là Near của lệnh JMP (lệnh nhảy) và báo cho hợp ngữ biết chỉ nhảy trong vòng từ -128 đến +127 so với vị trí lệnh JMP

THIS : tạo một toán hạng có giá trị tùy thuộc vào tham số của THIS

Ví dụ:

NH EQU THIS BYTE ; tương đương với NH LABEL BYTE

SCH = THIS NEAR ; tương đương với SCH LABEL NEAR

SEG : trả về địa chỉ *đoạn* của một nhãn hay biến

Ví dụ: lấy địa chỉ đoạn của biến TENB vào AX

MOV AX, SEG TENB

OFFSET: trả về địa chỉ *độ dời* của một nhãn hay biến

Ví dụ:

MOV BX, OFFSET FOO ; độ dời của biến FOO

TYPE: Xác định kiểu của biến hay nhãn

TYPE <*biến*>: cho số byte ô nhớ mà loại biến đó được khai báo

(BYTE = 1, WORD= 2, DWORD = 4 ...)

TYPE <*nhãn*>: 0FFFFh nếu nhãn Near; 0FFFEh nếu nhãn Far

LENGTH : Trả về số phần tử của biến mãng khai báo bằng DUP()

SIZE : trả về tổng số byte mà một biến chiếm

Ví dụ: FOO DW 100 DUP (?)

MOV AX, type FOO ; AX ← 2

MOV CX, length FOO ; CX ← 100

MOV BX, size FOO ; BX ← 200

MOV DX, type FOO * length FOO ; DX ← 200

B.2. Toán tử số học:

Ngoài các toán tử số học thông dụng (+, -, *, /) còn có các toán tử:

MOD : chia lấy số dư

SHR : dịch sang trái

SHL : dịch sang phải

– : Dấu trừ đứng trước một số để chỉ đó là số âm: -5, -300

Ví dụ: MOV AX, 100 **MOD** 17 ; *AX ← 15*

MOV AX, 1100000b **SHR** 5 ; *AX ← 11b*

B.3. Toán tử quan hệ:

Toán tử quan hệ dùng so sánh 2 toán hạng và thường được dùng trong việc dịch chương trình có điều kiện

Cú pháp:

<toán hạng 1> <Toán Tử so sánh> <toán hạng 2>

<toán tử>:

EQ (Equal): trả về 1 (TRUE) nếu 2 toán hạng bằng nhau

NE (not equal): trả về TRUE nếu 2 toán hạng không bằng nhau

LT (less than): trả về TRUE nếu (toán hạng 1) nhỏ thua (toán hạng 2).

GT (greater than): trả về TRUE nếu (toán hạng 1) lớn hơn (toán hạng 2)

LE (less than or equal): trả về TRUE nếu (t.hạng 1) nhỏ thua hoặc bằng (t.hạng 2)

GE (greater than or equal): trả về TRUE nếu (t.hạng 1) lớn hơn hoặc bằng (t.hạng 2)

B.4. Toán tử logic: So sánh từng bit tương ứng giữa 2 toán hạng.

NOT : trả về TRUE nếu hai toán hạng logic bên trái và phải khác nhau

AND : trả về TRUE nếu cả hai toán hạng logic đều là TRUE.

OR : trả về TRUE nếu một trong hai toán hạng là TRUE

XOR : trả về TRUE nếu hai toán hạng khác nhau.

Ví dụ:

MOV AH, 10 ; *AH ← 10*

MOV AL, 15 ; *AL ← 15*

MOV DH, AH *EQ* AL ; *DH ← 0 (vì AH ≠ AL)*

MOV DL, AH *EQ* 10 ; *DL ← 1 (vì AH = 10)*

MOV CH, AL *LT* AH ; *CH ← 0 (vì AL < AH)*

MOV BH, AH *AND* 0Ah ; *BH ← 1 (vì AL < AH)*

6.2 Cấu trúc của chương trình hợp ngữ

6.2.1 Cấu trúc chung

Một chương trình hợp ngữ có thể gồm nhiều module. Các module có thể viết riêng lẻ bằng một chương trình xử lý văn bản và dịch riêng lẽ bằng MASM để cho ra các chương trình đích. Các chương trình đích sẽ được chương trình liên kết (LINK) nối lại với nhau thành một chương trình chạy được có phần mở rộng là .EXE. Mỗi module có thể viết theo một cấu trúc tổng quát như sau:

```
PAGE 60, 132   ; Khai báo trang số dòng, số cột
TITLE   Cấu trúc chương trình          ; tiêu đề cho module.
        …. ; các phát biểu EXTRN hay PUBLIC (nếu có).
MCR MACRO ; Khai báo Macro  nếu có
        ..... ; viết macro
ENDM
SSEG SEGMENT PARA STACK 'STACK' ; Khai báo segment Stack
   ……     ; Kích thước Stack
SSEG ENDS
DSEG SEGMENT PARA PUBLIC 'DATA'    ; Khai báo segment dữ liệu
        ….     ; Khai báo biến
DSEG ENDS
CSEG SEGMENT PARA PUBLIC 'CODE' ; Khai báo segnment mã
BATDAU   PROC  FAR
        ASSUME CS:CSEG, DS:DSEG, SS:SSEG ; Phát biểu Assume
        MOV AX, DSEG
        MOV DS, AX                  ; khởi động DS
        ....                        ; các câu lệnh chương trình chính
      CTCon  PROC  NEAR             ; Nếu có
```

```
    ....                    ; viết chương trình con
CTCon   ENDP                ; Kết thúc CTcon
CSEG ENDS
END BATDAU ; Kết thúc chương trình và chọn địa chỉ bắt đầu
```

Đối với các chương trình đơn giản (Chương trình không liên kết với các chương trình khác) người ta còn sử dụng kiểu khai báo segment đơn giản:

```
.MODEL  <Kiểu[TINY/SMALL/MEDIUM/COMPACT/LARGE]>
.STACK    [Kích thước]
.DATA
.CODE     [Têncode]
```

Mỗi 1 chỉ dẫn trên làm cho trình dịch hợp ngữ tự động tạo ra SEGMENT và ENDS theo yêu cầu. Trong đó .MODEL quy định các kiểu bộ nhớ như sau:

TINY: Thường được sử dụng cho chương trình sẽ được hợp dịch và liên kết với chính chương trình này để tạo ra chương trình thực thi dạng .COM, chương trình có vùng dữ liệu, vùng mã và vùng stack ở cùng 1 Segment

SMALL: Số Segment mã và Segment dữ liệu là 1; vùng mã và vùng dữ liệu dài tố đa 64KB

MEDIUM: Số Segment mã lớn hơn 1; số Segment dữ liệu là 1

COMPACT: Số Segment mã là 1; số Segment dữ liệu nhiều hơn 1

LARGE: Số Segment mã lớn hơn 1; số Segment dữ liệu lớn hơn 1

Các lệnh khởi động thanh ghi dữ liệu DS là:

```
MOV AX, @data
MOV DS, AX
```

Chương trình mẫu cho loại này như sau:

```
PAGE 60, 132    ; Khai báo trang có 60 dòng và 132 cột
TITLE Chuong trinh don gian    ; Tiêu đề chương trình
.MODEL SMALL
.STACK 64
.DATA
……..      ;  Khai báo các biến
```

```
.CODE                    ;  Khai báo segment mã
CTCHINH  PROC    FAR
        MOV   AX, @DATA      ; 2 lệnh nạp đoạn dữ liệu cho DS
        MOV   DS, AX
        ……..                 ; Các câu lệnh
        MOV AX,4C00h         ; Lấy giá trị để thoát
        INT 21h ; Thoát về DOS
CTCHINH  ENDP
CTCon   PROC       NEAR
          ……..               ; Các câu lệnh
CTCon  ENDP
END CTCHINH
```

Hợp ngữ MASM cho phép tạo 2 loại tập tin thi hành. Tập tin dạng *.COM dùng cho các chương trình nhỏ và tập tin dạng *.EXE dùng để xây dựng các chương trình lớn. Mỗi loại có cấu trúc chương trình khác nhau.

6.2.2 Tập tin thi hành dạng .COM

- Chỉ dùng một đoạn duy nhất, các thanh ghi CS, DS, ES và SS đều có giá trị giống nhau.

- Kích thước chương trình tối đa là 64K.

- Tập tin .COM được nạp vào bộ nhớ trong và thực hiện nhanh hơn tập tin EXE. Khi DOS thực hiện tập tin COM, nó tạo một vùng ô nhớ từ độ dời 0 đến 0FFh để chứa thông tin cần thiết cho DOS. Vùng này gọi là vùng PSP (program segment prefix) và tất cả các thanh ghi đoạn đều phải chỉ tới vùng PSP này. Vì thế địa chỉ bắt đầu của tập tin COM phải có địa chỉ độ dời là 100h.

Cấu trúc chương trình điển hình của tập tin dạng COM như sau:

```
…..          ; Khai báo hằng số (Nếu có)
MNAME MACRO
…..          ; Viết Macro (Nếu có)
ENDM
CSEG SEGMENT
```

```
ASSUME CS: CSEG, DS: CSEG
ORG 100h
start: MOV AX, CSEG
       MOV DS, AX          ; khởi động DS
       …                   ; (Chương trình chính)
       …                   ; Khai báo dữ liệu (Nếu có)
       MOV AX,4C00h        ; Lấy giá trị để thoát
       INT 21h             ; Thoát về DOS
CT_CON PROC
       …..                 ; Viết Thủ tục (Nếu có)
       RET
CT_CON ENDP
CSEG ENDS
END start
```

6.2.3 Tập tin thi hành dạng .EXE

- Chương trình lớn và nằm ở nhiều đoạn khác nhau.
- Có thể gọi các chương trình con dạng Far.
- Kích thước tập tin tùy ý.
- Có header ở đầu tập tin để chứa các thông tin điều khiển cần thiết.
- Thi hành chậm hơn tập tin dạng COM.

Tập tin dạng EXE, không dùng lệnh ORG 100h ở đầu chương trình.

Cấu trúc chương trình điển hình của tập tin dạng EXE như sau:

```
    ….                  ; Khai báo hằng số (nếu có)
MNAME MACRO             ; (Nếu có)
      …                 ; Viết lệnh Macro
ENDM
DSEG SEGMENT
      …                 ; Khai báo dữ liệu
DSEG ENDS
CSEG SEGMENT
```

```
ASSUME CS: CSEG, DS: DSEG
start: MOV AX, DSEG
       MOV DS, AX          ; khởi động DS
        …….
        …….                ; (Chương trình chính)
        …….
        MOV AH, 4Ch        ; hay MOV AX,4C00h
        INT 21h            ; thoát
CT_CON PROC
    ……..                   ; Viết các Thủ tục (Nếu có)
RET
CT_CON ENDP
CSEG ENDS
END start
```

6.2.4 Ví dụ

Đoạn chương trình hợp ngữ sau thực hiện việc in ký tự 'B' lên màn hình sử dụng các câu lệnh được giải thích như sau:

MOV AH, 02h ; Sử dụng hàm in ký tự của ngắt 21h

MOV DL, 'B' ; DL chứa ký tự cần in

INT 21h ; Gọi ngắt để thực công việc

a). Muốn đoạn chương trình này thực thi trên máy tính ở dạng COM thì phải dùng cấu trúc chương trình dạng COM để viết, như sau:

```
PAGE 60,132
TITLE chuong trinh dang .COM
CSEG SEGMENT
ASSUME CS: CSEG
ORG 100h
Begin: MOV AH, 02h        ; Sử dụng hàm in ký tự của ngắt 21h
       MOV DL, 'B'        ; DL chứa ký tự cần in
       INT 21h            ; Gọi ngắt để thực công việc
       INT 20h
```

```
        CSEG ENDS
    END Begin
```

b) Muốn đoạn chương trình này thực thi trên máy tính ở dạng EXE thì phải dùng cấu trúc chương trình dạng EXE để viết, như sau:

```
    DSEG SEGMENT
    ; Vì không có biến để khai báo nên đoạn DSEG có thể bỏ đi
    DSEG ENDS
    CSEG SEGMENT
    ASSUME CS: CSEG, DS: DSEG
    Begin: MOV AX, DSEG
        MOV DS, AX
        MOV AH, 02h          ; Sử dụng hàm in 1 ký tự của ngắt 21h
        MOV DL, 'B'          ; DL chứa ký tự cần in
        INT 21h              ; Gọi ngắt để thực công việc
        MOV AH, 4Ch          ; AH chứa 4Ch để thoát khi gọi ngắt 21
        INT 21h
    CSEG ENDS
    END Begin
```

c) Viết lại chương trình trên nhưng không khai báo đoạn DSEG, khi đó chương trình chỉ có 1 đoạn duy nhất, như sau:

```
    CSEG SEGMENT
    ASSUME CS: CSEG, DS: CSEG
    Begin: MOV AX, CSEG
        MOV DS, AX
        MOV AH, 02h          ; Sử dụng hàm in ký tự của ngắt 21h
        MOV DL, 'B'          ; DL chứa ký tự cần in
        INT 21h              ; Gọi ngắt để thực công việc
        MOV AH, 4Ch          ; AH chứa trị 4Ch để thoát
        INT 21h
    CSEG ENDS
    END Begin
```

6.2.5 Cách tạo 1 chương trình bằng hợp ngữ

Để tạo một chương trình viết bằng hợp ngữ chạy trên máy tính PC, ta cần phải có bộ trình hợp dịch MASM (hay TASM). Trình hợp dịch MASM được cung cấp gồm 2 tập tin cơ bản: MASM.EXE (hợp dịch) và LINK.EXE (liên kết). Ngoài ra còn một số tập tin khác như EXE2BIN.COM dùng để chuyển sang dạng COM và DEBUG.COM dùng để gở rối chương trình. Các tập tin này được thực thi dưới MSDOS. Cần ba hoặc bốn bước để tạo chương trình hợp ngữ chạy được trên máy tính PC

BƯỚC 1: Soạn thảo tập tin nguồn và lưu tập tin lên đĩa có phần mở rộng là (*.ASM)

Ví dụ: Dùng trình soạn thảo Notepad hoặc trình soạn thảo văn bản TEXT soạn thảo chương trình như các ví dụ trên và lưu vào đĩa có tên VIDU**.ASM**

BƯỚC 2: Hợp dịch file ASM thành file đối tượng (VIDU.OBJ)

Giả sử các tập tin cần thiết: MASM.EXE (hợp dịch) và LINK.EXE (liên kết). EXE2BIN.COM (Chuyển file .EXE thành file .COM) có trong thư mục hiện hành là D:\ Ta ra lệnh hợp dịch để tạo ra file .OBJ như sau

C:\> MASM <filenamenguồn> [filenameđích.OBJ]

Trình hợp dịch kiểm tra lỗi cú pháp trên từng dòng lệnh. Nếu có lỗi thì phải quay về bước 1 để sửa chữa và dịch lại cho đến hết lỗi. Nếu không nêu filenameđích thì trình dịch sẽ lấy tên filenamenguồn với phần mở rộng là .OBJ

Ví dụ: Như ở ví dụ trên ta ra lệnh

D:\MASM vidu

BƯỚC 3: Liên kết một hay nhiều file OBJ thành file thi hành (*.EXE)

D:\> LINK <filename1;> [<filename2;> …] [<filename.exe>]

Ví dụ: Như ở ví dụ trên ta ra lệnh

D:\>LINK VIDU

BƯỚC 4: Đổi từ EXE sang COM (Dành cho chương trình viết dạng COM)

C:\> EXE2BIN <filename.EXE> <filename.COM>

6.3. Một số lệnh của hợp ngữ 16 bit

6.3.1 Toán hạng

Tập lệnh của 8086 có 111 lệnh với chiều dài có từ 1 đến đến vài byte. Các CPU sau này (80x86) được bổ sung thêm nhiều lệnh mới có chức năng phong phú hơn. CPU 80386 có 206 lệnh; CPU Pentium có hơn 400 lệnh. Lệnh của 8086 có tối đa 2 toán hạng, cú pháp chung như sau:

<Thaotác> Toánhạng1 , Toánhạng_2

Trong đó <Thao tác> (Operation) là bắt buộc, Toánhạng1 (Operand1) là toán hạng đích, Toánhạng2 (Operand2) là toán hạng nguồn. Toán hạng nguồn có thể chứa dữ liệu hoặc địa chỉ; toán hạng nguồn không thay đổi bởi thao tác của lệnh. Toán hạng đích chỉ ra nơi lưu trữ dữ liệu là thanh ghi hay trong bộ nhớ và dữ liệu này sẽ được sử dụng để thực thi. Toán hạng có thể là thanh ghi, bộ nhớ. Hai toán hạng phải có cùng kích thước. Ta có các loại toán hạng sau:

- Toán hạng thanh ghi (Register Operand): Với kiểu định địa chỉ này toán hạng là 1 trong các thanh ghi 8bit; 16bit; 32bit của tập các thanh ghi của bộ vi xử lý. Tuỳ thuộc vào lệnh/chỉ thị thanh ghi có thể ở toán hạng 1; toán hạng 2 hoặc cả 2.

```
Ví dụ: Bien_1    DW      ?
MOV  CX,Bien_1     ; Thanh ghi ở toán hạng 1
MOV  Bien_1,BX     ; Thanh ghi ở toán hạng 2
MOV CL,AH          ; Thanh ghi ở cả 2 toán hạng
```

- Toán hạng tức thời (Immediate operand): Với kiểu định địa chỉ tức thời toán hạng nguồn chứa 1 giá trị là hằng số; một biểu thức. Toán hạng đích có thể là 1 thanh ghi hoặc 1 vị trí nhớ

- Toán hạng bộ nhớ trực tiếp (Direct memory operand): Một trong 2 toán hạng tham chiếu đến 1 vị trí nhớ trong bộ nhớ và toán hạng còn lại tham chiếu đến 1 thanh ghi. Không có lệnh/chỉ thị nào có 2 toán hạng đồng thời tham chiếu đến các vị trí trong bộ nhớ.

- Toán hạng gián tiếp bộ nhớ (Indirect memory operand): Là kỹ thuật sử dụng kiểu định địa chỉ Segment:Offset. Các thanh ghi sử dụng cho mục đích này là BX, DI, SI, BP được đặt trong dấu ngoặc vuông [ ]. Các thanh ghi BX, DI, SI thường kết hợp với thanh ghi Segment DS thành DS:BX, DS:DI, DS:SI để xử lý dữ liệu trong

segment dữ liệu. Khi các thanh ghi BX, DI, SI hoặc 1 hằng số được đặt trong dấu ngoặc vuông mà ở trong 1 toán hạng thì chỉ ra rằng đó là địa chỉ Offset của DS.

6.3.2. Nhóm lệnh sao chép dữ liệu, địa chỉ:

1. Lệnh sao chép dữ liệu

Cú pháp:

MOV *Đích, Nguồn*

Trong đó:

Nguồn (toán hạng nguồn) : Reg, Mem, Immed.

Đích (toán hạng đích) : Reg, Mem.

Công dụng: Sao chép dữ liệu nguồn vào đích. Sau khi lệnh được thực hiện thì giá trị toán hạng nguồn và toán hạng đích bằng nhau.

Lưu ý:

- Lệnh MOV không cho phép cả 2 toán hạng là địa chỉ bộ nhớ, toán hạng tức thời cho thanh ghi segment và từ thanh ghi Segment đến thanh ghi Segment

- Lệnh MOV không ảnh hưởng đến thanh ghi trạng thái.

Ví dụ:

```
MOV CX, BX  ; Sao chép nội dung thanh ghi BX vào thanh ghi CX.
                    ;Sau khi thực hiện xong, BX=CX.
MOV DX, 1234h     ;  DL ← 34h, DH ← 12h
MOV DS, AX        ;  DS ← AX
MOV AH, [1234h]   ; AH ← M[DS:1234h]
MOV [1235h], DX   ; M[DS:1235h] ← DL, M[DS:1236h] ← DH
```

2. Lệnh hoán đổi dữ liệu

Cú pháp:

XCHG Đích, Nguồn

Nguồn, đích: Reg, Mem.

Công dụng: Lệnh XCHG hoán đổi dữ liệu giữa nguồn và đích. Cả hai toán hạng không đồng thời là Mem.

Ví dụ:

```
XCHG AL, BH         ; AL ↔ BH
```

XCHG AX, [1235h] ; AL ↔ M[DS:1235h], AH ↔ M[DS:1236h]

3. Lệnh lấy địa chỉ độ dời

Cú pháp:

LEA Reg16, Mem16

Công dụng: Lấy địa chỉ độ dời của biến hay nhãn (mem16) đưa vào thanh ghi đích Reg16 và khởi động thanh ghi đó.

Ví dụ:

LEA DX, bienA ; Lấy độ dời của biến bienA lưu vào DX

6.3.3. Nhóm lệnh tính toán số học.

1. Lệnh cộng hai số nguyên

Cú pháp:

ADD đích, nguồn

Nguồn : Reg, Mem, Immed.

Đích : Reg, Mem.

Công dụng: Lấy toán hạng nguồn cộng toán hạng đích và lưu kết quả vào toán hạng đích.

2. Lệnh trừ hai số nguyên

Cú pháp:

SUB đích, nguồn

Nguồn : Reg, Mem, Immed.

Đích : Reg, Mem.

Công dụng: Lấy toán hạng đích (số bị trừ) trừ toán hạng nguồn (số trừ) và lưu kết quả vào toán hạng đích.

Ví dụ:

ADD AL, 15 ; AL ← AL + 15

ADD AX, DX ; AX ← AX + DX

SUB BX, 15h ; BX ← BX – 15h

3. Lệnh tăng 1 đơn vị (Increment)

Cú pháp:

INC đích

Đích : Reg, Mem. Lệnh này chỉ có 1 toán hạng

Công dụng: Tăng nội dung của toán hạng đích thêm 1 đơn vị.

4. Lệnh giảm 1 đơn vị (Decrmemt)

Cú pháp:

DEC đích

Đích : Reg, Mem.

Công dụng: Giảm nội dung của đích đi 1 đơn vị.

Ví dụ:

```
INC    AL        ; AL ← AL + 1
DEC    AX        ; AX ← AX - 1
INC    bienC     ; bienC ← bienC + 1
```

5. Lệnh nhân hai số nguyên

Cú pháp:

MUL nguồn

Nguồn: Reg, Mem

Công dụng: Nhân thanh ghi tích lũy (AX) với toán hạng nguồn. Tùy vào kích thước của toán hạng nguồn mà CPU thực hiện phép nhân 8 hay 16 bit.

- Nhân 8 bit: dành cho toán hạng nguồn là 8 bit. Khi đó, CPU sẽ lấy thanh ghi AL nhân với toán hạng nguồn và lưu kết quả vào AX.

- Nhân 16 bit: dành cho toán hạng nguồn là 16 bit. Khi đó, CPU sẽ nhân thanh ghi AX với toán hạng nguồn và lưu kết quả vào cặp thanh ghi DX và AX *(nghĩa là kết quả có 32 bit, 16 bit cao lưu vào DX, còn 16 bit thấp lưu vào AX)*

Ví dụ:

```
MUL BL          ; AX ← AL * BL (nhân 8 bit)
MUL BX          ; DX:AX ← AX * BX (nhân 16 bit)
MUL [1235h]     ; AX ← AL * M[DS:1235h] (nhân 8 bit)
```

6. Lệnh chia hai số nguyên

Cú pháp:

DIV nguồn

Nguồn: Reg, Mem

Công dụng: lấy thanh ghi tích lũy (số bị chia) chia cho nguồn (số chia). Tùy vào kích thước của toán hạng nguồn mà CPU thực hiện phép chia 8 hay 16 bit.

- Chia 8 bit: dành cho toán hạng nguồn là 8 bit. Khi đó, CPU sẽ lấy thanh ghi AX chia cho toán hạng nguồn. Thương số của kết quả được lưu vào AL còn phần dư lưu vào AH.

- Chia 16 bit: dành cho toán hạng nguồn là 16 bit. Khi đó, CPU lấy cặp thanh ghi DX:AX (số 32 bit) chia cho toán hạng nguồn. Thương số của kết quả được lưu vào AX còn phần dư thì lưu vào DX.

Ví dụ:

```
DIV DL        ; AL, AH ← AX ÷ DL
DIV BX        ; AX, DX ← DX:AX ÷ BX
```

6.3.4. Nhóm lệnh logic và ghi dịch

1. AND, OR, XOR, TEST: Và, Hoặc, Cộng không nhớ, Kiểm tra bit

Cú pháp:

AND đích, nguồn

OR đích, nguồn

XOR đích, nguồn

TEST đích, nguồn

Nguồn: Reg, Mem hay Immed.

Đích: Reg, Mem.

Công dụng: Thực hiện phép toán logic (and, or, xor) theo từng bit tương ứng giữa toán hạng đích và toán hạng nguồn, lưu giữ kết quả ở toán hạng đích.

- Lệnh TEST thực hiện kiểm tra bit, giống lệnh AND nhưng không lưu giữ lại kết quả mà kết quả của phép toán chỉ ảnh hưởng đến các cờ trạng thái.

Ví dụ:

```
MOV AL, 01101110b    ; AL ← 01101110b
AND AL, 00110110b    ; AL ← AL  AND  01101110b (AL=00100110b)
XOR AX, AX           ; AX ← AX  XOR  AX (Kết quả: AX = 0)
AND DL, 11000011b    ; Xóa 4 bit giữa trong thanh ghi DL
TEST DH, 00010000b   ; Kiểm tra giá trị của bit 4 trong thanh ghi DH.
```

;Kết quả kiểm tra sẽ ảnh hưởng lên cờ Zero (ZF)

2. NOT – Đảo bit (Lấy Bù 1)

Cú pháp:

NOT đích

Đích: Reg, Mem

Công dụng: Đảo ngược từng bit (hay lấy bù 1) toán hạng đích.

Ví dụ:

```
MOV AH, 0Fh        ; AH ← 0Fh (00001111b)
NOT AH             ; AH = 0F0h (11110000b)
```

3. SHL – Dịch trái logic (Shift Left Logical)

Cú pháp:

SHL đích, 1

SHL đích, CL

Đích: Reg, Mem

Công dụng: Dịch toán hạng đích sang trái, có thể dịch 1 bit hoặc nhiều hơn 1 bit, nếu nhiều hơn 1 bit thì thanh ghi CL chứa số bit dịch. số 0 được sử dụng để làm đầy, còn bit bị đẩy ra ngoài lần lượt được đưa vào cờ Carry (CF).

Ví dụ:

```
MOV AL, 01101101b          ; AL = 01101101b
SHL AL, 1                  ; AL = 11011010b và CF=0
```

4. SHR (logical shift right) dịch phải logic.

Cú pháp:

SHR đích, 1

SHR đích, CL

Công dụng: Giống lệnh SHL nhưng bây giờ dịch toán hạng đích sang phải

Ví dụ:

```
MOV AL, 01101101b          ; AL = 01101101b
SHR AL, 1                  ; AL = 00110110b và CF=1
```

5. SAL (Shift arithmetic left): dịch trái số học

Cú pháp:

SAL đích, 1

SAL đích, CL

Đích: Reg, Mem

Công dụng: Giống lệnh SHL

Ví dụ:

```
MOV AL, 01101101b      ; AL = 01101101b
SAL AL, 1              ; AL = 11011011b và CF=0
```

6. SAR (Shift arithmetic right) : dịch phải số học

Cú pháp:

SAR đích, 1

SAR đích, CL

Đích: Reg, Mem

Công dụng: Lệnh này giống SHR nhưng MSB (bit cao nhất – bit dấu) được sử dụng để làm đầy (bảo toàn bit MSB – *Bit dấu*).

Ví dụ:

```
MOV AL, 01101101b      ; AL = 01101101b
SAR AL, 1              ; AL = 00110110b và CF=1
```

7. ROL (Rotate left) : quay vòng sang trái

Cú pháp:

ROL đích, 1

ROL đích, CL

Công dụng: Giống SHL, nhưng thực hiện việc quay vòng, Bit bị đẩy ra ngoài được lấy vào cờ CF và điền vào phần trống của đích

Ví dụ:

```
MOV AL, 01101101b ; AL = 01101101b
ROL AL, 1 ; AL = 11011010b và CF=0
```

8. ROR (Rotate right) : Quay vòng sang phải

Cú pháp:

ROR đích, 1

ROR đích, CL

Đích: Reg, Mem

Công dụng: Lệnh nầy giống ROL nhưng quay vòng sang phải.

Ví dụ:

MOV AL, 01101101b ; AL = 01101101b

SAR AL, 1 ; AL = 00110110b và CF=1

9. RCL (Rotate through carry left): Quay trái qua cờ Carry

Cú pháp:

RCL đích, 1

RCL đích, CL

Công dụng: Giống như lệnh ROL nhưng có sự tham gia của cờ carry (CF). Bit bị đẩy ra ngoài điền vào cờ CF, Giá trị trong cờ CF lần lượt điền vào phần trống của đích. Cờ CF cũng là 1 bit tham gia vào vòng quay

10. RCR (Rotate through carry right): Quay vòng qua cờ sang phải

Cú pháp:

RCR đích, 1

RCR đích, CL

Đích: Reg, Mem

Công dụng: Giống RCL nhưng quay vòng sang phải.

6.3.5. Nhóm lệnh vào ra ngoại vi.

1. IN - Lấy số liệu từ ngoại vi.

Cú pháp: IN AL, địa chỉ cổng 8 bít

IN AL, DX ; DX chứa địa chỉ cổng 16 bit

Ý nghĩa: Lấy dữ liệu ở ngoại vi có địa chỉ 8 bit đưa vào thanh ghi AL. Nếu địa chỉ của

ngoại vi là 16 bít thì phải dùng DX chứa địa chỉ này.

Ví dụ : - Đọc dữ liệu ở ngoại vi 3Fh

IN AL, 3Fh ; AL ← port(3Fh) *(3Fh là địa chỉ cổng 8 bít)*

- Đọc dữ liệu ở ngoại vi 3F8h

MOV DX, 3F8h ; DX ← 3F8h *(3F8h là địa chỉ cổng 16 bit)*

IN AL, DX ; AL ← port(DX)

2. OUT - Đưa số liệu ra ngoại vi.

Cú pháp: OUT địa chỉ cổng 8 bít, AL ; port(8 bit) ← AL

OUT DX, AL ; port(DX) ← AL

Ý nghĩa: Giống lệnh IN nhưng là xuất dữ liệu trong AL ra ngoại vi.

Ví dụ: MOV DX, 04A5h ; DX ← 04A5h

OUT DX, AL ; port(DX) ← AL

6.3.6. Nhóm lệnh hệ thống

Nhóm lệnh hệ thống bao gồm các lệnh điều khiển hoạt động của CPU hay thiết lập giá trị các cờ. Các lệnh này không có toán hạng, do đó cú pháp của lệnh rất đơn giản chỉ gồm các tác vụ lệnh sau:

CLC ; Xóa cờ CF (CF = 0) – *Clear Carry flag*

CMC ; Đảo ngược cờ CF – *Complement Carry flag*

CLD ; Xóa cờ hướng (DF=0) – *Clear Diretion flag*

STD ; Thiết lập cờ hướng (DF=1) – *Set Diretion flag*

CLI ; Xóa cờ ngắt (IF=0) – *Clear Interrupt flag*

STI ; Thiết lập cờ ngắt (IF=1) – *Set Interrupt flag*

HLT ; Dừng mọi hoạt động của CPU (treo máy) – *Halt*

INT ; Gọi ngắt phần mềm – *Interrupt*

IRET ; Trở về chương trình chính từ chương trình phục vụ ngắt – *Return from Interrupt*

LOCK ; Khóa Bus ngoài. CPU sẽ không giao tiếp với BUS

NOP ; Không có tác vụ – *No operation*

WAIT ; Tạm dừng CPU cho đến khi có tín hiệu điện (mức thấp) ở chân TEST của CPU-8086 thì hoạt động tiếp.

6.3.7 Nhóm lệnh nhảy (Chuyển điều khiển)

Cách thực hiện các lệnh trong chương trình của bộ xử lý là tuần tự thực hiện các lệnh từ trên xuống, hết lệnh này đến lệnh khác. Lệnh nhảy là lệnh làm thay đổi thứ tự thực hiện lệnh của bộ xử lý. Đây cũng là cơ sở của việc xây dựng các cấu trúc rẽ nhánh trong hợp ngữ.

Có 2 loại lệnh nhảy: nhảy không có điều kiện và nhảy có điều kiện.

1. Lệnh nhảy không điều kiện

Cú pháp:

JMP Đích

Đích: Immed16, Reg16, Mem16, là 1 định danh

Ý nghĩa: Luôn luôn nhảy đến thực hiện lệnh có địa chỉ là Đích. Phạm vi nhảy là trong đoạn (near jump).

Ví dụ: Xem đoạn chương trình sau (Để phân biệt các lệnh được đánh số)

1. MOV AX, 1

2. JMP cong2 ; nhảy đến lệnh có nhãn cong2

3. cong1: INC AX

4. JMP setAX ; nhảy đến lệnh có nhãn setAX

5. cong2: ADD AX, 2

6. setAX: ……

Các dạng của lệnh JMP:

• JMP SHORT Đích ; (short jump). Nhảy trong phạm vi từ –128 đến +127 byte so với vị trí hiện tại.

Ví dụ: JMP SHORT Calculate

• JMP FAR PTR Đích ; (far jump). Nhảy đến bất kì chỗ nào, có thể khác đoạn.

Ví dụ: JMP FAR PTR Calculate

• JMP <con trỏ 2 byte> ; (near indirect jump). Khi thực hiện, thanh ghi PC sẽ được gán bằng giá trị lưu tại địa chỉ này. Có thể kết hợp dùng với định vị chỉ số.

Ví dụ: có khai báo biến myPointer như sau:

myPointer DW   Prepare, Calculate, Check, Output

Xem đoạn lệnh:

MOV BX, 2 ; chỉ số trong mảng con trỏ

SHL BX, 1 ; nhân đôi

JMP myPointer[BX] ; nhảy đến Check

...

Prepare: ……. ; công việc 0

.....

Calculate: ……. ; công việc 1

.....

Check: ……. ; công việc 2 – nơi cần nhảy đến

..... ;

Output: ……. ; công việc 3

• JMP <con trỏ 4 byte> ; (far indirect jump). Tương tự trường hợp trên, nhưng con trỏ gồm cả segment và offset. Chỉ khác ở khai báo con trỏ

• JMP <thanh ghi 2 byte> ; (indirect jump via regs). Nhảy đến địa chỉ lưu trong thanh ghi AX.

Ví dụ: MOV AX, offset Calculate

JMP AX ; IP ← AX *(nhảy đến lệnh có địa chỉ trong AX)*

2. Lệnh nhảy có điều kiện:

Lệnh nhảy có điều kiện sẽ thực hiện việc nhảy đến nhãn hay không tùy thuộc vào điều kiện của lệnh nhảy. Lệnh nhảy có điều kiện chỉ nhảy trong phạm vi 128 byte. Muốn nhảy xa hơn, phải kết hợp với lệnh JMP.

Cú pháp tổng quát:

Jxx *Đích*

Trong đó: ***xx*** là các ký tự mô tả điều kiện

Đích: Immed, Reg, Mem

Công dụng: Nếu điều kiện đúng thì nhảy tới thực hiện lệnh toán hạng *Đích*, ngược lại thì tiếp tục thi hành lệnh ngay sau lệnh nhảy.

Các lệnh nhảy có điều kiện được tạo ra bằng cách ghép chữ J (JUMP) với các chữ cái mô tả điều kiện để nhảy. Các điều kiện này thường được thiết lập thông qua trạng thái thanh ghi cờ.

Ví dụ: Lệnh JNE (Jump if Not Equal), nhảy nếu không bằng, điều kiện này đúng khi cờ ZF = 0. Như vậy, giá trị các cờ trạng thái hiện tại là điều kiện để lệnh nhảy xem xét khi thi hành. Thông thường, lệnh so sánh (CMP) hay lệnh TEST được sử dụng để thiết lập điều kiện lệnh nhảy.

Lệnh so sánh:

Cú pháp:

CMP Trái, Phải

Nếu Trái > Phải ⇨ Trái - Phải > 0 thì cờ CF = 0 và ZF = 0

Nếu Trái < Phải ⇨ Trái - Phải < 0 thì cờ CF = 1 và ZF = 0

Nếu Trái = Phải ⇨ Trái - Phải = 0 thì cờ CF = 0 và ZF = 1

Trong đó: Trái, Phải: Immed, Reg, Mem

Bản chất của lệnh CMP là lệnh SUB (thực hiện phép tính Đích – Nguồn) nhưng kết quả của phép tính không được lưu vào Đích như trong lệnh SUB mà tính chất của kết quả được thể hiện thông qua cờ

Ví dụ: so sánh hai số nguyên dương

```
MOV AH, 1      ; AH ← 1
MOV AL, 2      ; AL ← 2
CMP AH, AL   ; CF ← 1, ZF ← 0 vì AH < AL
```

Sau khi thực hiện các lệnh trên, cờ Carry bật (CF=1) nghĩa là AH < AL

Ví dụ: kiểm tra hai bit cuối cùng của AL

```
TEST AL, 3 ; 3h = 00000011b
```

Nếu cờ Zero bật (ZF=1), có nghĩa là hai bit 0 và 1 của AL đều bằng 0

Bảng sau trình bày các lệnh nhảy thường dùng

Lệnh	**Ý nghĩa**	**Điều kiện**
JB JNAE JC	Nhảy nếu nhỏ hơn (Jump if Below) Nhảy nếu không lớn hơn hoặc bằng Nhảy nếu có nhớ	CF = 1
JAE JNB JNC	Nhảy nếu lớn hơn hoặc bằng (Jump if Above or Equal) Nhảy nếu không nhỏ hơn Nhảy nếu không có nhớ	CF = 0
JBE JNA	Nhảy nếu nhỏ hơn hoặc bằng (Jump if Below or Equal) Nhảy nếu không lớn hơn	CF = 1 hay ZF = 1
JA JNBE	Nhảy nếu lớn hơn (Jump if Above) Nhảy nếu không nhỏ hơn hoặc bằng	CF = 0 và ZF = 0
JE JZ	Nhảy nếu bằng (Jump if Equal) Nhảy nếu bằng (Jump if Zero)	ZF = 1

JNE JNZ	Nhảy nếu không bằng (Jump if Not Equal) Nhảy nếu không bằng (Jump if Not Zero)	ZF = 0
JL JNGE	Nhảy nếu nhỏ hơn Nhảy nếu không lớn hơn hoặc bằng	SF ≠ OF
JGE JNL	Nhảy nếu lớn hơn Nhảy nếu không nhỏ hơn	SF = OF
JLE JNG	Nhảy nếu nhỏ hơn hay bằng Nhảy nếu không lớn hơn	ZF=1 và SF ≠ OF
JG JNLE	Nhảy nếu lớn hơn Nhảy nếu không nhỏ hơn hay bằng	ZF=0 và SF = OF
JP JPE	Nhảy nếu có bít kiểm chẵn lẽ Nhảy nếu kiểm tra chẵn	PF = 1
JNP JPO	Nhảy nếu không có bít kiểm chẵn lẽ Nhảy nếu kiểm tra lẻ	PF = 0
JS JNS JO JNO	Nhảy nếu có dấu Nhảy nếu không có dấu Nhảy nếu có tràn Nhảy nếu không có tràn	SF = 1 SF = 0 OF = 1 OF = 0
JCXZ	Nhảy nếu CX = 0	CX = 0

Ví dụ :

```
MOV AX, 1000h      ; AX=1000h
CMP AX, 200h       ; so sánh AX với 200h
JZ NH              ; Nếu AX=200h thì nhảy đến nhãn NH
MOV CX, BX         ; Nếu AX ≠ 200h thì thi hành các lệnh khác
NH: ADD AX, BX ; Vị trí nhảy đến được gắn nhãn NH
```

Khi sử dụng lệnh nhảy có điều kiện sau khi thực hiện phép so sánh, phải đặc biệt lưu ý toán hạng trong phép so sánh là số có dấu (signed) hay không có dấu (unsigned) để lựa chọn lệnh cho phù hợp.

Ví dụ:

```
MOV AH, AL     ; Giả sử AL = 128
CMP AH, 1
JGE Greater    ; AH > 1 nhưng không nhảy
. . .
Greater: …….
```

Ví dụ: nếu AL là số nguyên không dấu thì đoạn chương trình ở trên phải sửa lại như sau:

```
MOV AH,AL
CMP AH,1
JAE Greater
. . . . . .
Greater: . . . . . .
```

Ví dụ: Viết chương trình nhận 1 ký tự từ bàn phím, sau đó:

- Nếu ký tự nhận được là 'C' thì in ra: "Welcome to C!"
- Nếu ký tự nhận được là 'A' thì in ra: "Welcome to Assembly!"
- Nếu là ký tự khác thì thoát khỏi chương trình.

```
DSEG SEGMENT
        msg DB 10, 13, "Hay nhap 1 ky tu: $"
        msgC DB 10, 13, "Welcome to C!$"
        msgA DB 10, 13, "Welcome to Assembly!$"
DSEG ENDS
CSEG SEGMENT
    ASSUME CS: CSEG, DS: DSEG
    start: MOV AX, DSEG
    MOV DS, AX
    MOV AH, 09h
    LEA DX, msg
    INT 21h
    MOV AH, 01h
    INT 21h
```

```
        CMP AL, 'C'
        JE in_C
        CMP AL, 'A'
        JE in_A
        JMP exit
in_C:   MOV AH, 09h
        LEA DX, msgC
        INT 21h
        JMP start
in_A:   MOV AH, 09h
        LEA DX, msgA
        INT 21h
        JMP start
exit:   MOV AH, 4Ch
        INT 21h
CSEG ENDS
        END start
```

6.3.8 Nhóm lệnh lặp

Vòng lặp là một đoạn chương trình được thực hiện nhiều lần cho đến khi thỏa một điều kiện nào đó thì dừng lại, do đó vòng lặp thường kết thúc bằng một lệnh nhảy có điều kiện. Tuy nhiên, ngoài các lệnh nhảy đã biết, Intel-8086 còn cung cấp thêm các lệnh vòng lặp như LOOP, LOOPE, LOOPZ, LOOPNE, LOOPNZ… các lệnh này đều có cú pháp giống nhau.

Cú pháp:

LOOP *Đích*

Trong đó: Đích có thể là: Immed, Reg, Mem

Công dụng: Tự động giảm CX một đơn vị, nếu CX ≠ 0 thì nhảy đến Đích, ngược lại nếu CX = 0 thì không nhảy đến Đích mà thực hiện lệnh ngay sau LOOP. Nói cách khác, vòng lặp LOOP dừng lại khi CX=0. Đây là vòng lặp *for* có số lần lặp được xác định trong CX.

Cấu trúc vòng lặp for viết bằng LOOP:

MOV CX, n ; n là số lần lặp

nhan: …….. ; Các lệnh cần lặp lại

……...

LOOP *nhan*

Ví dụ: Đoạn lệnh in ra màn hình các ký tự từ A đến Z

MOV DL, 'A' ; DL ← 'A'

MOV CX, 26 ; A→Z: 26 ký tự

inkytu: MOV AH, 02h

INT 21h

INC DL ; DL tăng lên 1 để có ký tự kế

LOOP **inkytu**

Lặp với LOOPZ/LOOPE:

Tự động giảm CX một đơn vị, nếu CX ≠ 0 và ZF = 1 thì nhảy đến Đích để thực hiện lệnh. Ngược lại, nếu CX = 0 hay ZF = 0 thì không nhảy, khi đó lệnh viết sau lệnh nhảy được thực hiện.

Cấu trúc vòng lặp do … while viết bằng LOOPZ:

MOV CX, n

nhan: <Các lệnh cần lập>

LOOPZ *nhan*

Lặp với LOOPNZ/LOOPNE:

Tự động giảm CX một đơn vị, nếu CX ≠ 0 và ZF = 0 thì nhảy đến Đích để thực hiện lệnh. Ngược lại, nếu CX = 0 hay ZF = 1 thì không nhảy, khi đó lệnh viết sau lệnh nhảy được thực hiện

Cấu trúc vòng lặp do … while viết bằng LOOPNZ:

MOV CX, n

nhan: <Các lệnh cần lập>

LOOPNZ *nhan*

Lưu ý: Khi sử dụng các lệnh vòng lặp cần phải chú ý đến giá trị của CX.

- Nếu CX=0, vì LOOP giảm CX trước khi kiểm tra nên khi thực hiện lệnh LOOP thì CX = CX–1 = 0–1 = –1 = 0FFFFh. Như vậy LOOP sẽ thực hiện thêm 65535 lần nữa.

- Lệnh JCXZ nhảy khi CX = 0 thường được dùng để kiểm tra giá trị CX trước khi thực hiện vòng lặp.

Ví dụ: Nhập mảng A gồm 10 ký tự, dừng lại nếu gặp phím Enter

```
MOV SI, 0          ; chỉ số mảng
MOV CX, 10         ; số lần lặp
LAP: MOV AH, 1     ; nhập ký tự
INT 21H
MOV A[SI], AL
INC SI
CMP AL, 0Dh
LOOPNE LAP
```

Ví dụ: Viết chương trình sử dụng hàm 01/21h để nhập chuỗi ký tự dài tối đa 128 ký tự hoặc kết thúc bằng phím Enter

```
DSEG SEGMENT
chuoi DB 128 DUP(?)
DSEG ENDS
CSEG SEGMENT
ASSUME CS: CSEG, DS: DSEG
start: MOV AX, DSEG
MOV DS, AX
LEA SI, chuoi ; SI ← địa chỉ chuỗi
MOV CX, 128 ; Chiều dài chuỗi tối đa (số vòng lặp)
key_in: MOV AH, 01h ; Hàm nhập 1 ký tự
INT 21h
MOV [SI], AL ; Cất ký tự vào biến chuỗi
INC SI
CMP AL, 0Dh ; Ký tự vừa nhập là Enter? nếu không phải Enter
```

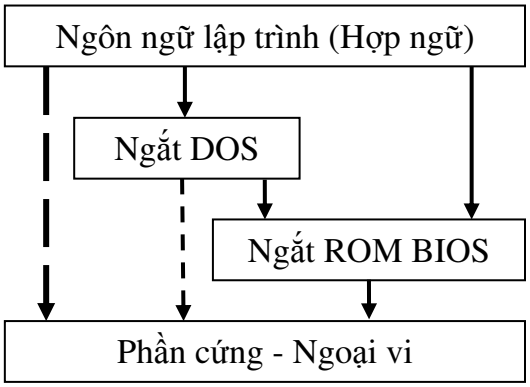
```
LOOPNE key_in ; hay chưa đủ 128 ký tự thì nhập tiếp
MOV AH, 4Ch
INT 21h
CSEG ENDS
END start
```

6.4. Một số ngắt của DOS và BIOS

6.4.1. Tổng quan về ngắt

Một bí quyết để lập trình thành công và hiệu quả cao trên máy tính kiểu IBM PC là việc sử dụng có hiệu quả các chương trình phục vụ của BIOS (Basic Input Output System) được cài đặt sẵn ngay trong vùng nhớ ROM-BIOS của máy tính hay các chương trình phục vụ của hệ điều hành DOS. Ngắt mềm (hay Ngắt) là cách gọi tên ngắn gọn của các chương trình phục vụ này.

Ngắt mềm là chương trình mã máy nhị phân (chính xác hơn là chương trình con) được thiết kế sẵn và tồn tại trong bộ nhớ của máy tính nhằm thực hiện các chức năng điều khiển phần cứng máy tính mỗi khi được gọi thực hiện. Khi đó, để điều khiển phần cứng nào thì người lập trình chỉ cần gọi các ngắt tương ứng để thực hiện công việc mà không cần phải biết đến cấu trúc của phần cứng là như thế nào, nguyên lý vận hành của phần cứng đó ra sao…. đây là công việc rất khó khăn đối với người làm phần mềm không quen thuộc với phần cứng là những mạch điện tử.



Hình 6.1: Giao tiếp phần cứng

Các ngắt mềm được xem như là cầu nối giữa phần cứng và các ngôn ngữ lập trình (kể cả hệ điều hành). Chúng làm việc trực tiếp với phần cứng và các thiết bị ngoại vi, thực hiện những chức năng cơ bản nhất của hệ thống như đọc/ghi từng byte dữ liệu ở bàn phím, ra màn hình hoặc đĩa từ…

Hình 6.1 trình bày các cách giao tiếp với phần cứng của ngôn ngữ lập trình thông qua các ngắt hay trực tiếp với các thiết bị phần cứng. Giao tiếp trực tiếp sẽ khó khăn hơn rất nhiều so với giao tiếp thông qua các Ngắt.

Các ngắt của BIOS luôn sẵn có trong máy tính IBM-PC, còn các ngắt DOS chỉ tồn tại đối với hệ điều hành DOS. Nghĩa là, nếu máy tính IBM-PC được cài đặt hệ điều hành UNIX thì trên máy đó không tồn tại các ngắt DOS. Hệ điều hành Windows được phát triển trên cơ sở của hệ điều hành DOS nên các ngắt DOS vẫn sử dụng được trên hệ điều hành Windows. Tất nhiên có một vài ngắt DOS không được phép sử dụng vì để đảm bảo độ an toàn cho vận hành của hệ điều hành Windows. Tài liệu này chỉ giới thiệu những ngắt rất cơ bản và thường dùng.

6.4.2. Sử dụng ngắt trong hợp ngữ

Khi sử dụng ngắt cần lưu ý một số yếu tố sau:

- Chức năng của ngắt/hàm.
- Tên ngắt/hàm: Dùng số hex từ 0h đến 0FFh
- Tham số vào: Thanh ghi hay bộ nhớ do hàm qui định.
- Tham số ra: Thanh ghi hay bộ nhớ do hàm qui định.

Lệnh gọi 1 ngắt hay 1 hàm:

```
MOV AH, <tên hàm>
.... ; Khai báo tham số vào (nếu có)
INT <tên ngắt>
.... ; Lưu trữ tham số ra (nếu có)
```

Ví dụ: Hàm 02h của ngắt 21h thực hiện việc in ký tự trong thanh ghi DL ra màn hình. Như vậy, khi muốn hiển thị ký tự B trên màn hình, ta viết như sau đoạn lệnh như sau:

```
MOV AH, 02h        ; Chọn hàm 02h
MOV DL, 'B'        ; DL chứa ký tự cần in 'B'
INT 21h            ; Gọi ngắt thực hiện để in ký tự
```

Lưu ý: Việc sử dụng thanh ghi DL để chứa ký tự cần in là do hàm 02h qui định, không thể sử dụng thanh ghi khác để thay thế.

6.4.3. Ngắt MS-DOS

Các ngắt của DOS được đánh số từ 20h đến 0FFh. Ngắt quan trọng nhất là ngắt 21h – gồm nhiều chức năng (hàm) của DOS.

INT 20h: Chấm dứt chương trình và trở về DOS từ một chương trình có phần mở rộng .COM.

INT 23h: Chặn tổ hợp phím Ctrl- Break

Người sử dụng có thể chấm dứt sớm một chương trình bằng cách bấm tổ hợp phím Ctrl-Break. Tổ hợp phím nầy làm CPU nhảy tới chương trình phục vụ ngắt 23h và sau đó đến một thủ tục của DOS để kết thúc chương trình đang chạy và trở về DOS. Nếu muốn không cho phép chương trình kết thúc sớm bằng Ctrl-Break, ta có thể cho ngắt 23h trỏ đến một chương trình đặt biệt nào đó để thông báo lên màn hình rằng không cho chương trình đang chạy kết thúc sớm.

INT 27h: Kết thúc và đặt thường trú

Ngắt này cho phép lưu trong bộ nhớ trong một chương trình nào đó cho đến khi tắt máy. Để sử dụng ngắt 27h, ta cho chạy một chương trình gồm 2 phần. Một phần sẽ phụ trách việc thường trú cho phần kia.

INT 21h: Các chức năng của DOS

Hàm 01h Đợi đọc 1 ký tự từ bàn phím *(có hiện trên màn hình)*

Vào: Không

Ra AL : Mã ASCII của phím nhận được

Đợi đọc một ký tự từ bàn phím (phím nhận được sẽ hiển thị trên màn hình). Khi một phím được ấn thì ký tự tương ứng với phím đó được lưu trong thanh ghi AL. Nếu phím được ấn là một phím đặt biệt thì AL=0. Tổ hợp phím Ctrl-Break kết thúc công việc của hàm này.

Hàm 02h: Hiển thị 1 ký tự tại vị trí con trỏ màn hình

Vào DL ← Mã ASCII của ký tự cần hiển thị

Ra: Không

Hàm 03h: Đọc dữ liệu vào từ cổng nối tiếp chuẩn (COM1)

Vào Không

Ra AL : byte dữ liệu đọc được

Hàm 04h: Xuất ký tự ra cổng nối tiếp chuẩn (COM1)

Vào DL ← ký tự cần xuất

Ra Không

Hàm 05h: Xuất byte dữ liệu ra cổng máy in chuẩn (LPT1)

Vào DL : Byte dữ liệu cần xuất

Ra Không

Hàm 06h: Nhập / Xuất từ thiết bị chuẩn (không đợi)

Vào DL ← Mã ASCII ký tự cần xuất lên màn hình

Nếu DL ← 0FFh : Thực hiện chức năng nhập ký tự

Ra Nếu ZF = 0 thì AL chứa Mã ASCII ký tự nhận được

Nếu ZF = 1 thì không nhận được ký tự

Hàm 07h Giống hàm 1, nhưng không hiện ký tự ra màn hình

Vào Không

Ra AL : Mã ASCII ký tự nhận được

Hàm 08h Giống hàm 1, nhưng không hiện ký tự ra màn hình

Vào Không

Ra AL : Mã ASCII ký tự nhận được

Lưu ý: Ctrl-Break không ảnh hưởng đến hoạt động của hàm 07h và 08h

Hàm 09h Xuất 1 chuỗi ký tự lên màn hình

Vào DS:DX ← địa chỉ logic của chuỗi ký tự cần xuất

Ra Không

Lưu ý: DS ← địa chỉ segment của chuỗi trong bộ nhớ

DX ← địa chỉ offset của chuỗi trong bộ nhớ

Chuỗi ký tự phải chấm dứt bằng ký tự '**$**'

Ví dụ: In chuỗi "Hello Assembly" ra màn hình.

Khai báo biến chứa chuỗi trong đoạn dữ liệu DS:

chuoi DB 'Hello Assembly**$**'

Viết chương trình trong đoạn lệnh:

```
MOV AH, 09h     ; Gọi hàm 09h
LEA DX, chuoi   ; DX ← Offset(chuoi)
INT 21h         ; Thực hiện hàm
```

Hàm 0Ah Đợi đọc 1 chuỗi ký tự từ bàn phím, kết thúc bằng Enter

Vào DS:DX ← địa chỉ của vùng đệm bàn phím trong bộ nhớ

Ra LEN: Tổng số ký tự nhận được

BUFF: Mã ASCII của các ký tự nhận được

Hàm 0Ah nhận 1 chuỗi ký tự từ bàn phím, kết thúc bằng phím Enter (Mã ASCII là 0Dh). Hàm trả về chuỗi nhận được và chiều dài chuỗi trong vùng đệm bàn phím. Vùng đệm bàn phím là 1 biến mảng trong bộ nhớ gồm 3 phần MAX, LEN và BUFF có cấu trúc như sau:

MAX	LEN	BUFF	BUFF+1	BUFF+2	BUFF+…	BUFF+n

MAX: chứa số ký tự tối đa có thể nhận được gán khi khai báo.

LEN: chứa tổng số ký tự của chuỗi nhận được, do hàm trả về sau khi nhận được chuỗi.

BUFF: chứa mã ASCII của từng ký tự trong chuỗi nhận được *(mỗi ký tự chiếm 1 byte nhớ)*. Như vậy, số byte của BUFF phải bằng hay lớn hơn giá trị của MAX.

Ví dụ: Nhận chuỗi tối đa 30 ký tự từ bàn phím

Khai báo vùng đệm trong đoạn dữ liệu:

```
MAX DB 30 ; Tạo biến MAX
LEN DB ?     ; Tạo biến LEN
BUFF DB 31 DUP(?) ; Vùng nhớ lưu trữ ký tự
```

Viết chương trình trong đoạn lệnh:

```
MOV AH, 0Ah
LEA DX, MAX ; DX ← Đ/c vùng đệm bàn phím
INT 21h
```

Cách khai báo biến vùng đệm bàn phím khác:

```
BANPHIM DB 30, ?, 31 DUP(?)
```

Hàm 0Bh Cho trạng thái bàn phím

Vào DS:DX ← địa chỉ logic của vùng đệm bàn phím trong bộ nhớ

Ra LEN: Tổng số ký tự nhận được

BUFF: Mã ASCII của các ký tự nhận được

Hàm 3Ch Tạo tập tin mới

Vào:

DS:DX ← Địa chỉ chuỗi tên của tập tin mới.

CX ← thuộc tính của tập tin (0: bình thường, 1: chỉ đọc, 2: ẩn, 4: tập tin hệ thống)

Ra AX : Thẻ tập tin

Hàm 3Dh Mở tập tin trên đĩa

Vào DS:DX ← Địa chỉ chuỗi tên của tập tin mới.

AL ← kiểu truy xuất tập tin (0: chỉ đọc, 1: chỉ ghi, 2: đọc/ghi)

Ra AX : Thẻ tập tin

Khai báo tên tập tin và biến chứa thẻ file:

```
Tenf   DB   "C:\DULIEU\DATA.TXT",0
Thef DW ?
```

Chuỗi Tên tập tin có chứa cả tên đường dẫn của file. Nếu không có tên đường dẫn thì sẽ truy xuất trong thư mục hiện hành.

Thẻ tập tin là thành phần quan trọng trong việc truy xuất file nên sau khi mở hay tạo file phải lưu trữ lại để sử dụng trong việc đọc/ghi file. Mỗi file phải có một thẻ file, biến thẻ file phải là 2 bytes.

```
MOV Thef,AX       ;Lưu trữ thẻ file vào biến.
MOV BX,Thef       ;Đưa thẻ file vào BX để truy xuất file
```

Có thể mở hay tạo nhiều tập tin khác nhau trong cùng chương trình để xử lý. Khi đó, mỗi thẻ tập tin phải được lưu trữ trong mỗi biến khác nhau.

Hàm 3Eh Đóng tập tin đang mở.

Vào BX ← Thẻ tập tin

Ra AX : Mã lỗi (nếu có)

Sau khi xử lý xong tập tin và trước khi thoát khỏi chương trình, tập tin phải được đóng lại để cập nhật thông tin hoặc nội dung mới của tập tin.

Hàm 3Fh Đọc nội dung tập tin vào bộ nhớ

Vào

DS:DX ← Địa chỉ vùng nhớ chứa dữ liệu đọc được

BX ← Thẻ tập tin cần đọc

CX ← số byte cần đọc

Ra AX : Số byte đọc được

Hàm 40h Ghi dữ liệu trong bộ nhớ vào tập tin

Vào

DS:DX ← Địa chỉ vùng nhớ chứa dữ liệu cần ghi

BX ← Thẻ tập tin cần đọc

CX ← số byte cần ghi

Ra AX : Số byte ghi được

Các hàm 3Fh và 40h cũng có thể được sử dụng đọc hoặc ghi dữ liệu cho ngoại vi, vì DOS quản lý việc truy xuất ngoại vi bằng thẻ

Hàm 41h Xóa tập tin trên đĩa

Vào DS:DX ← Địa chỉ chuỗi tên tập tin

Ra AX : Mã lỗi (nếu có)

Hàm 42h Đời con trỏ tập tin hiện hành

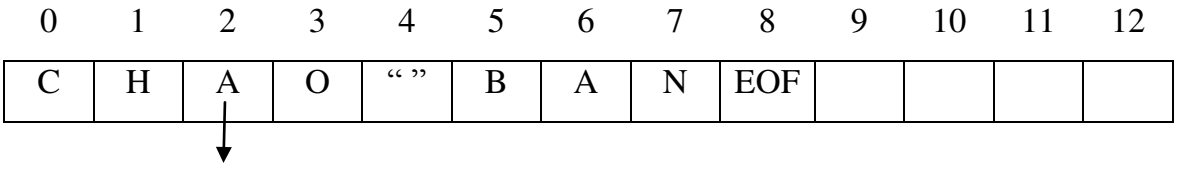
Vào AL ← Hướng dời

CX:DX ← Cự ly dời = (CX*65536) + DX

BX ← Thẻ tập tin

Ra DX:AX : Vị trí con trỏ mới = (DX*65536)+AX

Cấu trúc dữ liệu trong tập tin:



Con trỏ tập tin hiện hành

Hàm 43h Đọc hoặc thay đổi thuộc tính tập tin trên đĩa

Vào

DS:DX ← Địa chỉ chuỗi tên tập tin

Nếu Đọc thuộc tính: AL ← 0

Nếu Thay đổi thuộc tính: AL ← 1 và CX ← thuộc tính mới

Ra Nếu Đọc thuộc tính: CX : chứa thuộc tính tập tin

Hàm 4Ch Kết thúc chương trình và trở về DOS

Vào Không

Ra Không

Hàm 56h Đổi tên tập tin

Vào DS:DX ← Địa chỉ chuỗi tên tập tin cũ

ES:DI ← Địa chỉ chuỗi tên tập tin mới

Ra Không

6.4.4 Ngắt của BIOS

INT 10h: Ngắt cho màn hình

Hàm 0: Chọn chế độ hiển thị cho màn hình

Vào: AH=0, AL = Chế độ

Ra: Không

Hàm 1h: Thay đổi kích thước con trỏ

Vào:

AH=1h

4 bit thấp của CH = dòng quét đầu

4 bit cao của CH = dòng quét cuối

Ra: Không

Hàm 2h: Dịch chuyển con trỏ

Vào: AH=2

BH=trang số

DH= hàng

DL = cột

Ra: không

Hàm 3h: Xác định vị trí và kích thước hiện thời của con trỏ

Vào AH=3, BH=trang số

Ra: CH=dòng quét đầu

CL=dòng quét cuối

DH=dòng

DL=cột

Hàm 5h: Chọn trang hiển thị

Vào: AH=5

AL=trang số

DH=dòng

DL=cột

Ra: Không

Hàm 6h: Cuộn màn hình/cửa sổ lên một số dòng xác định

Vào: AH=6

AL = Số dòng cuốn (AL=0 cuốn cả màn hình hay cửa sổ)

BH=thuộc tính của các dòng trống

CH,CL=dòng,cột của góc trái trên cửa sổ

DH,DL=dòng, cột của góc phải dưới

Ra: Không

Hàm 7: Cuốn cửa sổ hay màn hình xuống

Vào: AH=7

AL = Số dòng cuốn (AL=0 cuốn cả màn hình hay cửa sổ)

BH=thuộc tính của các dòng trống

CH,CL=dòng,cột của góc trái trên cửa sổ

DH,DL=dòng, cột của góc phải dưới

Ra: Không

Hàm 0Ah: Viết ký tự tại vị trí con trỏ

Vào: AH=0Ah

AL=Mã của ký tự

BH=Trang số

CX=số lần viết ký tự

Ra: Không

6.5. Một số ví dụ

Ví dụ 1: Chương trình nhận 1 ký tự từ bàn phím, sau đó cất ký tự nhận được vào 1 biến.

```
DSEG SEGMENT
        msg DB ‘Hay nhap 1 ky tu tu ban phim: $’
```

```
        kytu DB ? ; Biến chứa ký tự nhận được
DSEG ENDS
CSEG SEGMENT
        ASSUME CS: CSEG, DS: DSEG
        start: MOV AX, DSEG
        MOV DS, AX
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg ; DX chứa địa chỉ chuỗicần in
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 01 ; Hàm nhập 1 ký tự, AL chứa Ký tự
        INT 21h
        MOV kytu, AL ; Cất Ký tự vào biên
        MOV AH, 4Ch ; thoát khỏi chương trình
        INT 21h
CSEG ENDS
END start
```

Ví dụ 2: Chương trình nhận 1 ký tự từ bàn phím, sau đó in ra màn hình ký tự kế sau của ký tự nhận được.

```
DSEG SEGMENT
        msg1 DB ‘Hay nhap 1 ky tu tu ban phim: $’
        msg2 DB 10,13,‘Ky tu ke sau ky tu nhan duoc la: $’
        kytu DB ?
DSEG ENDS
CSEG SEGMENT
ASSUME CS: CSEG, DS: DSEG
        start: MOV AX, DSEG
        MOV DS, AX
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg1 ; DX chứa địa chỉ chuỗi msg1
        INT 21h ; gọi ngắt thực hiện
```

```
        MOV AH, 01 ; Hàm nhập 1 ký tự, AL chứa Ký tự
        INT 21h
        MOV kytu, AL ; cất ký tự vào biến
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg2 ; DX chứa địa chỉ chuỗi msg2
        INT 21h ; gọi ngắt thực hiện
        MOV DL, kytu
        ADD DL, 1 ; Ký tự kế sau
        MOV AH, 02 ; In ký tự
        INT 21h
        MOV AH, 4Ch ; thoát khỏi chương trình
        INT 21h
CSEG ENDS
END start
```

Ví dụ 3: Chương trình nhận chuỗi A và B từ bàn phím, sau đó in ra màn hình B trước, chuỗi A sau.

```
DSEG SEGMENT
        msg1 DB 'Hay nhap vao chuoi A: $'
        msg2 DB 10,13, 'Hay nhap vao chuoi B: $'
        msg3 DB 10,13,'Cac chuoi nhan duoc la: $'
        max1 DB 30
        len1 DB ?
        strA DB 31 DUP('$')
        max2 DB 30
        len2 DB ?
        strB DB 31 DUP('$')
DSEG ENDS
CSEG SEGMENT
ASSUME CS: CSEG, DS: DSEG
        start: MOV AX, DSEG
```

```
        MOV DS, AX
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg1 ; DX chứa địa chỉ chuỗi msg1
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 0Ah ; Nhập chuỗi A
        LEA DX, max1
        INT 21h
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg2 ; DX chứa địa chỉ chuỗi msg2
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 0Ah ; Nhập chuỗi B
        LEA DX, max2
        INT 21h
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg3 ; DX chứa địa chỉ chuỗi msg3
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, strB ; DX chứa địa chỉ chuỗi B
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, strA ; DX chứa địa chỉ chuỗi A
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 4Ch ; thoát khỏi chương trình
        INT 21h
CSEG ENDS
END start
```

Ví dụ 4: Chương trình ghi chuỗi "Welcome to Assembly!" vào tập tin mới có tên là W2A.TXT.

```
DSEG SEGMENT
        msg1 DB 'Da ghi xong file.$'
```

```
        msg2 DB ‘Welcome to Assembly!’
        tenfile DB ‘D:\W2A.TXT’,0
        thefile DW ?
DSEG ENDS
CSEG SEGMENT
ASSUME CS: CSEG, DS: DSEG
        start: MOV AX, DSEG
        MOV DS, AX
        MOV AH, 3Ch ; Tạo file mới
        LEA DX, tenfile
        MOV CX, 0 ; thuộc tính bình thường
        INT 21h
        MOV thefile, BX ; cất thẻ file
        MOV AH, 40h ; ghi file
        MOV BX, thefile
        MOV CX, 20 ; số byte cần ghi
        LEA DX, msg2 ; chuỗi cần ghi
        INT 21h
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg1 ; DX chứa địa chỉ chuỗi msg1
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 3Eh ; Đóng file
        MOV BX, thefile
        INT 21h
        MOV AH, 4Ch ; thoát khỏi chương trình
        INT 21h
CSEG ENDS
END start
```

Ví dụ 5: Chương trình xóa tập tin trên đĩa. Tên file được nhập từ bàn phím khi chạy chương trình.

```
DSEG SEGMENT
        msg1 DB ‘Ten file can xoa: $’
        msg2 DB ‘Da xoa xong file.$’
        max DB 30
        len DB ?
        tenf DB 31 DUP(?)
DSEG ENDS
CSEG SEGMENT
ASSUME CS: CSEG, DS: DSEG
        start: MOV AX, DSEG
        MOV DS, AX
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg1 ; DX chứa địa chỉ chuỗi msg1
        INT 21h
        MOV AH, 0Ah ; Đọc tên file
        LEA DX, max
        INT 21h
        XOR CX, CX ; đoạn chèn giá trị 0 vào cuối tên
        MOV CL, len ; tập tin
        LEA SI, tenf
        ADD SI, CX
        MOV [SI], CH ; CH = 0
        MOV AH, 41h ; Tạo file mới
        LEA DX, tenfile
        INT 21h
        MOV AH, 09 ; Hàm 9 in chuỗi ký tự ra màn hình
        LEA DX, msg1 ; DX chứa địa chỉ chuỗi msg1
        INT 21h ; gọi ngắt thực hiện
        MOV AH, 4Ch ; thoát khỏi chương trình
        INT 21h
```

```
CSEG ENDS
END start
```

6.6 Hợp ngữ và một số ngôn ngữ bậc cao:

6.6.1 Ngôn ngữ C và hợp ngữ

Để kết hợp giữa chương trình viết bằng ngôn ngữ C và hợp ngữ có thể thực hiện các cách sau:

a/ Viết chen các lệnh hợp ngữ trong chương trình C (Inline-Assembly):

Cú pháp: asm [nhãn:] lệnh toán hạng

b/ Viết chen khối lệnh hợp ngữ vào chương trình C

Cú pháp:
```
asm {
        [nhãn1:]  lệnh1   toán hạng
        [nhãn2]   lệnh2   toán hạng
        ……
       [nhãnn]    lệnhn   toán hạng
       }
```

c/ Viết chương trình với các modun tách biệt

Chương trình được viết với 2 chương trình nguồn tách biệt sau đó dịch và liên kết để trở thành 1 file thi hành .exe

Giả sử ta viết xong 2 chương trình nguồn có tên viduc.c và vidua.asm. Ta có thể dịch và liên kết bằng lệnh:

```
tcc viduc vidua.asm
```

Lệnh trên sẽ thực hiện:

- Trình biên dịch C dịch file viduc.c thành file viduc.asm
- Trình biên dịch C sẽ gọi trình biên dịch Asembler dịch file vidua.asm thành file vidua.obj và file viduc.asm thành viduc.obj
- Trình biên dịch C sẽ gọi trình liên kết để liên kết 2 file để thành file thi hành .exe.

6.6.2 Ngôn ngữ Pascal và hợp ngữ

Tương tự như ngôn ngữ C, ngôn ngữ Pascal cũng có thể kết hợp với hợp ngữ bằng các cách sau:

a/ Viết chen các lệnh hợp ngữ trong chương trình Pascal (Inline-Assembly):

Cú pháp: asm

[nhãn 1] lệnh1 tóan hạng

[nhãn 2] lệnh2 toán hạng

……

[nhãn n] lệnhn tóan hạng

End;

b/ Viết chương trình trên các file tách biệt: Có thể viết chương trình trên các file tách biệt sau đó dịch và liên kết chúng để tạo file thi hành .exe

TÀI LIỆU THAM KHẢO

[1] Ths. Phạm Văn Cường, *Lập trình hệ thống và điều khiển thiết bị*, Học viện Bưu chính Viễn thông, 2006

[2] Nguyễn Mạnh Giang, *Lập trình bằng ngôn ngữ Assembly,* NXB Giáo dục 2004

[3] Nguyễn Hứa Duy Khang - Trần Hữu Danh, *Giáo trình lập trình hệ thống*, Trường Đại học Cần Thơ, 2008

[4] Phan Văn Nghĩa, *Giáo trình Kiến trúc máy tính và hợp ngữ*, Trường Đại học Đà lạt, 2002

[5] Tống Văn On - Hoàng Đức Hải, *Hợp ngữ và lập trình ứng dụng*, NXB Giáo Dục, 2001

[6] Lê Hải Sâm, *Kiến trúc máy tính và vi xử lý*, NXB Giáo dục 2004

[7] Nguyễn Minh Tuấn, *Cấu trúc máy tính và hợp ngữ*, NXB Đà Nẵng, 1977

[8] Nguyễn Quang Tân, *Lập trình hợp ngữ*, NXB Thống Kê, 1977

[9]A.S.Tanenbaum - *Structured Computer Organization* - Fourth Edition
Prentice Hall International, 1999
Bản dịch: *Giáo trình Cấu trúc máy tính* - Tống Văn On - NXB Giáo dục, 2000

[10] Paul A.Carter - *PC Assembly Language*, 2004

MỤC LỤC

Chương 1. TỔNG QUAN VỀ MÁY TÍNH ĐIỆN TỬ 1
1.1 Giới thiệu 1
1.1.1 Lịch sử phát triển 1
1.1.2 Phân loại máy tính 4
1.2 Tổ chức hệ thống máy tính 4
1.2.1 Sơ đồ cấu trúc chung 4
1.2.2 Chức năng cơ bản của các bộ phận 6
1.2.3 Quá trình thực hiện lệnh 9
1.3 Giới thiệu tổ chức bên trong của máy vi tính 10
1.3.1 Sơ đồ 10
1.3.2 Cấu hình và các đặc trưng kỹ thuật 11
Chương 2. CƠ SỞ SỐ VÀ LOGIC 12
2.1. Các hệ thống số dùng trong máy tính điện tử 12
2.2 Biểu diễn thông tin trong máy tính điện tử 13
2.3.8 Các phép tính trên hệ nhị phân 19
2.4 Cơ sở logic trong máy tính 24
2.4.1 Hàm Boolean và bảng sự thật 24
2.4.2 Cực tiểu hàm Boolean 29
Chương 3: TỔ CHỨC BỘ XỬ LÝ TRUNG TÂM 34
3.1 Sơ đồ khối tổng quát của bộ xử lý trung tâm (CPU - Central Processing Unit) 34
3.2 ALU (Arithmetic Logic Unit): 35
2.1.4 Tổ chức thanh ghi 36
3.4 Đơn vị điều khiển 40
3.4.1. Điều khiển bằng mạch điện tử 40
3.4.2. Điều khiển bằng vi chương trình: 41
3.5. Các bộ vi xử lý cơ bản của Intel 42
3.5.1 Một số đặc điểm của các vi xử lý 42
3.5.2 Cách ghi dữ liệu lên bộ nhớ 46
3.5.3 Sơ đồ tổ chức của 8088 và 80286 46
3.6 Kiến trúc CISC – RISC và Pentium của Intel 57
3.6.1 Hệ vi xử lý SISC (Complex Instruction Set Computer) 57
3.7 Tập lệnh hợp ngữ của họ vi xử lý 61
3.8 Đa xử lý và các kiến trúc tiên tiến 62
3.8.1 Cơ chế đường ống (pipeline) 62
3.8.2 Cơ chế siêu phân luồng (hyper-threading) 63
3.8.3 Kỹ thuật siêu phân luồng và đa luồng song song (SMT -Simultaneous Multi-Threading) 65
Chương 4. BỘ NHỚ CHÍNH 71
4.1 Các khái niệm cơ bản 71
4.3 Phân loại bộ nhớ và cấu trúc mạch nhớ 72
4.3.1 Phân loại bộ nhớ 72
4.3.2 Cấu trúc mạch nhớ 75

4.4 Thiết kê modul nhớ 83
Chương 5. QUÁ TRÌNH VÀO RA VÀ THIẾT BỊ NGOẠI VI 85
5.1 Quá trình vào ra....85
5.2 Tổ chức điều khiển vào/ra trong máy tính 87
5.3 Các phương pháp vào ra số liệu 87
5.3.1 Vào ra điều khiển bằng ngắt 88
5.3.2 Vào ra điều khiển theo DMA 89
5.4 Các mạch ghép nối và điều khiển thiết bị vào ra 90
5.4.1 Cổng song song (Parallel) 90
5.4.2 Cổng nối tiếp (Serial) 91
5.5 Các thiết bị ngoại vi thông dụng trong hệ máy vi tính....92
5.5.1 Bộ nhớ ngoài: 92
Chương 6. HỢP NGỮ 108
6.1. Hợp ngữ và các trình biên dịch hợp ngữ 16 bit 108
6.1.1 Ngôn ngữ máy và hợp ngữ....108
6.1.2 Hợp ngữ MASM (Hợp ngữ của CPU 8086) 109
6.1.3 Cấu trúc của một hàng lệnh....110
6.1.3 Toán hạng và toán tử....119
6.2 Cấu trúc của chương trình hợp ngữ....123
6.2.1 Cấu trúc chung 123
6.2.2 Tập tin thi hành dạng .COM 125
6.2.3 Tập tin thi hành dạng .EXE....126
6.2.4 Ví dụ....127
6.2.5 Cách tạo 1 chương trình bằng hợp ngữ....129
6.3. Một số lệnh của hợp ngữ 16 bit....129
6.3.1 Toán hạng....129
6.3.2. Nhóm lệnh sao chép dữ liệu, địa chỉ:....131
6.3.3. Nhóm lệnh tính toán số học. 132
6.3.4. Nhóm lệnh logic và ghi dịch 134
6.3.5. Nhóm lệnh vào ra ngoại vi. 137
6.3.6. Nhóm lệnh hệ thống....138
6.3.7 Nhóm lệnh nhảy (Chuyển điều khiển) 138
6.3.8 Nhóm lệnh lặp....144
6.4. Một số ngắt của DOS và BIOS 147
6.4.1. Tổng quan về ngắt....147
6.4.2. Sử dụng ngắt trong hợp ngữ....148
6.4.3. Ngắt MS-DOS....148
6.4.4 Ngắt của BIOS 154
6.5. Một số ví dụ 155
6.6 Hợp ngữ và một số ngôn ngữ bậc cao:....161
6.6.1 Ngôn ngữ C và hợp ngữ....161
6.6.2 Ngôn ngữ Pascal và hợp ngữ 161